RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 – Fundamentals of Data Science Lab
Laboratory Record Note Book

Name :	…….
Year / Branch / Section :	…….
University Register No. :	…….
College Roll No. :	…….
Semester :	…….
Academic Year :	…….

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 – Fundamentals of Data Science Lab

NAME	
ROLL NO	
YEAR	
SEC	

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ……

Academic Year: …… Semester: …… Branch: ……

Register No. | |

Certified that this is the bonafide record of work done by the above student in the…… Laboratory during the academic year 2026 – 2027.

Signature of Faculty in-charge

Submitted for the Practical Examination held on ……

Internal Examiner External Examiner

INDEX

Sl.NO	Experiment	Date	GitHub
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	29/07/26	
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	29/07/26	
1.c	Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.	05/08/26	
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	05/08/26	
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	12/08/26	
3.a	Conduct an experiment to show data visualization using line plot	19/08/26	
3.b	Conduct an experiment to show data visualization using bar chart	19/08/26	
3.c	Conduct an experiment to show data visualization using pie chart	19/08/26	
4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	26/08/26	
5	Data Collection and Initial Exploration	16/09/26	
6	Time Series Analysis of Website Traffic	16/09/26	
7	Random Sampling and Sampling	23/09/26	

	Distribution		
8	Conduct Z test for the Data given using Python	23/09/26	
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	23/09/26	
10	Experiment to understand array function in Data science.	23/09/26	
11	Experiment to understand pandas library use cases in Data science.	30/09/26	
12	Experiment to detect outliers in a given data set.	30/09/26	
13	Experiment to handle missing values and inappropriate data in a given data set.	30/09/26	
14	Experiment to understand feature scaling	30/09/26	
15	Experiment to understand the data preprocessing in Data science	30/09/26	
16	Experiment to understand EDA-Quantitative and Qualitative analysis.	07/10/26	
17	Experiment to understand Linear Regression for a given data set.	07/10/26	
18	Experiment to understand KNN algorithm for a given data set.	07/10/26	
19	Experiment to understand Logistic Regression for a given data set.	07/10/26	
20	Experiment to understand K-means clustering algorithm for a given data set.	07/10/26	

Experiment No:1.a
Analyze the trend of data science job postings over The last decade

AIM:

To Analyze the trend of data science job postings over the last Decade

DESCRIPTION:

Use web scraping (e.g., BeautifulSoup) or APIs (e.g., LinkedIn API) to gather data on the number of data science job postings each year.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    'Year': list(range(2015, 2026)),
    'Job Postings': [180, 320, 480, 620, 850, 1250, 1650, 2150,
2800, 3500, 4300]
}

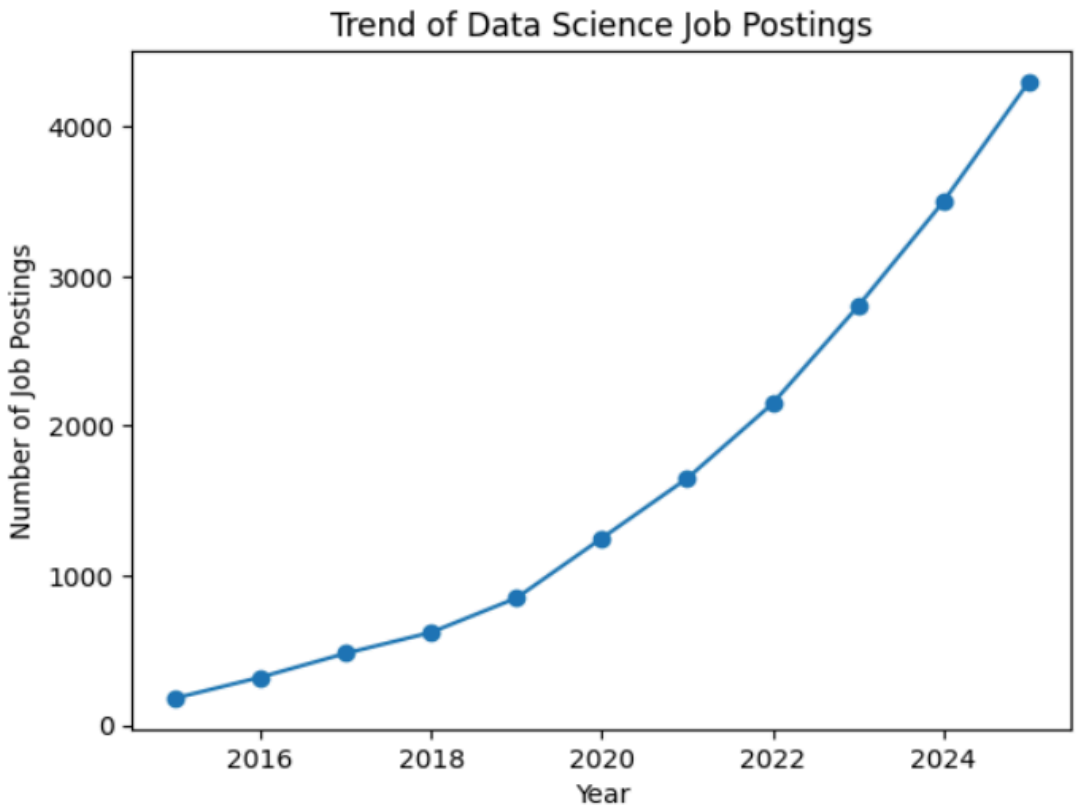
df = pd.DataFrame(data)

plt.plot(df['Year'], df['Job Postings'], marker='o')

plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')

plt.show()
```

OUTPUT:



RESULT:

The graph shows that data science job posting increases from 2015 to 2026 .The highest number of job postings was in 2026

Experiment No:1.b

Analyze and visualize the distribution of various data science roles (DataAnalyst, Data Engineer, Data Scientist, etc.) from a dataset.

AIM:

To Analyze and visualize the distribution of various data science roles from a dataset.

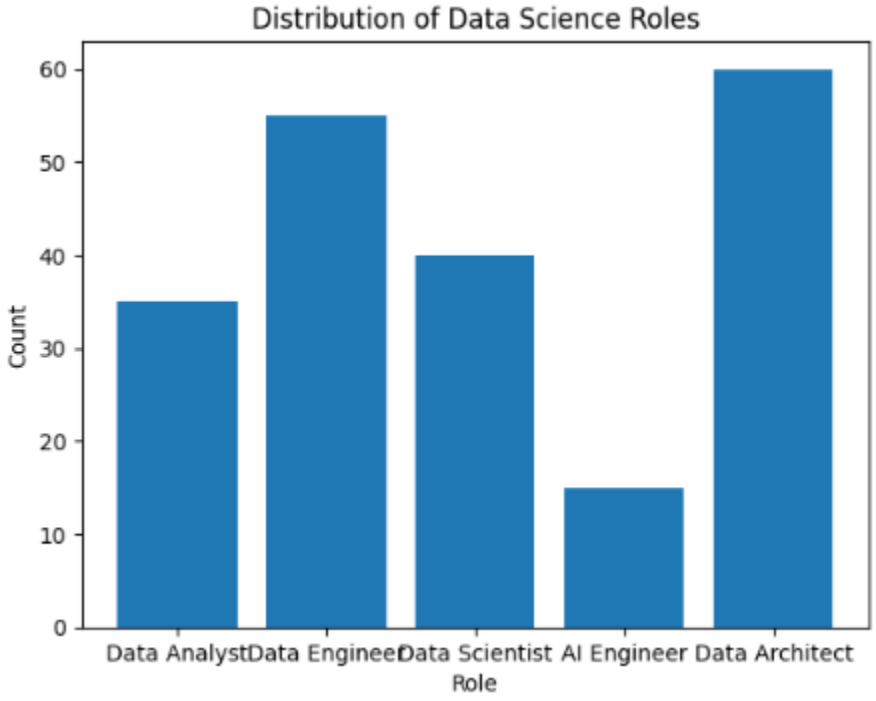
DESCRIPTION:

Use a dataset of job postings and categorize them into different roles. Visualize the distribution using pie charts or bar plots.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist',
'AI Engineer',
'Data Architect']
counts = [35, 55, 40, 15, 60]
plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```

OUTPUT:



RESULT:

The bar chart shows that Data Architect has the highest count (59), followed by Data Engineer (55), Data Scientist (40), Data Analyst (35), and AI Engineer (15). Therefore, Data Architect is the most common role, while AI Engineer is the least common.

Experiment No:1.c
Conduct an experiment to differentiate Structured , Un-structured and Semistructured data based on data sets given.

AIM:

To Conduct an experiment to differentiate Structured , Un-structured and Semistructured data based on data sets given.

DESCRIPTION:

Create small datasets for each type and explain their characteristics.

CODE:

```
structured_data = pd.DataFrame({
    'ID': [101, 102, 103],
    'Name': ['David', 'Emma', 'Frank'],
    'Age': [22, 28, 40]
})
print("Structured Data:\n", structured_data)

unstructured_data = "Machine learning helps computers learn from data."
print("\nUnstructured Data:\n", unstructured_data)

semi_structured_data = {
    'ID': 101,
    'Name': 'David',
    'Attributes': {
        'Height': 178,
        'Weight': 75
    }
```

```
}
print("\nSemi-structured Data:\n", semi_structured_data)
```

OUTPUT:

```
Structured Data:
     ID   Name  Age
0  101  David   22
1  102   Emma   28
2  103  Frank   40

Unstructured Data:
 Machine learning helps computers learn from data.

Semi-structured Data:
 {'ID': 101, 'Name': 'David', 'Attributes': {'Height': 178, 'Weight': 75}}
```

[

RESULT:

Structured data is the easiest to query via sql

semi structured is self describing but schema flexible.

unstructured data carries no explicit schema

Experiment No:1.d
Conduct an experiment to encrypt and decrypt given sensitive data.

AIM:

To Conduct an experiment to encrypt and decrypt given sensitive data.

DESCRIPTION:

Use the cryptography library to encrypt and decrypt a piece of data.

CODE:

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"")
token
b'...'
f.decrypt(token)
b'fundamental of data science'
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"fundamental of data science"
cipher_text = cipher_suite.encrypt(plain_text)
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

OUTPUT:

```
Original Data: b'fundamental of data science'
Encrypted Data:
b'gAAAAABqhGX8OLUp-yzVPjwSzPanb9xYdER9UBcN1fPXXBZV1SOd0Rn-ndBY
mx-SNEWz6AgMvMl0tG8AOiBPlUVVSF_odKDiZklvH894Tv0RwOvwP2coDR8='
Decrypted Data: b'fundamental of data science'
```

RESULT:

The fernet Symmetric -key Algorithm successfully encrypted back original data text,verifying both any tumering with the cipherset would cause decryption to fail.

Experiment No:2
Upload and Analyze the data set given in csv format and performance data preprocessing and visualization.

AIM:

To Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

DESCRIPTION:

Use sample data set sales-data.csv.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the data into a pandas DataFrame
file_path = '/content/sales_data.csv'
df = pd.read_csv(file_path)

# Display the first few rows
print(df.head())

# Check for missing values
print(df.isnull().sum())

# Fill missing Sales values with the mean
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())

# Drop rows with missing Product, Quantity, or Region
```

```
df.dropna(subset=['Product', 'Quantity', 'Region'],
inplace=True)

# Summary statistics
print(df.describe())

# Group by Product and calculate total Sales and Quantity
product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()

print(product_summary)

# Bar plot of total sales by product
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'],
product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

# Convert Date column to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Line plot of sales over time
sales_over_time = df.groupby('Date').agg({
    'Sales': 'sum'
}).reset_index()
```

```
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'],
sales_over_time['Sales'], marker='o')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Pivot table to analyze sales by region and product
pivot_table = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Product',
    aggfunc=np.sum,
    fill_value=0
)

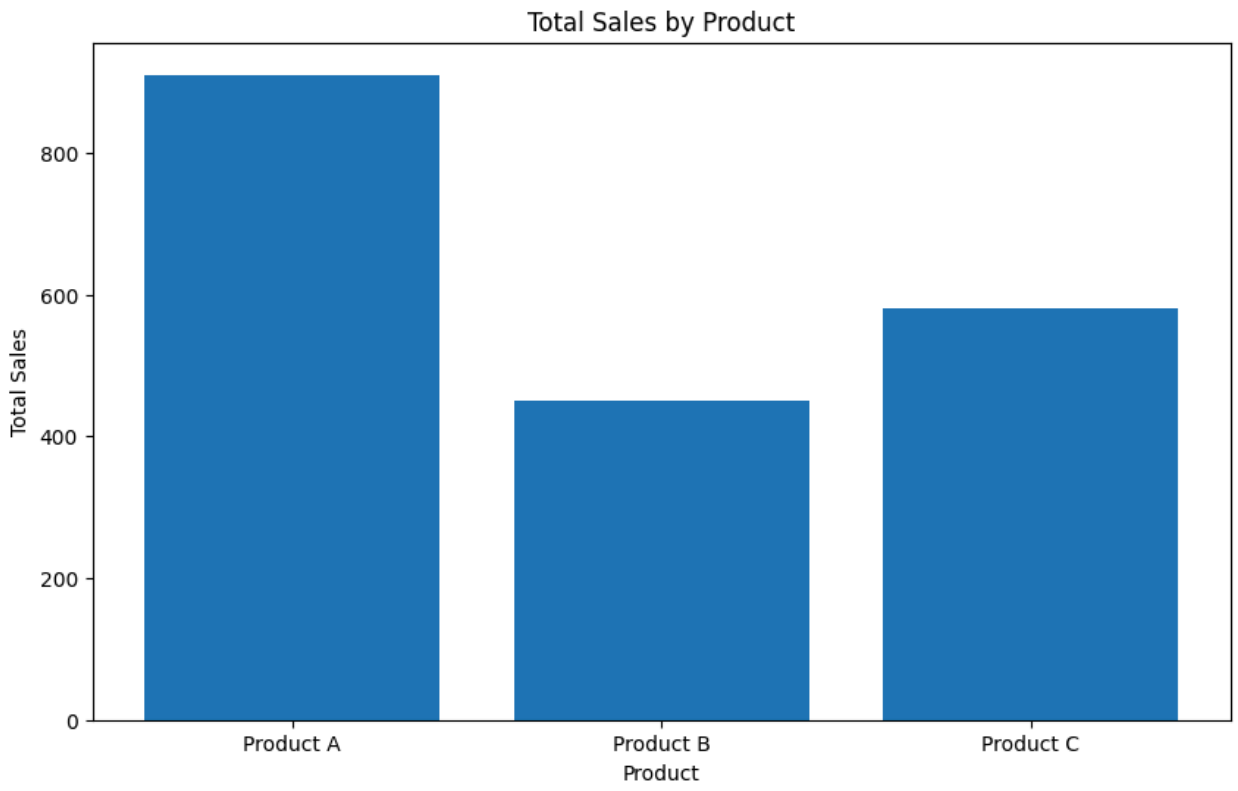
print(pivot_table)

# Correlation matrix for numeric columns only
correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)

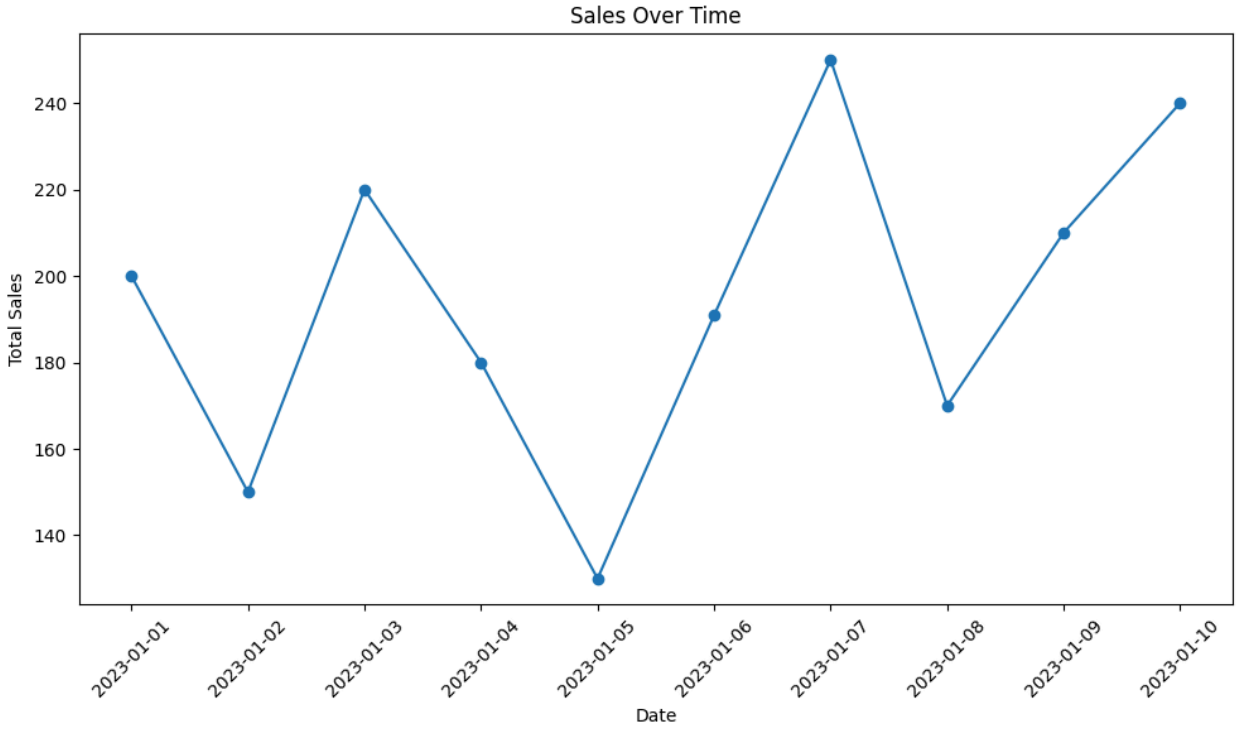
# Heatmap of the correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True,
cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

OUTPUT:

```
        Date    Product  Sales  Quantity Region
0  2023-01-01  Product A  200.0       4.0  North
1  2023-01-02  Product B  150.0       3.0  South
2  2023-01-03  Product A  220.0       5.0  North
3  2023-01-04  Product C  180.0       2.0   East
4  2023-01-05  Product B  130.0       2.0  South
Date        0
Product     0
Sales       1
Quantity    1
Region      1
dtype: int64
            Sales   Quantity
count   10.000000  10.000000
mean   194.090909   3.800000
std     38.054743   1.316561
min    130.000000   2.000000
25%    172.500000   3.000000
50%    195.454545   4.000000
75%    217.500000   4.750000
max    250.000000   6.000000
     Product       Sales  Quantity
0  Product A  910.000000      19.0
1  Product B  450.000000       8.0
2  Product C  580.909091      11.0
```




```
/tmp/ipykernel_1895/1755118830.py:59
```



```
: FutureWarning: The provided callable <function sum at
0x7f59fe584ea0> is currently using DataFrameGroupBy.sum. In a
future version of pandas, the provided callable will be used
directly. To keep current behavior pass the string "sum"
instead.
  pivot_table = df.pivot_table(
```

Product	Product A	Product B	Product C
Region			
East	0.0	0.0	580.909091
North	910.0	0.0	0.000000
South	0.0	450.0	0.000000

	Sales	Quantity
Sales	1.000000	0.838703
Quantity	0.838703	1.000000



RESULT:

Thus, the sales dataset was successfully uploaded and analyzed using Pandas. Data preprocessing was performed by handling missing values, and data visualization was carried out using a bar chart, line chart, and correlation heatmap. Product-wise sales and quantity totals were calculated, and a pivot table was created to analyze sales by region and product.

Experiment No:3.a
Conduct an experiment to show data visualization Using line plot

AIM:

To Conduct an experiment to show data visualization using line plot

DESCRIPTION:

Take any sample data either through csv file or data fetched directly through Code.

CODE:

```
import matplotlib.pyplot as cricket
Overs = list(range(5, 51, 5))
Australia_Score = [35, 68, 105, 145, 180, 215, 250, 285,
325, 365]
England_Score = [28, 62, 98, 130, 160, 195, 225, 255, 290,
330]
cricket.plot(Overs, Australia_Score)
cricket.plot(Overs, England_Score)

cricket.title("AUSTRALIA Vs ENGLAND")
cricket.xlabel("Overs")
cricket.ylabel("Score")

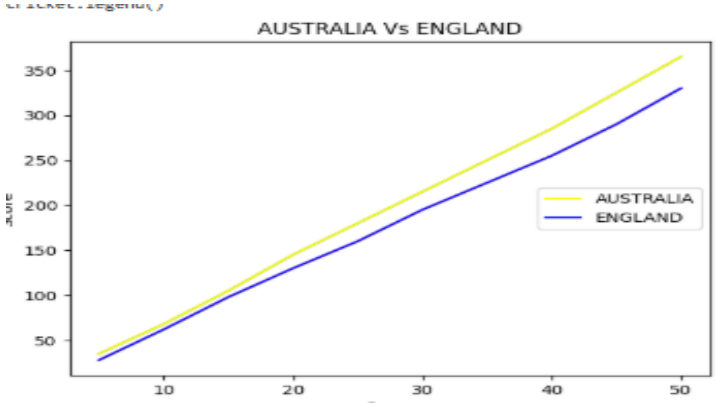
cricket.legend()

cricket.plot(Overs, Australia_Score, color="yellow",
label="AUSTRALIA")
cricket.plot(Overs, England_Score, color="blue",
label="ENGLAND")
```

```
cricket.legend(loc="center right")

cricket.show()
```

OUTPUT:



RESULT:

The data visualization of cricket match for Australia and England was successfully performed using line plot.

.

Experiment No:3.b
Conduct an experiment to show data visualization using Bar chart

AIM:

To Conduct an experiment to show data visualization using bar chart

DESCRIPTION:

Take any sample data either through csv file or data fetched directly through Code.

CODE:

```
import matplotlib.pyplot as hscmark
import numpy as np

Names = ['ARUN', 'RAHUL', 'MEENA', 'DIVYA', 'KARTHIK',
'ANU']

xaxis = np.arange(len(Names))

Percentage_hsc = [88, 95, 82, 91, 67, 78]

hscmark.bar(Names, Percentage_hsc)

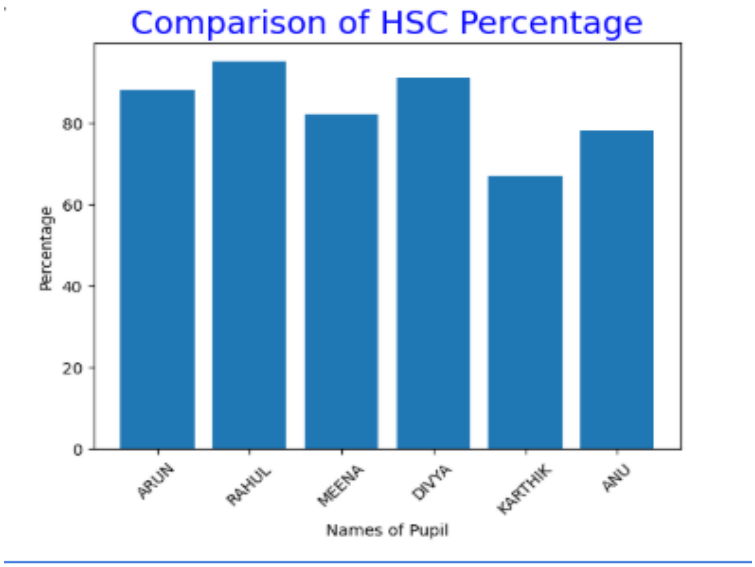
hscmark.xticks(xaxis, Names, rotation=45)

hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")

hscmark.title("Comparison of HSC Percentage",
fontsize=20, color="blue")
```

```
hscmark.show()
```

OUTPUT:



RESULT:

The data visualization of hscmark was successfully performed using a barchart . the name of different student were represented clearly using bars

Experiment No:3.c
Conduct an experiment to show data visualization using pie chart

AIM:

To Conduct an experiment to show data visualization using pie chart

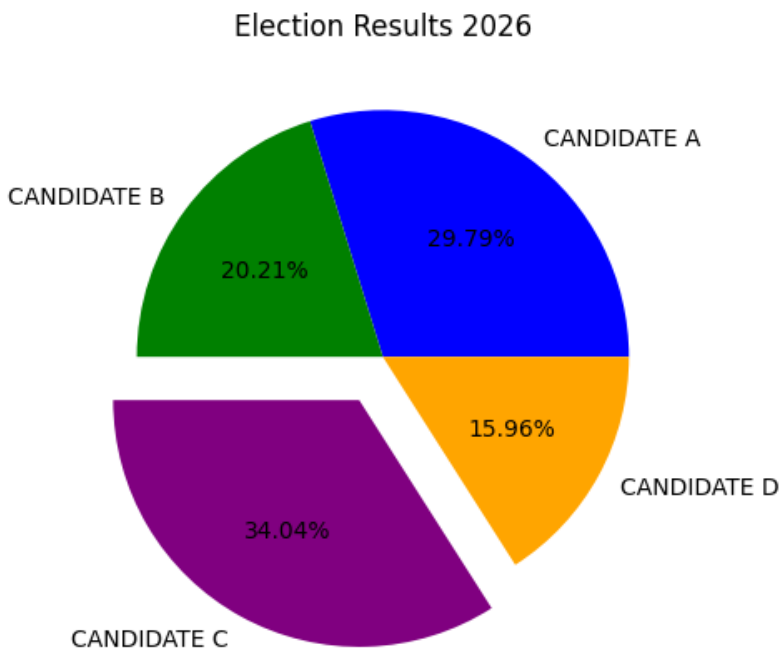
.DESCRIPTION:

Take any sample data either through csv file or data fetched directly through code.

CODE:

```
import matplotlib.pyplot as election
# Election data
labels = ['CANDIDATE A', 'CANDIDATE B', 'CANDIDATE C',
'CANDIDATE D']
Votes = [280, 190, 320, 150]
colors = ['blue', 'green', 'purple', 'orange']
explode = (0, 0, 0.2, 0)
# Plotting the pie chart
election.pie(Votes, labels=labels, colors=colors,
             explode=explode, autopct='%0.2f%%')
election.title('Election Results 2026')
election.show()
```

OUTPUT:



RESULT:

The data visualization of election result was successfully performed using pie chart.

Experiment No:4
To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

AIM:
To Count the frequency of occurrence of a word in a body of text is often needed during text processing

DESCRPTION:

Import the word_tokenize function and gutenberg.

CODE:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg

# Download the required resources
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')

# Load text from the available Gutenberg corpus
sample = gutenberg.raw("shakespeare-hamlet.txt")

# Tokenize the sample text
token = word_tokenize(sample)

# Create a list of the first 50 tokens
wlist = []

for i in range(50):
```

```
    wlist.append(token[i])

# Calculate the frequency of each word
wordfreq = [wlist.count(w) for w in wlist]

# Print the word-frequency pairs
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

OUTPUT

```
[nltk_data] Downloading package gutenberg to
/root/nltk_data...
[nltk_data]   Package gutenberg is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
/root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!

Pairs
[('[', 1), ('The', 1), ('Tragedie', 1), ('of', 1), ('Hamlet',
1), ('by', 1), ('William', 1), ('Shakespeare', 1), ('1599',
1), (']', 1), ('Actus', 1), ('Primus', 1), ('.', 7),
('Scoena', 1), ('Prima', 1), ('.', 7), ('Enter', 1),
('Barnardo', 3), ('and', 1), ('Francisco', 1), ('two', 1),
('Centinels', 1), ('.', 7), ('Barnardo', 3), ('.', 7), ('Who',
1), ("'s", 1), ('there', 1), ('?', 2), ('Fran', 2), ('.', 7),
('Nay', 1), ('answer', 1), ('me', 1), (':', 1), ('Stand', 1),
('&', 1), ('vnfold', 1), ('your', 1), ('selfe', 1), ('Bar',
1), ('.', 7), ('Long', 1), ('liue', 1), ('the', 1), ('King',
1), ('Fran', 2), ('.', 7), ('Barnardo', 3), ('?', 2)]
```

RESULT:

Thus, the frequency of occurrence of words in a body of text was successfully calculated using NLTK. The text was tokenized using `word_tokenize()`, the first 50 tokens were stored in a list, and the frequency of each token was calculated and displayed as word-frequency pairs.

Experiment No:5
Data Collection and Initial Exploration

AIM:

To collect, load, and perform initial exploration of the diabetes dataset.

Materials Needed:

Diabetes dataset (available from sources such as UCI Machine Learning Repository or Kaggle)

Python with libraries: pandas, numpy, matplotlib, seaborn

CODE:

```
  import pandas as pd
db=pd.read_csv("/content/diabetes_prediction_dataset.csv")
print(db.head())
print(db.info())
print(db.describe())
import matplotlib.pyplot as plt
import seaborn as sns
db.hist(bins=50,figsize=(20,15))
plt.show()
sns.pairplot(db)
plt.show()
```

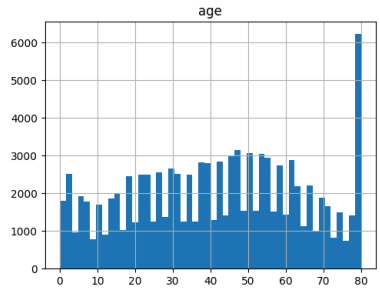
OUTPUT:

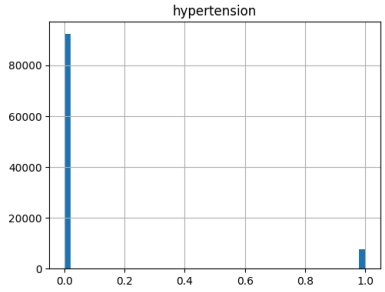
```
  gender   age  hypertension  heart_disease smoking_history    bmi  \
0  Female  80.0           0              1           never  25.19
1  Female  54.0           0              0         No Info  27.32
2    Male  28.0           0              0           never  27.32
3  Female  36.0           0              0         current  23.45
4    Male  76.0           1              1         current  20.14

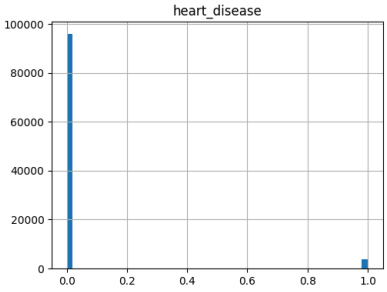
   HbA1c_level  blood_glucose_level  diabetes
0          6.6                  140         0
1          6.6                   80         0
2          5.7                  158         0
3          5.0                  155         0
4          4.8                  155         0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column               Non-Null Count   Dtype
```

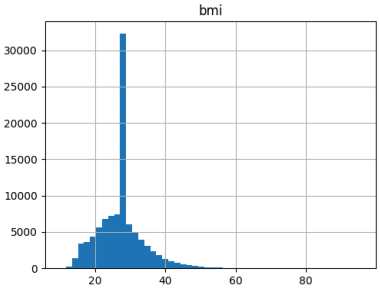
```
---  ------               --------------   -----
 0   gender               100000 non-null  object
 1   age                  100000 non-null  float64
 2   hypertension         100000 non-null  int64
 3   heart_disease        100000 non-null  int64
 4   smoking_history      100000 non-null  object
 5   bmi                  100000 non-null  float64
 6   HbA1c_level          100000 non-null  float64
 7   blood_glucose_level  100000 non-null  int64
 8   diabetes             100000 non-null  int64
dtypes: float64(3), int64(4), object(2)
memory usage: 6.9+ MB
None
                age  hypertension  heart_disease            bmi  \
count  100000.000000  100000.00000  100000.000000  100000.000000
mean       41.885856       0.07485       0.039420      27.320767
std        22.516840       0.26315       0.194593       6.636783
min         0.080000       0.00000       0.000000      10.010000
25%        24.000000       0.00000       0.000000      23.630000
50%        43.000000       0.00000       0.000000      27.320000
75%        60.000000       0.00000       0.000000      29.580000
max        80.000000       1.00000       1.000000      95.690000

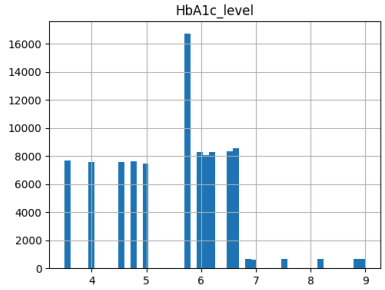
         HbA1c_level  blood_glucose_level       diabetes
count  100000.000000        100000.000000  100000.000000
mean        5.527507           138.058060       0.085000
std         1.070672            40.708136       0.278883
min         3.500000            80.000000       0.000000
25%         4.800000           100.000000       0.000000
50%         5.800000           140.000000       0.000000
75%         6.200000           159.000000       0.000000
max         9.000000           300.000000       1.000000
```

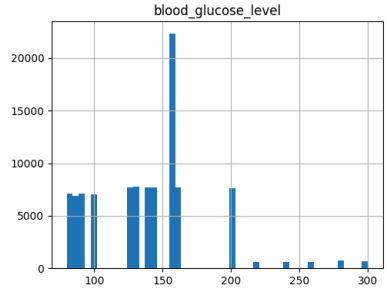

age
0
1000
2000
3000
4000
5000
6000
0
10
20
30
40
50
60
70
80

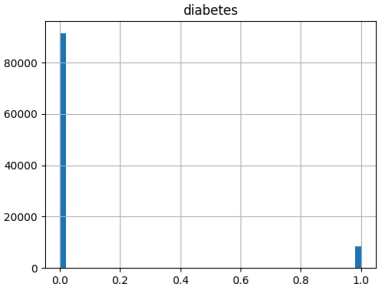

hypertension
0
20000
40000
60000
80000
0.0
0.2
0.4
0.6
0.8
1.0

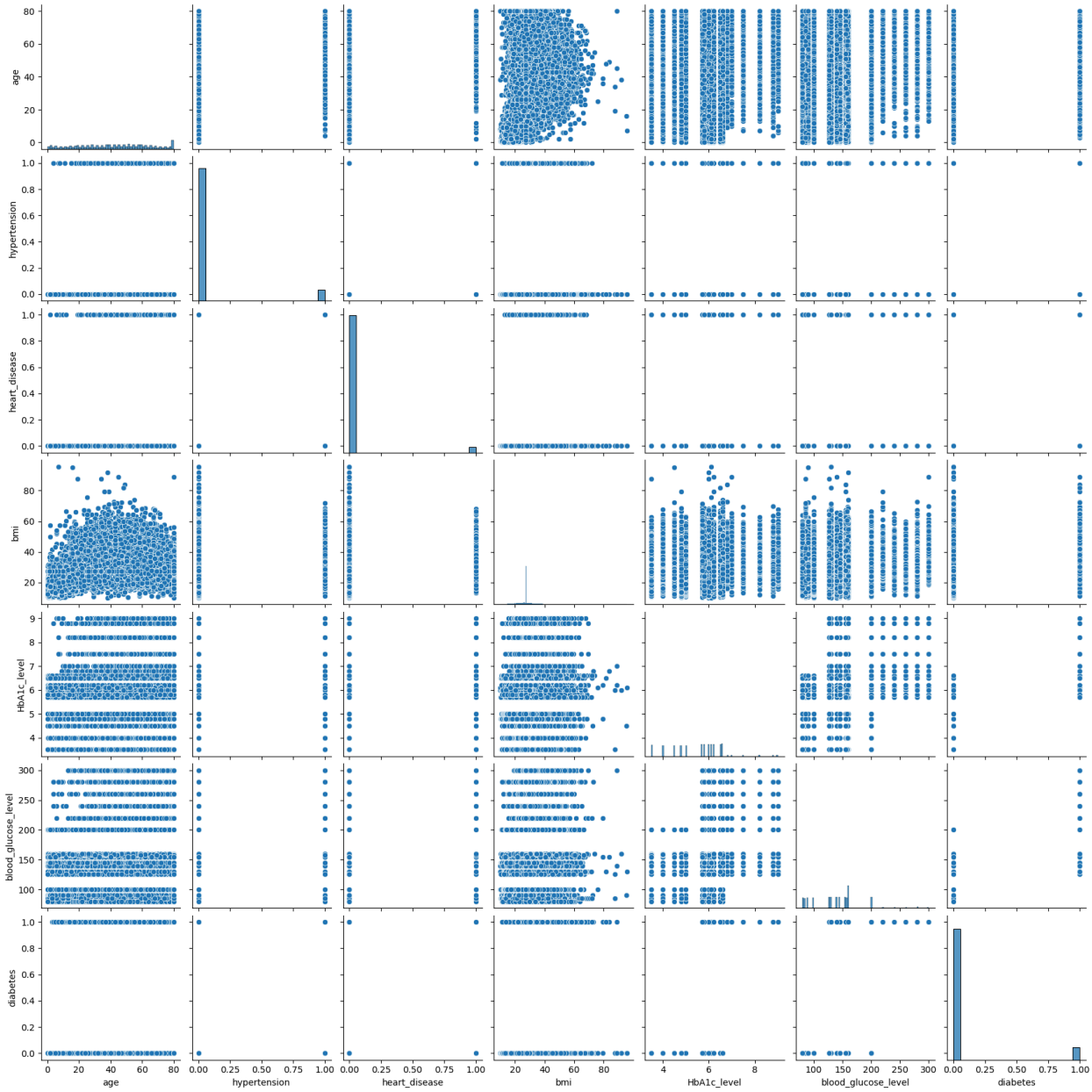

heart_disease
0
20000
40000
60000
80000
100000
0.0
0.2
0.4
0.6
0.8
1.0


bmi
0
5000
10000
15000
20000
25000
30000
20
40
60
80


HbA1c_level
0
2000
4000
6000
8000
10000
12000
14000
16000
4
5
6
7
8
9


blood_glucose_level
0
5000
10000
15000
20000
100
150
200
250
300


diabetes
0
20000
40000
60000
80000
0.0
0.2
0.4
0.6
0.8
1.0



RESULT:

The diabetes dataset was successfully loaded and initially explored.the first few records,dataset,structured and summary statistics were displayed histogram showed the distribution of each features and the pair plot showed the relationship between the variables.

Experiment No:6
Time Series Analysis of Website Traffic

Objective:

To analyze the time series data of website traffic to identify trends, seasonality, and anomalies.

Dataset: A dataset containing daily website traffic data with columns such as Date, Page_Views, Unique_Visitors, and Bounce_Rate.

Methodology:

1. Load and Clean Data:

o Import the dataset and parse the Date column as a datetime object.
o Handle missing values by interpolation.

2. Time Series Decomposition:

o Decompose the time series data of Page_Views to identify trend, seasonality, and residuals.

3. Visualization:

o Plot the time series data for Page_Views, Unique_Visitors, and Bounce_Rate.
o Use moving averages to smooth the time series and highlight trends.
o Create seasonal plots to visualize patterns over time.

4. Anomaly Detection:

o Identify and visualize anomalies in the time series data using statistical methods such as the Z-score.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# Load dataset
data = pd.read_csv("/content/website_wata_with_date.csv",
parse_dates=['Date'], index_col='Date')

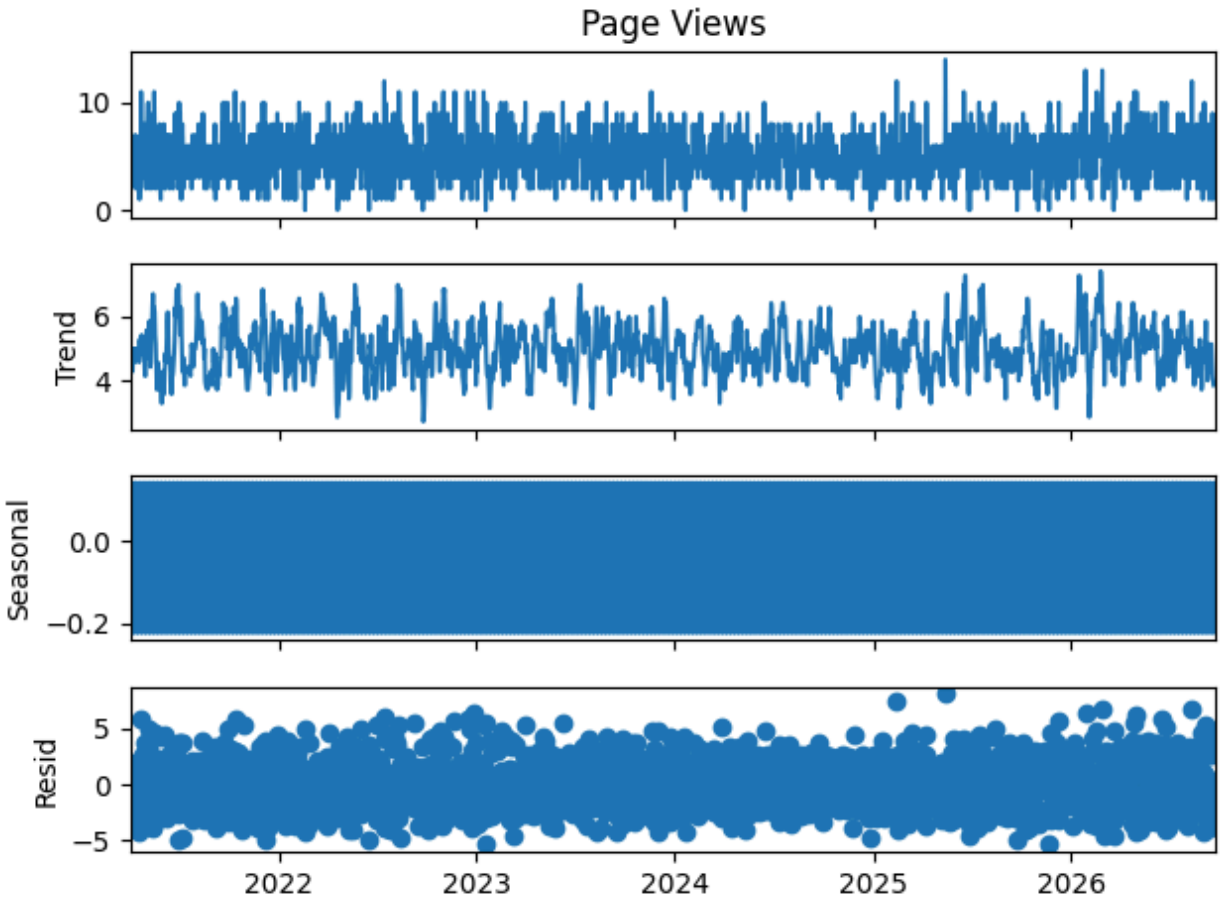
# Time Series Decomposition
decomposition = seasonal_decompose(data['Page Views'], model='additive')
decomposition.plot()
```

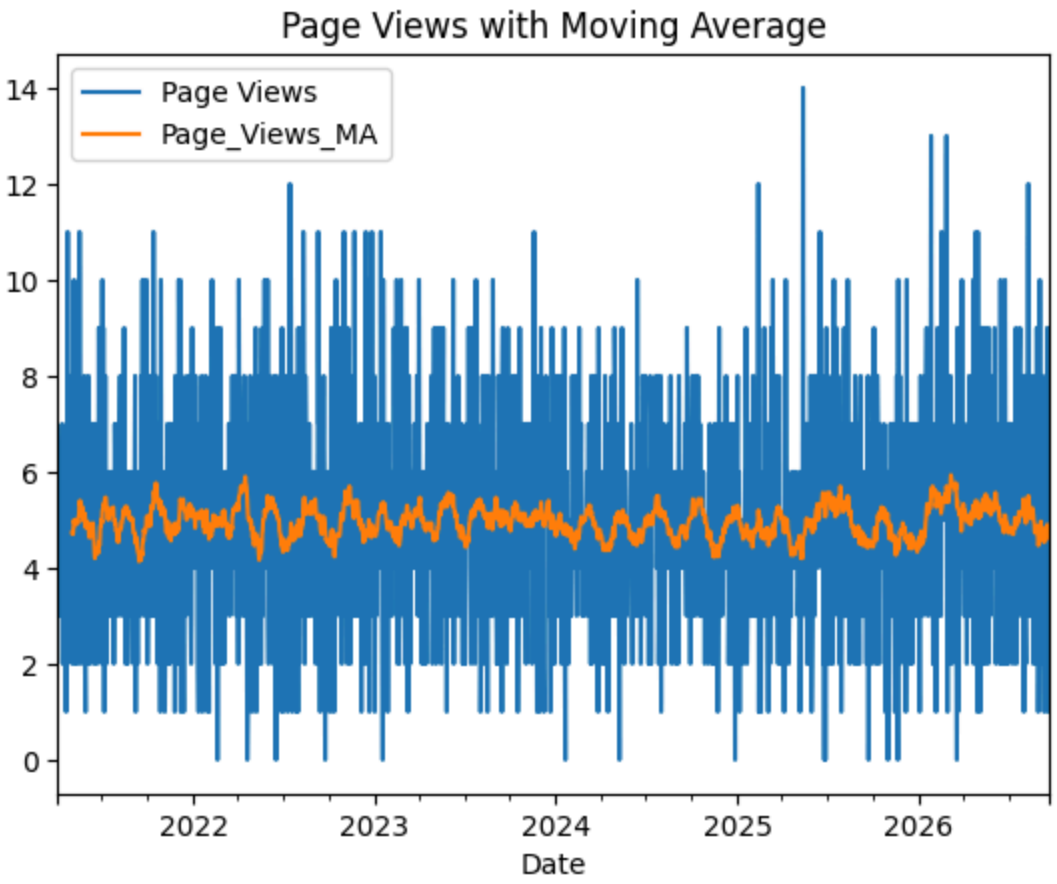
```
plt.show()

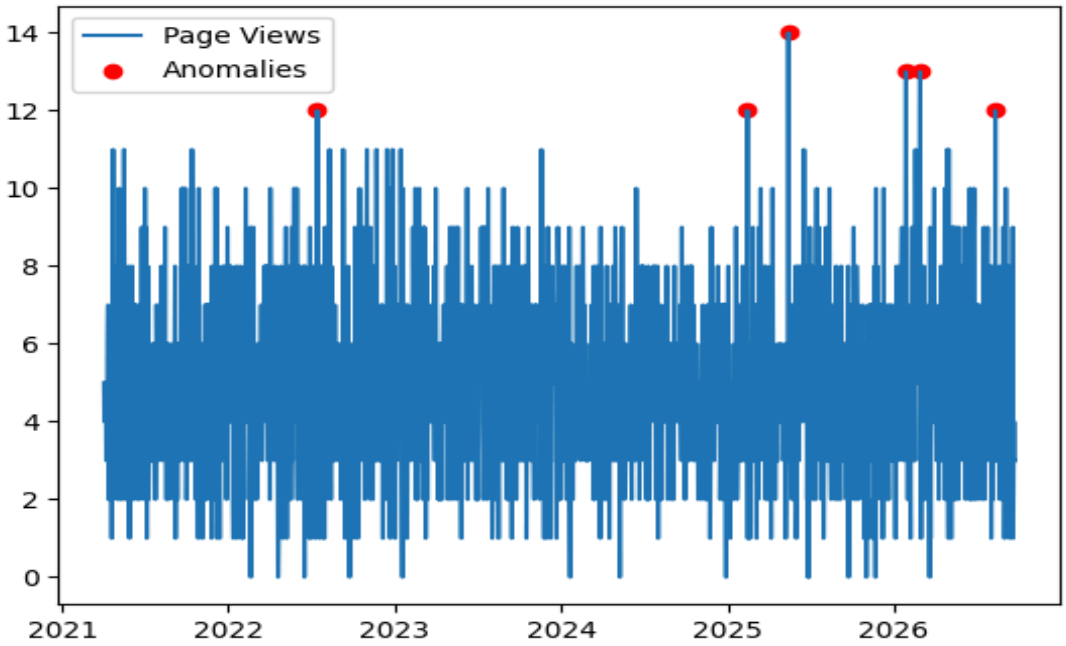
# Moving Average
data['Page_Views_MA'] = data['Page Views'].rolling(window=30).mean()
data[['Page Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()

# Anomaly Detection
data['Page_Views_Z'] = (data['Page Views'] - data['Page Views'].mean())
/data['Page Views'].std()
anomalies = data[data['Page_Views_Z'].abs() > 3]
plt.plot(data.index, data['Page Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page Views'], color='red',
label='Anomalies')
plt.legend()
plt.show()
```

OUTPUT:


Page Views
10
0
Trend
6
4
Seasonal
0.0
−0.2
Resid
5
0
−5
2022
2023
2024
2025
2026


Page Views with Moving Average
Page Views
Page_Views_MA
14
12
10
8
6
4
2
0
2022
2023
2024
2025
2026
Date


Page Views
Anomalies
14
12
10
8
6
4
2
0
2021
2022
2023
2024
2025
2026

RESULT:

The website is visualized using a line plot,showing the variations and sudden increases in page views

Experiment 7:
Random Sampling and Sampling Distribution

AIM:

To explore random sampling from a population and understand the concept of sampling distribution
using Python in Jupyter Notebook.

Steps:

1. Generate a Population:
○ Create a population of data with a specified distribution (e.g., normal distribution).
2. Random Sampling:
○ Perform random sampling from the population to create multiple samples of different sizes.
○ Compute sample statistics (mean, standard deviation, etc.) for each sample.
3. Sampling Distribution:
○ Plot histograms or density plots of sample statistics (e.g., sample means).
○ Compare the sampling distribution of the sample statistic (mean) with the population distribution.
4. Central Limit Theorem (Optional):
○ Demonstrate the Central Limit Theorem by showing that as sample size increases, the sampling distribution of the sample mean approaches a normal distribution regardless of the population distribution.

Code:

```
import numpy as np
import matplotlib.pyplot as plt



population_mean = 75
population_std = 15
population_size = 300000
population = np.random.normal(population_mean, population_std,
population_size)



sample_sizes = [20, 40, 80]
num_samples = 1500
```

```
sample_means = {}

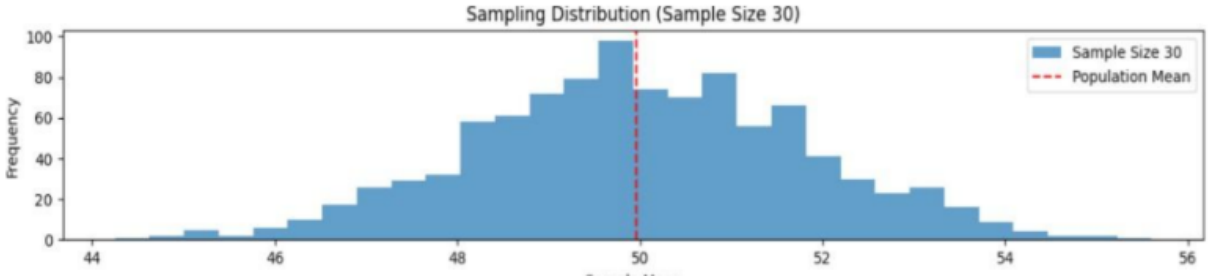
for size in sample_sizes:
    sample_means[size] = []
for _ in range(num_samples):
    sample = np.random.choice(population, size=size,
replace=False)
sample_means[size].append(np.mean(sample))

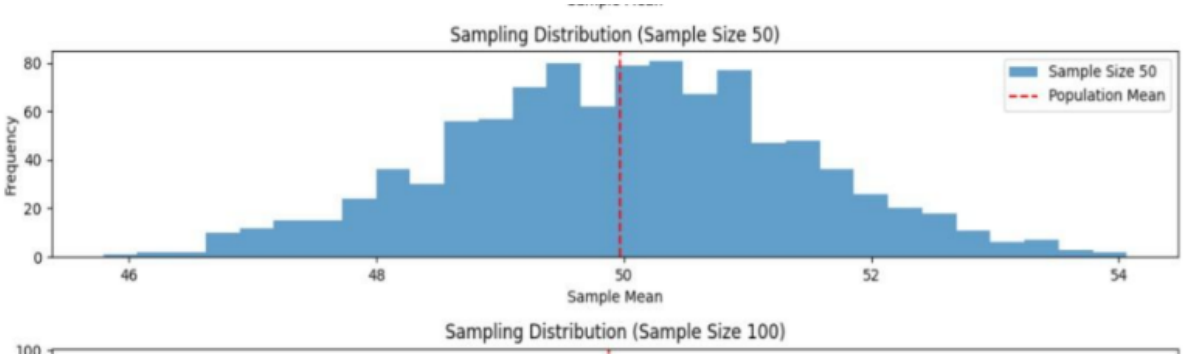
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
  plt.subplot(len(sample_sizes), 1, i+1)
  plt.hist(sample_means[size], bins=25, alpha=0.7,
label=f'Sample Size {size}')
  plt.axvline(np.mean(population), color='red',
linestyle='dashed', linewidth=1.5, label='Population Mean')
  plt.title(f'Sampling Distribution (Sample Size {size})')
  plt.xlabel('Sample Mean')
  plt.ylabel('Frequency')
  plt.legend()

plt.tight_layout()
plt.show()
```

Output:




Sampling Distribution (Sample Size 50)
Sample Size 50
Population Mean
Frequency
Sample Mean
0
20
40
60
80
46
48
50
52
54

Sampling Distribution (Sample Size 100)

Result:

The result show that as the sample size increases the sampling distribution becomes more concentrated around the population mean,demonstration the central limit theorem.

Experiment 8:

Conduct Z test for the Data Given using Python

AIM:

To test whether the average weight of a species of birds differs from 150 grams.

PROCEDURE:

1. Null Hypothesis (H0): The average weight of the birds is 150 grams.
2. Alternative Hypothesis (H1): The average weight of the birds is not 150 grams.
3. Sample: Measure the weights of 30 birds randomly selected from the population.
4. Z-Test: Conduct a Z-test to compare the sample mean to 150 grams.
5. Decision Rule: Use a significance level of α = 0.05.

CODE:

```
import numpy as np
import scipy.stats as stats


sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152,
149,151, 150, 149, 152, 151,
148, 150, 152, 149, 150,148, 153, 151, 150, 149, 152, 148, 151, 150, 153])

population_mean = 150

sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)


n = len(sample_data)

z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))



p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))


print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
```

```
print(f"P-Value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
  print("Reject the null hypothesis: The average weight is significantly
different from 150 grams.")
else:
  print("Fail to reject the null hypothesis: There is no significant
difference in average weight from 150 grams.")
```

OUTPUT:

```
Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218
Fail to reject the null hypothesis: There is no significant
difference in average weight from 150 grams.
```

RESULT:

The Z-test was conducted to determine whether the average weight of the bridge differ from 160 grams.

Experiment No:9
Experiment to understand Matplotlib library use cases in Data Science through visualization

AIM:

To understand Matplotlib library use cases in Data Science through visualization

DESCRIPTION:

Use Iris data set to understand the Matplot lib library

CODE:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

%matplotlib inline

# Load the dataset
data = pd.read_csv('/content/Iris.csv')

# Display the dataset
data

# Display dataset information
data.info()

# Statistical summary
data.describe()

# Count the number of flowers in each species
data.value_counts('Species')
```

```
# Count plot
sns.countplot(x='Species', data=data)
plt.show()

# Convert categorical data into dummy variables
dummies = pd.get_dummies(data['Species'])

# Combine dummy variables with numerical columns
FinalDataset = pd.concat(
    [dummies, data.iloc[:, [0, 1, 2, 3]]],
    axis=1
)

# Display the final dataset
FinalDataset.head()

# Scatter plot: Sepal Length vs Sepal Width
sns.scatterplot(
    x='SepalLengthCm',
    y='SepalWidthCm',
    hue='Species',
    data=data
)
plt.show()

# Scatter plot: Petal Length vs Petal Width
sns.scatterplot(
    x='PetalLengthCm',
    y='PetalWidthCm',
    hue='Species',
    data=data
)
```

```
plt.show()

# Pair plot
sns.pairplot(data, hue='Species', height=3)
plt.show()

# Histogram: Petal Length
sns.FacetGrid(data, hue='Species', height=5).map(
    sns.histplot, 'PetalLengthCm'
).add_legend()
plt.show()

# Histogram: Petal Width
sns.FacetGrid(data, hue='Species', height=5).map(
    sns.histplot, 'PetalWidthCm'
).add_legend()
plt.show()

# Histogram: Sepal Length
sns.FacetGrid(data, hue='Species', height=5).map(
    sns.histplot, 'SepalLengthCm'
).add_legend()
plt.show()

# Histogram: Sepal Width
sns.FacetGrid(data, hue='Species', height=5).map(
    sns.histplot, 'SepalWidthCm'
).add_legend()
plt.show()
```

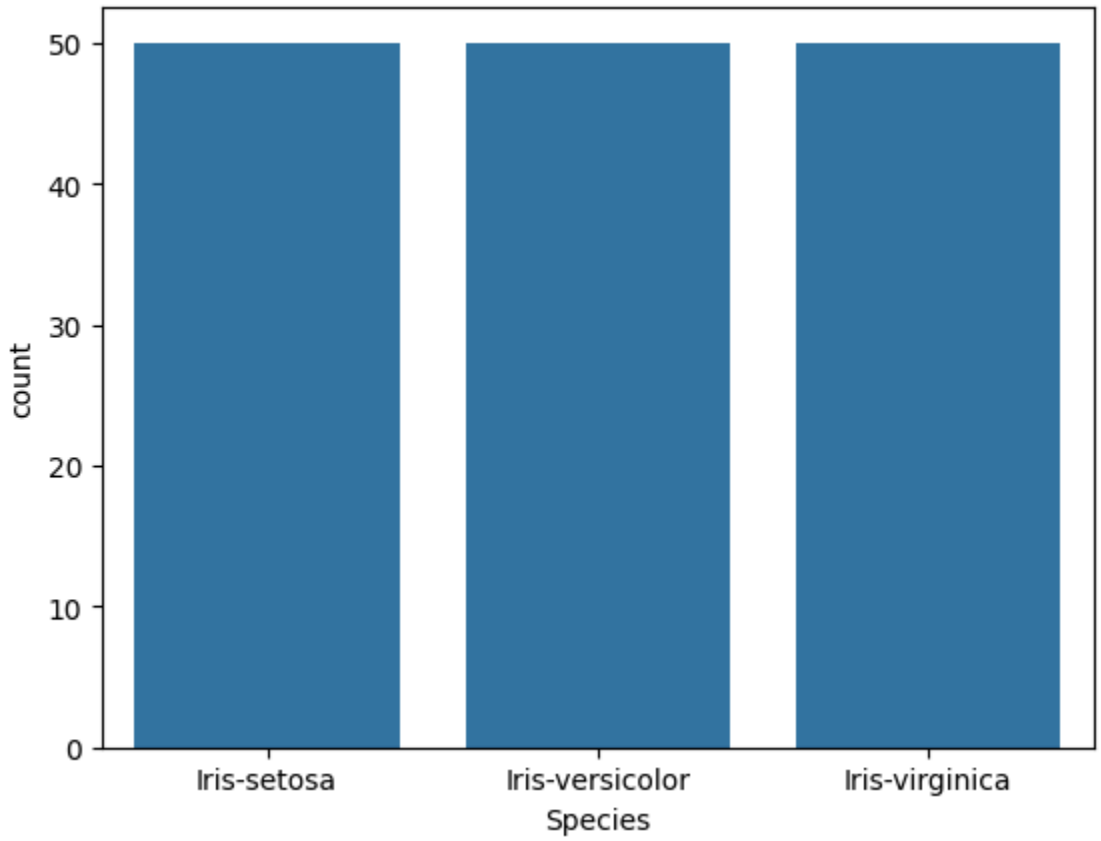
OUTPUT:

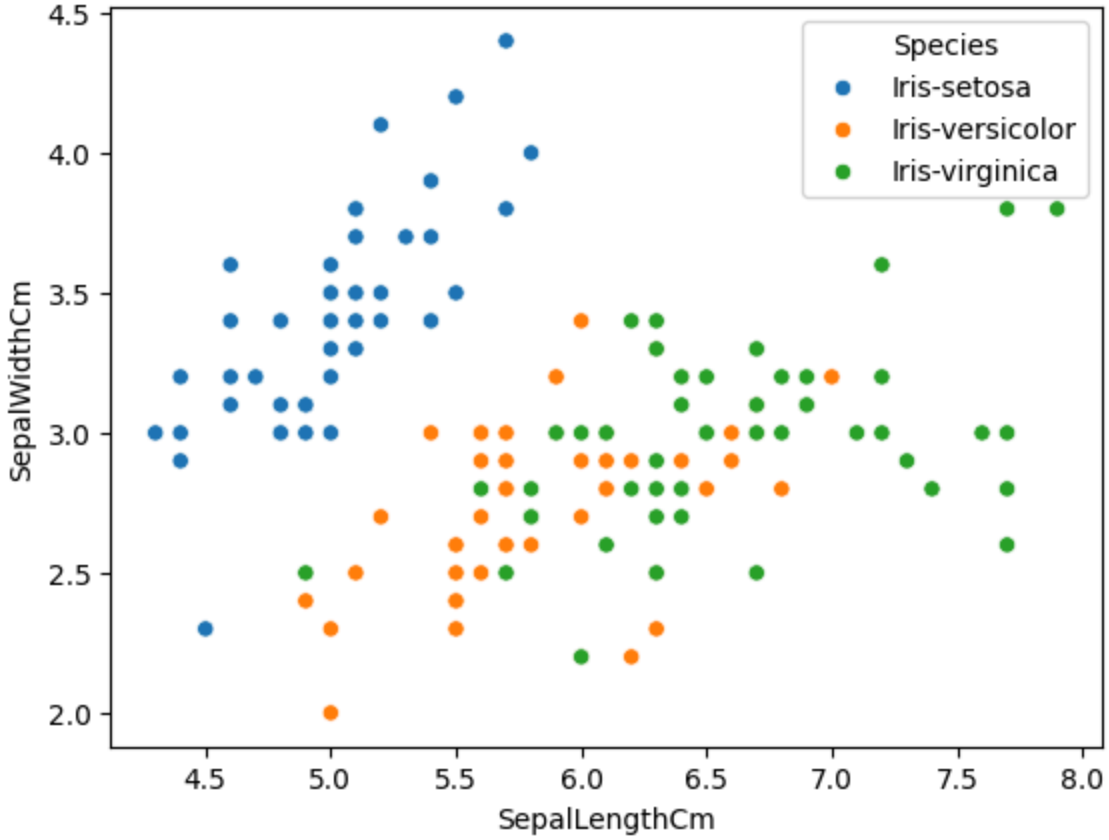
```
<class 'pandas.core.frame.DataFrame'>
```

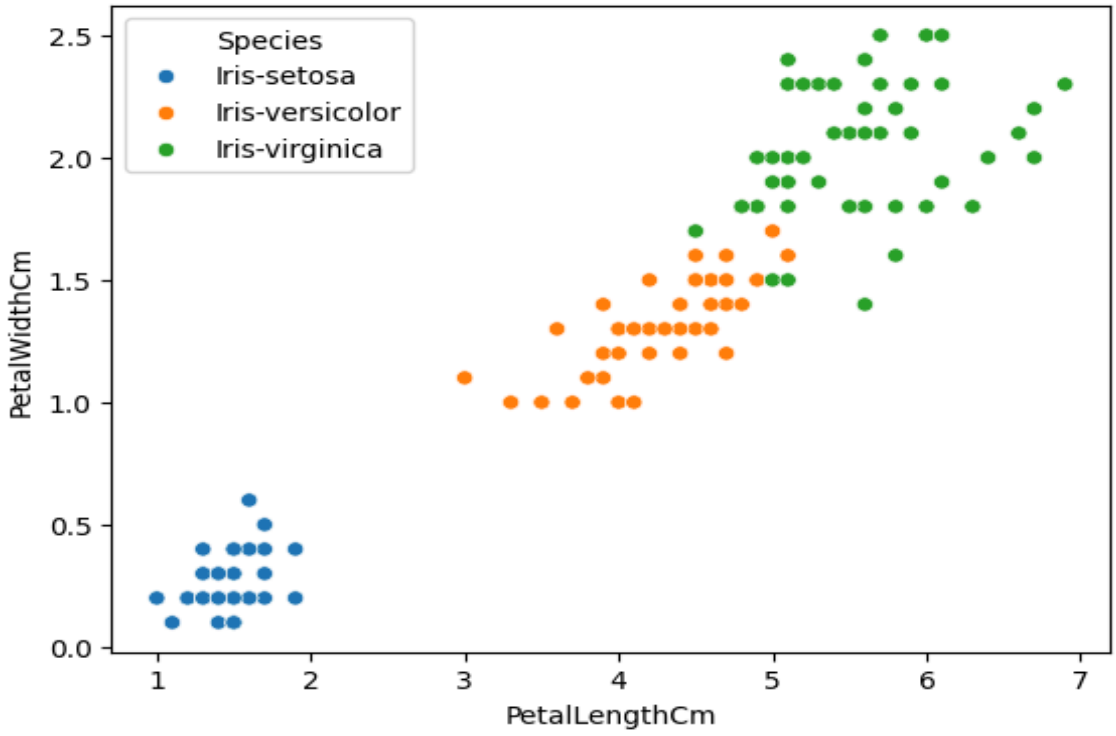
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):

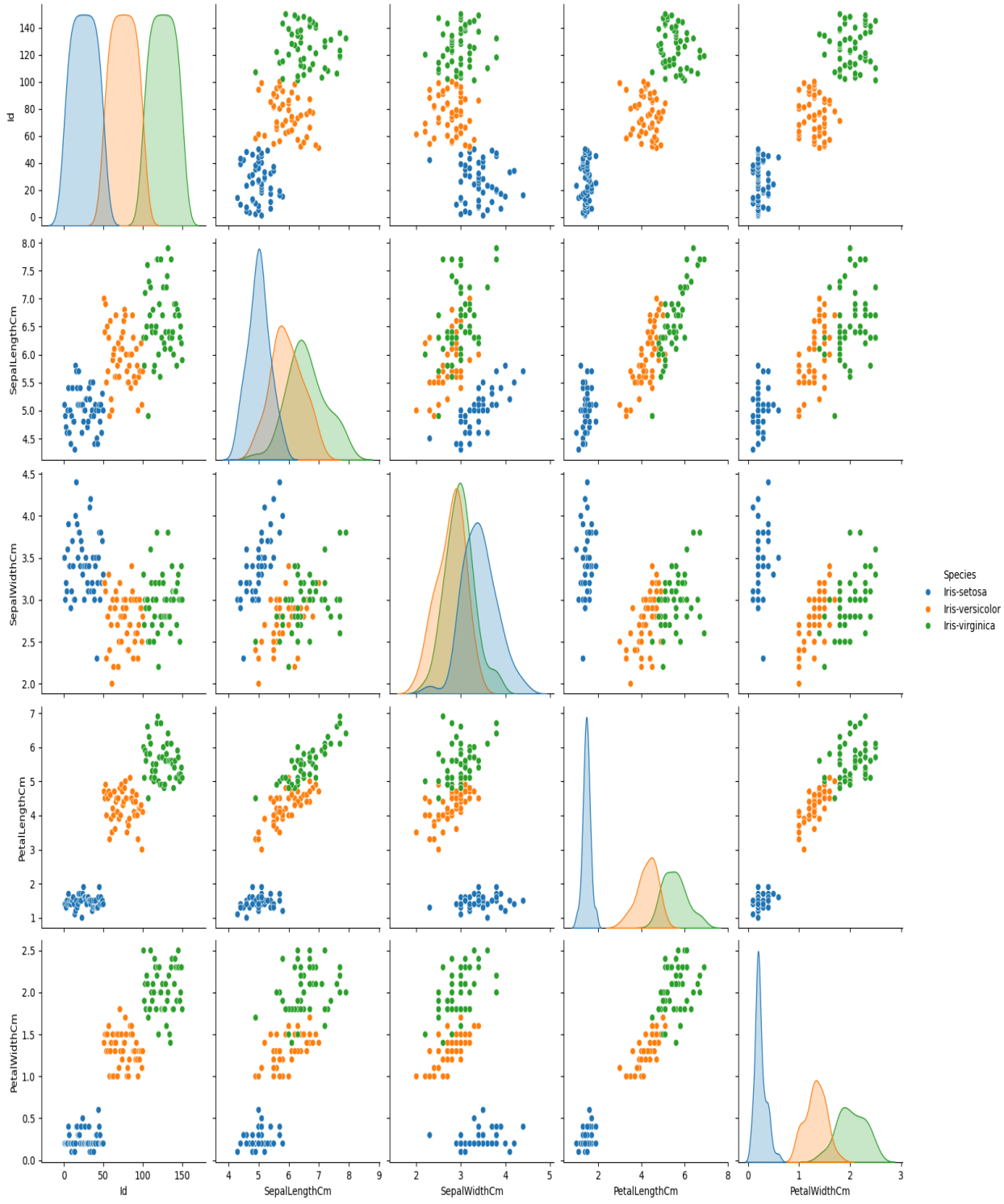
#	Column	Non-Null Count	Dtype
0	Id	150 non-null	int64
1	SepalLengthCm	150 non-null	float64
2	SepalWidthCm	150 non-null	float64
3	PetalLengthCm	150 non-null	float64
4	PetalWidthCm	150 non-null	float64
5	Species	150 non-null	object

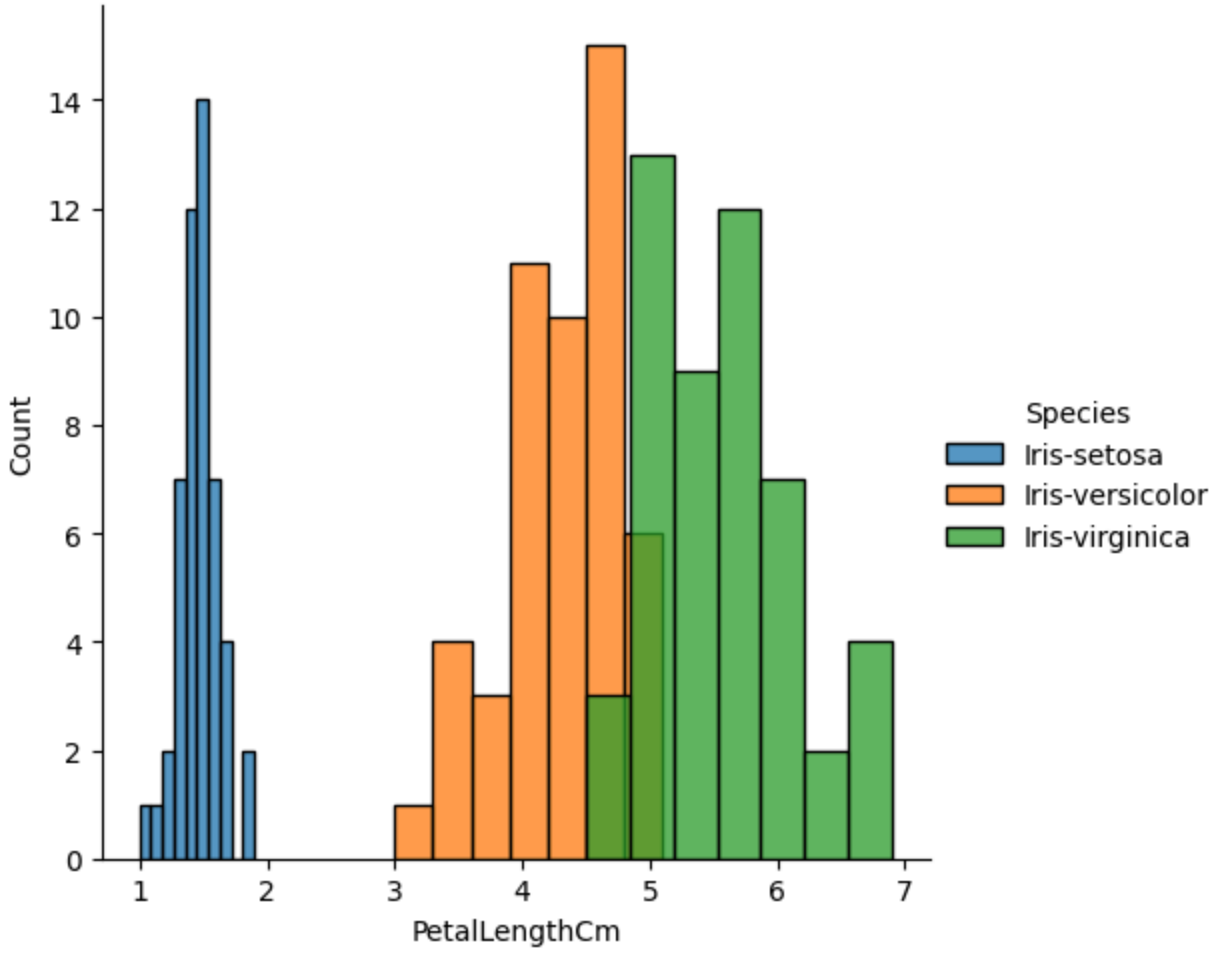
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB

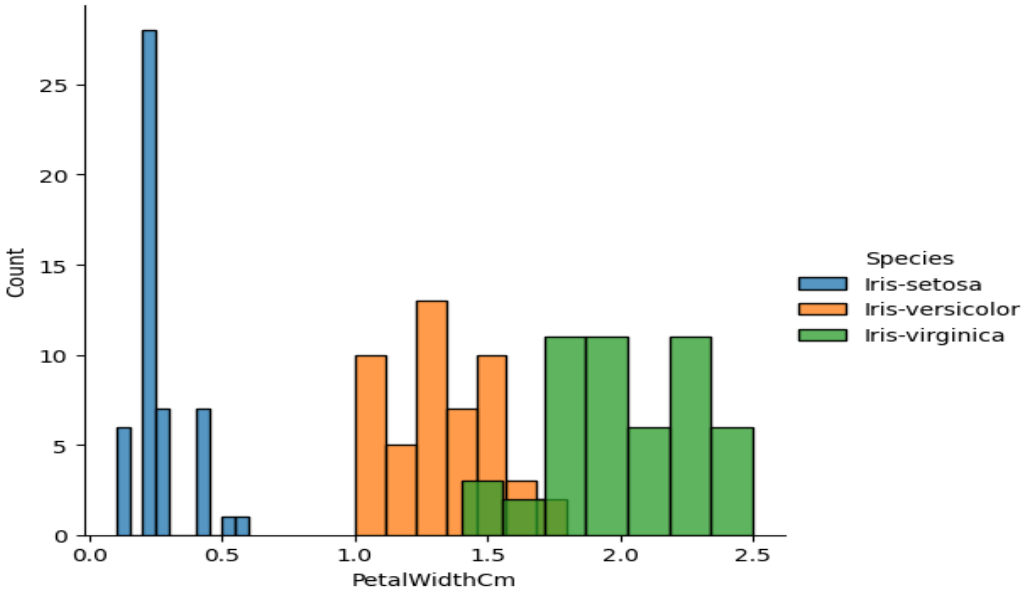


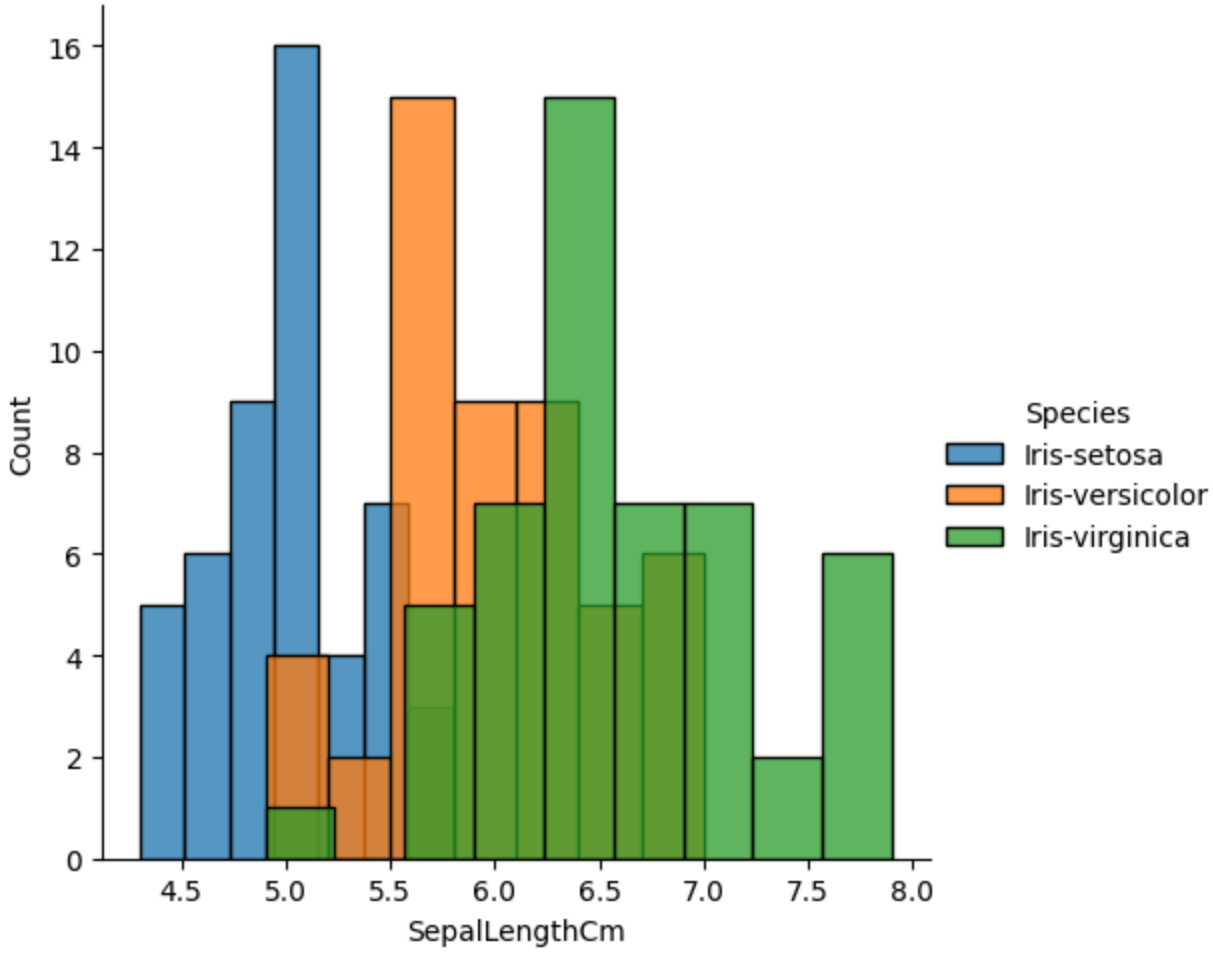

Species
Iris-setosa
Iris-versicolor
Iris-virginica
SepalWidthCm
SepalLengthCm
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0

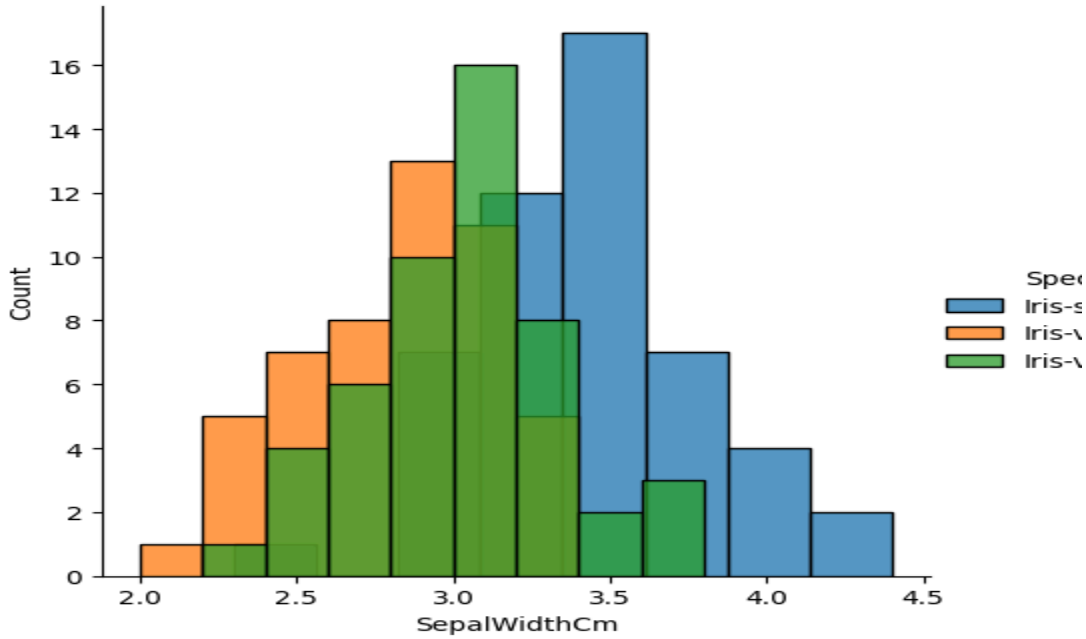

Species
Iris-setosa
Iris-versicolor
Iris-virginica
PetalWidthCm
PetalLengthCm
2.5
2.0
1.5
1.0
0.5
0.0
1
2
3
4
5
6
7


Id
SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm
Species
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalLengthCm
Species
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalWidthCm
Species
Iris-setosa
Iris-versicolor
Iris-virginica


Count
SepalLengthCm
Species
Iris-setosa
Iris-versicolor
Iris-virginica


Count
SepalWidthCm

RESULT:

Thus, the Matplotlib library was successfully studied and implemented using the Iris dataset for data visualization. The dataset was analyzed using statistical summaries, count plots, scatter plots, pair plots, and histograms to visualize the distribution of flower features and compare the three Iris species.

Experiment No:10
Experiment to understand array function in Data Science

AIM:

To understand array function in Data science.

DESCRIPTION:

Understand array function using Numpy library

CODE:

```
import numpy as np

# Create an array with different input values
array = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90])
print("Original array:")
print(array)

# Square root
print("\nSquare root:")
print(np.sqrt(array))

# Number of dimensions
print("\nDimensions:")
print(array.ndim)

# Reshape into 3x3
new_array = array.reshape(3, 3)
print("\n3x3 array:")
print(new_array)

# Number of dimensions
print("\nDimensions of new_array:")
print(new_array.ndim)

# Convert to 1D
print("\nRavel:")
print(new_array.ravel())

# Reshape again
newm = new_array.reshape(3, 3)
```

```
print("\nnewm:")
print(newm)

# Select row 3, columns 2 and 3
print("\nnewm[2,1:3]:")
print(newm[2, 1:3])

# Select row 2, columns 2 and 3
print("\nnewm[1:2,1:3]:")
print(newm[1:2, 1:3])

# Select first column
print("\nnew_array[0:3,0:1]:")
print(new_array[0:3, 0:1])

# Select first column of first 2 rows
print("\nnew_array[0:2,0:1]:")
print(new_array[0:2, 0:1])

# Select rows 2 and 3
print("\nnew_array[1:3]:")
print(new_array[1:3])

# Change a value
new_array[0, 0] = 100

print("\nAfter changing first value:")
print(new_array)
```

OUTPUT:

```
Original array:
[10 20 30 40 50 60 70 80 90]

Square root:
[3.16227766 4.47213595 5.47722558 6.32455532 7.07106781 7.74596669
 8.36660027 8.94427191 9.48683298]

Dimensions:
1

3x3 array:
```

```
[[10 20 30]
 [40 50 60]
 [70 80 90]]

Dimensions of new_array:
2

Ravel:
[10 20 30 40 50 60 70 80 90]

newm:
[[10 20 30]
 [40 50 60]
 [70 80 90]]

newm[2,1:3]:
[80 90]

newm[1:2,1:3]:
[[50 60]]

new_array[0:3,0:1]:
[[10]
 [40]
 [70]]

new_array[0:2,0:1]:
[[10]
 [40]]

new_array[1:3]:
[[40 50 60]
 [70 80 90]]

After changing first value:
[[100  20  30]
 [ 40  50  60]
 [ 70  80  90]]
```

RESULT:

The array operations were successfully performed using numpy including reshaping ,indexing,slicing and raveling.

Experiment No:11

Experiment to understand pandas library use cases in Data science

AIM:

To understand pandas library use cases in Data science.

DESCRIPTION:

Understand data frame use cases using pandas library

CODE:

```
import numpy as np
import pandas as pd


# Create a DataFrame using a list
list = [[1, 'Smith', 50000],
        [2, 'Jones', 60000]]


df = pd.DataFrame(list)
df
# Rename the columns
df.columns = ['Empd', 'Name', 'Salary']
df
# Display DataFrame information
```

```
df.info()


# Read Student Marks dataset

df = pd.read_csv('/content/Student_Marks.csv')

df.info()

# Display first five rows

df.head()

# Display last five rows

df.tail()

# Read Housing dataset

df = pd.read_csv('/content/housing.csv (1).zip',
compression='zip')

# Display first five rows

df.head()

# Display last five rows

df.tail()

# Display dataset information

df.info()

# Display median house value

df['median_house_value']

# Display data type
```

```
type(df['median_house_value'])

# Calculate mean

df['median_house_value'].mean()

# Calculate median

df['median_house_value'].median()

# Calculate mode

df['median_house_value'].mode()

# Calculate variance

df['median_house_value'].var()

# Calculate standard deviation

df['median_house_value'].std()

# Statistical summary

df.describe()

# Statistical summary including categorical columns

df.describe(include='all')

# Get column names

empCol = df.columns

empCol

# Convert DataFrame into NumPy array

emparray = df.values

emparray
```

```
# Convert NumPy array back into DataFrame
employee_DF = pd.DataFrame(emparray, columns=empCol)
employee_DF
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
```

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	number_courses	100 non-null	int64
1	time_study	100 non-null	float64
2	Marks	100 non-null	float64

dtypes: float64(2), int64(1)

memory usage: 2.5 KB

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 20640 entries, 0 to 20639

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	longitude	20640 non-null	float64
1	latitude	20640 non-null	float64
2	housing_median_age	20640 non-null	float64
3	total_rooms	20640 non-null	float64
4	total_bedrooms	20433 non-null	float64
5	population	20640 non-null	float64
6	households	20640 non-null	float64
7	median_income	20640 non-null	float64
8	median_house_value	20640 non-null	float64
9	ocean_proximity	20640 non-null	object

dtypes: float64(9), object(1)

memory usage: 1.6+ MB

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY
...	...	...	...	...	...	...	...	...	...	...
20635	-121.09	39.48	25.0	1665.0	374.0	845.0	330.0	1.5603	78100.0	INLAND

20636	-121.21	39.49	18.0	697.0	150.0	356.0	114.0	2.5568	77100.0	INLAND
20637	-121.22	39.43	17.0	2254.0	485.0	1007.0	433.0	1.7	92300.0	INLAND
20638	-121.32	39.43	18.0	1860.0	409.0	741.0	349.0	1.8672	84700.0	INLAND
20639	-121.24	39.37	16.0	2785.0	616.0	1387.0	530.0	2.3886	89400.0	INLAND

20640 rows × 10 columns

RESULT:

Thus, the Pandas library was studied and implemented successfully to understand its use cases in Data Science, including creating DataFrames, reading CSV and ZIP datasets, displaying dataset information, performing statistical operations, and converting DataFrames into NumPy arrays and back

Experiment No:12
Experiment to detect outliers in a given data set.

AIM:
To detect outliers in a given data set.

DESCRIPTON:
Understand the procedure to identify the outliers in a given dataset

CODE:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Sample calculation for low range (lr), upper range (ur),
percentile

df = pd.read_csv("Student_Marks.csv")
array = df["Marks"].dropna().to_numpy()

array

array.mean()

np.percentile(array, 25)

np.percentile(array, 50)

np.percentile(array, 75)

np.percentile(array, 100)
```

```
# Outliers detection

def outDetection(array):
    sorted(array)
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur

lr, ur = outDetection(array)
lr, ur

# First diagram
sns.displot(array)
plt.show()

# Second diagram
sns.histplot(array, kde=True)
plt.show()

# Remove outliers
new_array = array[(array > lr) & (array < ur)]
new_array

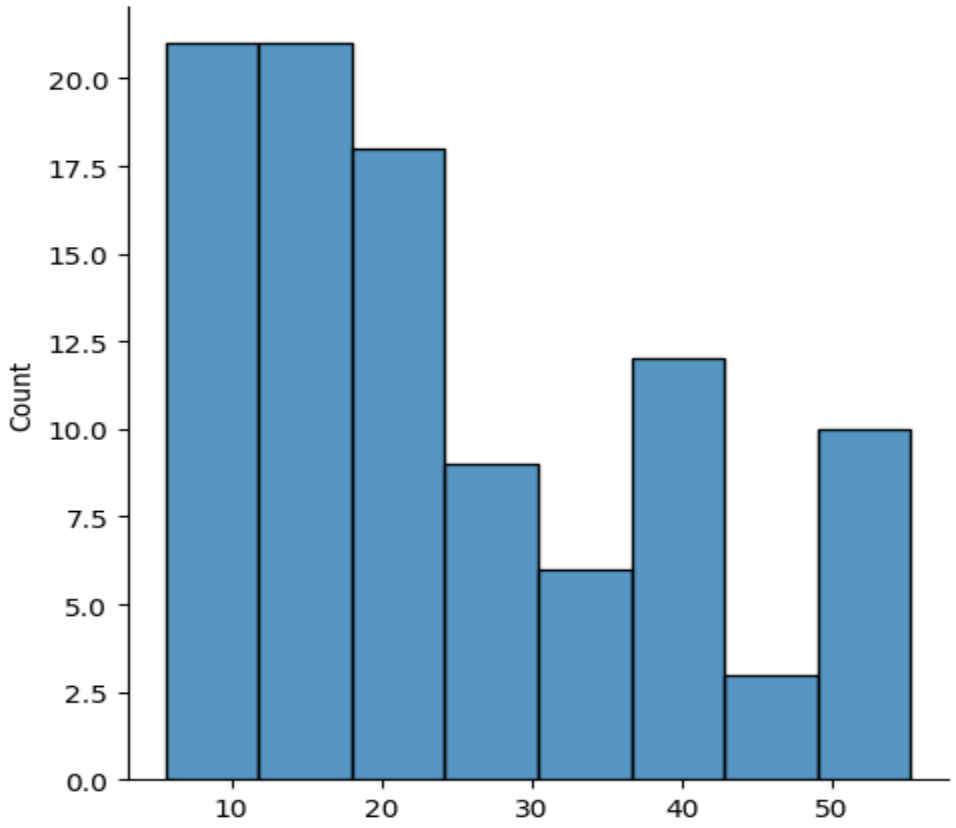
# Third diagram
sns.displot(new_array)
plt.show()

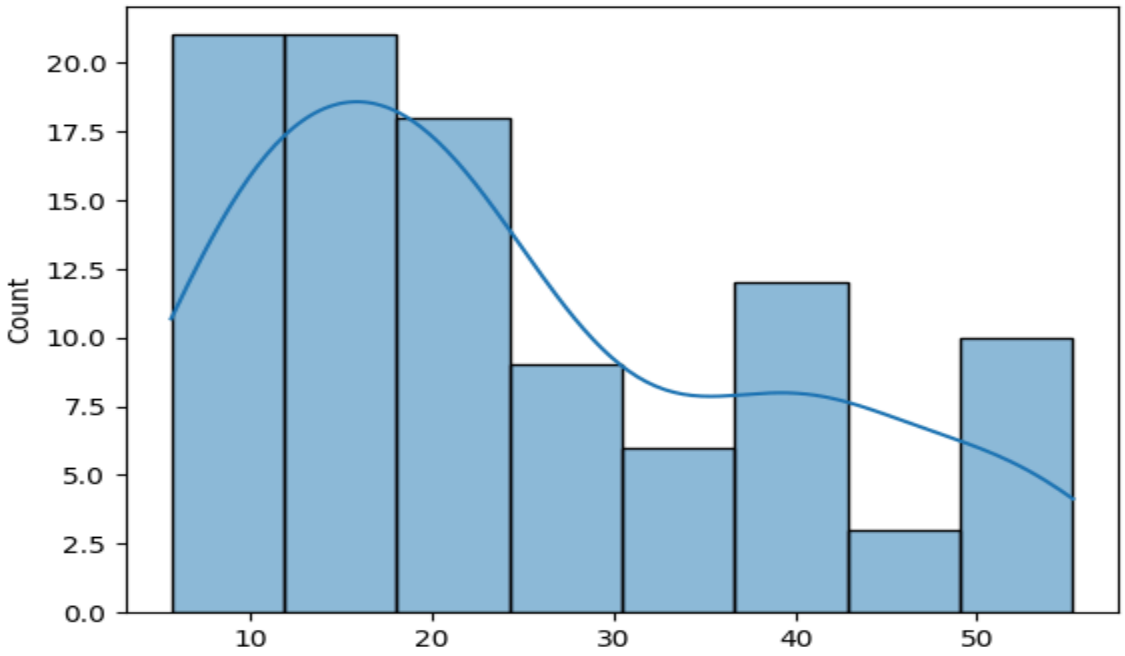
# Calculate new lower and upper ranges
lr1, ur1 = outDetection(new_array)
lr1, ur1
```

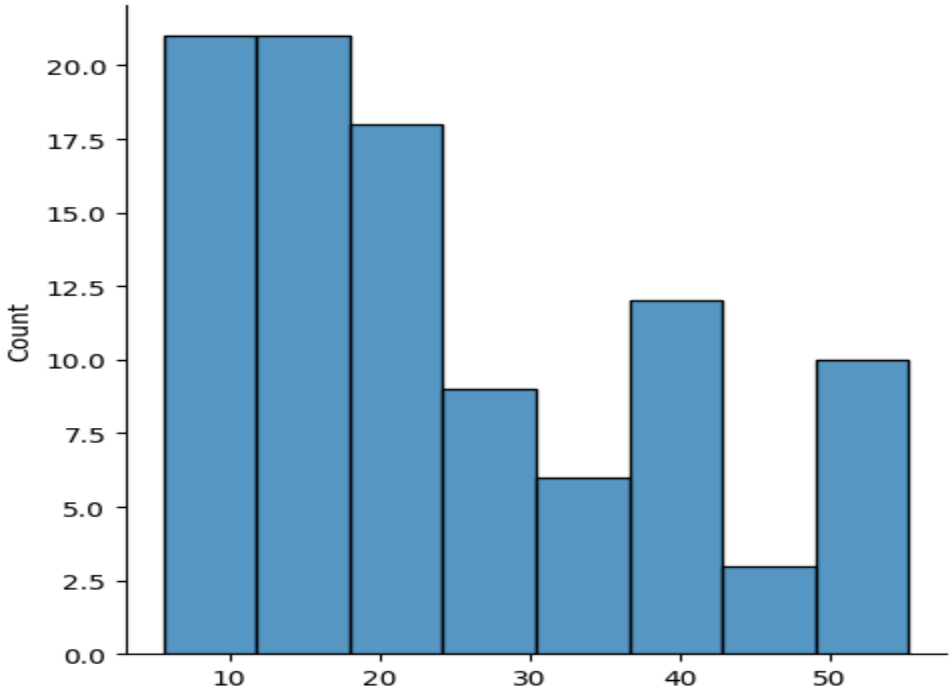
```
# Remove outliers again
final_array = new_array[(new_array > lr1) & (new_array < ur1)]
final_array

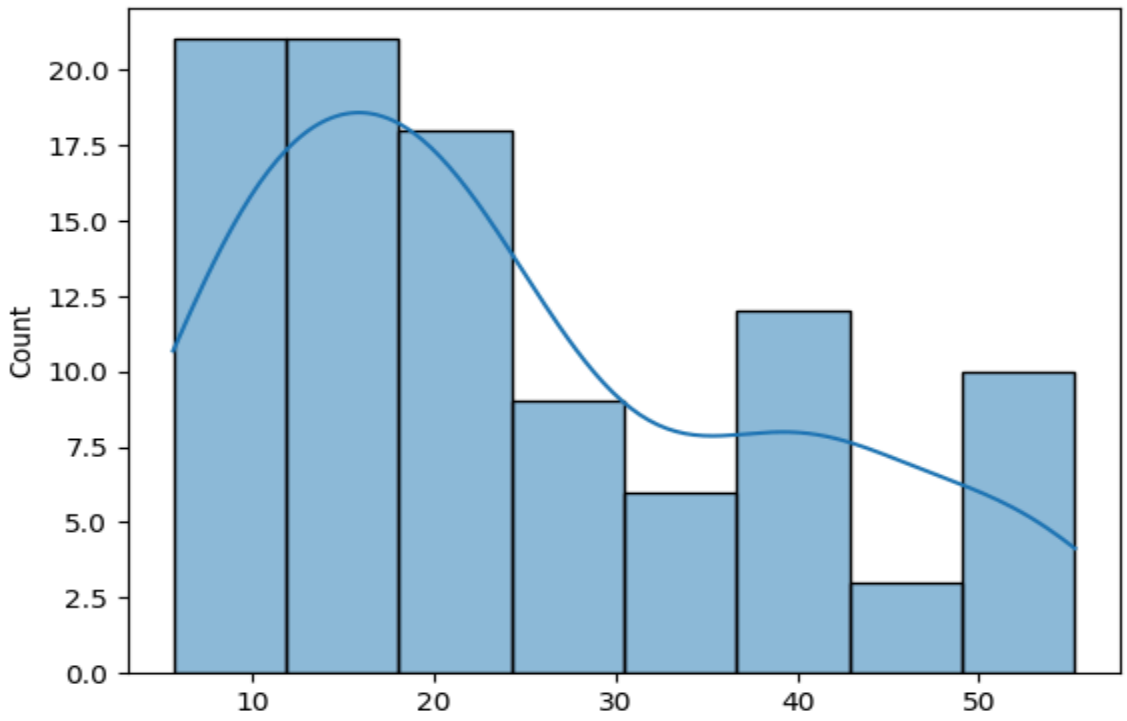
# Fourth diagram
sns.histplot(final_array, kde=True)
plt.show()
```

OUTPUT:




Count
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
10
20
30
40
50


Count
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
10
20
30
40
50



RESULT:

Thus, outliers in the given dataset were successfully detected using the Interquartile Range (IQR) method. The lower and upper limits were calculated using the first and third quartiles, and outliers were removed from the dataset. The distributions before and after removing outliers were visualized using Seaborn plots.

Experiment No:13
Experiment to handle missing values and inappropriate data in a given data set

AIM:
To handle missing values and inappropriate data in a given data set

DESCRIPTION:
Understand the procedure to handle missing and inappropriate data

CODE:
```
import numpy as np
import pandas as pd

# Read the dataset
df = pd.read_csv("/content/student_data.csv")

# Display the original dataset
print("Original Dataset:")
print(df)

# Check duplicate rows
print("\nDuplicate Rows:")
print(df.duplicated())

# Display dataset information
print("\nDataset Information:")
df.info()

# Remove duplicate rows
df.drop_duplicates(inplace=True)

print("\nDataset after removing duplicates:")
```

```
print(df)

# Reset the index
df.reset_index(drop=True, inplace=True)

# Check the column names
print("\nColumn Names:")
print(df.columns)

# Check unique values in categorical columns
print("\nUnique School Names:")
print(df["school"].unique())

print("\nUnique Gender Values:")
print(df["sex"].unique())

print("\nUnique Address Values:")
print(df["address"].unique())

# Replace inappropriate age values with NaN
df.loc[(df["age"] < 10) | (df["age"] > 25), "age"] = np.nan

# Replace inappropriate grade values with NaN
for col in ["G1", "G2", "G3"]:
    df.loc[(df[col] < 0) | (df[col] > 20), col] = np.nan

# Replace negative absences with NaN
df.loc[df["absences"] < 0, "absences"] = np.nan

# Standardize categorical text
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].str.strip().str.lower()
```

```
# Fill missing numerical values using the median
for col in df.select_dtypes(include=np.number).columns:
    df[col] = df[col].fillna(df[col].median())

# Fill missing categorical values using the mode
for col in df.select_dtypes(include="object").columns:
    if df[col].isnull().any() and not df[col].mode().empty:
        df[col] = df[col].fillna(df[col].mode()[0])

# Display the final cleaned dataset
print("\nFinal Cleaned Dataset:")
print(df)

# Check for remaining missing values
print("\nMissing Values in Each Column:")
print(df.isnull().sum())

# Display final dataset information
print("\nFinal Dataset Information:")
df.info()

# Save the cleaned dataset
df.to_csv("/content/Cleaned_Student_Dataset.csv",
index=False)

print("\nCleaned dataset saved successfully!")
```

OUTPUT:

Original Dataset:

school sex age address famsize Pstatus Medu Fedu Mjob Fjob \

0	GP	F	18	U	GT3	A	4	4	at_home	teacher	
1	GP	F	17	U	GT3	T	1	1	at_home	other	
2	GP	F	15	U	LE3	T	1	1	at_home	other	
3	GP	F	15	U	GT3	T	4	2	health	services	
4	GP	F	16	U	GT3	T	3	3	other	other	
..	...	..	...	...	...	...	...	...	...	...	
390	MS	M	20	U	LE3	A	2	2	services	services	
391	MS	M	17	U	LE3	T	3	1	services	services	
392	MS	M	21	R	GT3	T	1	1	other	other	
393	MS	M	18	R	LE3	T	3	2	services	other	
394	MS	M	19	U	LE3	T	1	1	other	at_home	

	...	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	...	4	3	4	1	1	3	6	5	6	6
1	...	5	3	3	1	1	3	4	5	5	6
2	...	4	3	2	2	3	3	10	7	8	10
3	...	3	2	2	1	1	5	2	15	14	15
4	...	4	3	2	1	2	5	4	6	10	10
..	...	...	...	...	...	...	...	...	..	..	..
390	...	5	5	4	4	5	4	11	9	9	9
391	...	2	4	5	3	4	2	3	14	16	16
392	...	5	5	3	3	3	3	3	10	8	7
393	...	4	4	1	3	4	5	0	11	12	10
394	...	3	2	3	3	3	5	5	8	9	9

[395 rows x 33 columns]

Duplicate Rows:

0	False
1	False

```
2      False
3      False
4      False
      ...
390    False
391    False
392    False
393    False
394    False
Length: 395, dtype: bool

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    object
 1   sex         395 non-null    object
 2   age         395 non-null    int64
 3   address     395 non-null    object
 4   famsize     395 non-null    object
 5   Pstatus     395 non-null    object
 6   Medu        395 non-null    int64
 7   Fedu        395 non-null    int64
 8   Mjob        395 non-null    object
 9   Fjob        395 non-null    object
 10  reason      395 non-null    object
 11  guardian    395 non-null    object
 12  traveltime  395 non-null    int64
```

```
 13  studytime    395 non-null    int64
 14  failures     395 non-null    int64
 15  schoolsup    395 non-null    object
 16  famsup       395 non-null    object
 17  paid         395 non-null    object
 18  activities   395 non-null    object
 19  nursery      395 non-null    object
 20  higher       395 non-null    object
 21  internet     395 non-null    object
 22  romantic     395 non-null    object
 23  famrel       395 non-null    int64
 24  freetime     395 non-null    int64
 25  goout        395 non-null    int64
 26  Dalc         395 non-null    int64
 27  Walc         395 non-null    int64
 28  health       395 non-null    int64
 29  absences     395 non-null    int64
 30  G1           395 non-null    int64
 31  G2           395 non-null    int64
 32  G3           395 non-null    int64
dtypes: int64(16), object(17)
memory usage: 102.0+ KB

Dataset after removing duplicates:
   school sex  age address famsize Pstatus  Medu  Fedu      Mjob
Fjob  \
0      GP   F   18       U     GT3       A     4     4   at_home   teacher
1      GP   F   17       U     GT3       T     1     1   at_home     other
2      GP   F   15       U     LE3       T     1     1   at_home     other
3      GP   F   15       U     GT3       T     4     2    health  services
```

```
4      GP   F   16      U     GT3      T     3     3     other     other
..     ...  ..  ...     ...   ...      ...   ...   ...   ...       ...
390    MS   M   20      U     LE3      A     2     2   services  services
391    MS   M   17      U     LE3      T     3     1   services  services
392    MS   M   21      R     GT3      T     1     1     other     other
393    MS   M   18      R     LE3      T     3     2   services     other
394    MS   M   19      U     LE3      T     1     1     other   at_home

     ... famrel freetime  goout  Dalc  Walc health absences  G1  G2  G3
0    ...     4        3      4     1     1      3        6   5   6   6
1    ...     5        3      3     1     1      3        4   5   5   6
2    ...     4        3      2     2     3      3       10   7   8  10
3    ...     3        2      2     1     1      5        2  15  14  15
4    ...     4        3      2     1     2      5        4   6  10  10
..   ...   ...      ...    ...   ...   ...    ...      ...  ..  ..  ..
390  ...     5        5      4     4     5      4       11   9   9   9
391  ...     2        4      5     3     4      2        3  14  16  16
392  ...     5        5      3     3     3      3        3  10   8   7
393  ...     4        4      1     3     4      5        0  11  12  10
394  ...     3        2      3     3     3      5        5   8   9   9

[395 rows x 33 columns]

Column Names:
Index(['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu',
'Fedu',
       'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime',
       'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery',
       'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout', 'Dalc',
       'Walc', 'health', 'absences', 'G1', 'G2', 'G3'],
```

dtype='object')

Unique School Names:
['GP' 'MS']

Unique Gender Values:
['F' 'M']

Unique Address Values:
['U' 'R']

Final Cleaned Dataset:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob
0	gp	f	18.0	u	gt3	a	4	4	at_home	teacher
1	gp	f	17.0	u	gt3	t	1	1	at_home	other
2	gp	f	15.0	u	le3	t	1	1	at_home	other
3	gp	f	15.0	u	gt3	t	4	2	health	services
4	gp	f	16.0	u	gt3	t	3	3	other	other
..	...	..	...	...	...	...	...	...	...	...
390	ms	m	20.0	u	le3	a	2	2	services	services
391	ms	m	17.0	u	le3	t	3	1	services	services
392	ms	m	21.0	r	gt3	t	1	1	other	other
393	ms	m	18.0	r	le3	t	3	2	services	other
394	ms	m	19.0	u	le3	t	1	1	other	at_home

	...	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	...	4	3	4	1	1	3	6.0	5.0	6.0	6.0
1	...	5	3	3	1	1	3	4.0	5.0	5.0	6.0

```
2   ...     4      3     2     2     3     3    10.0   7.0   8.0  10.0
3   ...     3      2     2     1     1     5     2.0  15.0  14.0  15.0
4   ...     4      3     2     1     2     5     4.0   6.0  10.0  10.0
..  ...    ...     ...   ...   ...   ...   ...     ...   ...   ...   ...
390 ...     5      5     4     4     5     4    11.0   9.0   9.0   9.0
391 ...     2      4     5     3     4     2     3.0  14.0  16.0  16.0
392 ...     5      5     3     3     3     3     3.0  10.0   8.0   7.0
393 ...     4      4     1     3     4     5     0.0  11.0  12.0  10.0
394 ...     3      2     3     3     3     5     5.0   8.0   9.0   9.0

[395 rows x 33 columns]
```

Missing Values in Each Column:

```
school        0
sex           0
age           0
address       0
famsize       0
Pstatus       0
Medu          0
Fedu          0
Mjob          0
Fjob          0
reason        0
guardian      0
traveltime    0
studytime     0
failures      0
schoolsup     0
famsup        0
```

```
paid          0
activities    0
nursery       0
higher        0
internet      0
romantic      0
famrel        0
freetime      0
goout         0
Dalc          0
Walc          0
health        0
absences      0
G1            0
G2            0
G3            0
dtype: int64

Final Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    object
 1   sex         395 non-null    object
 2   age         395 non-null    float64
 3   address     395 non-null    object
 4   famsize     395 non-null    object
 5   Pstatus     395 non-null    object
```

```
 6   Medu         395 non-null    int64
 7   Fedu         395 non-null    int64
 8   Mjob         395 non-null    object
 9   Fjob         395 non-null    object
 10  reason       395 non-null    object
 11  guardian     395 non-null    object
 12  traveltime   395 non-null    int64
 13  studytime    395 non-null    int64
 14  failures     395 non-null    int64
 15  schoolsup    395 non-null    object
 16  famsup       395 non-null    object
 17  paid         395 non-null    object
 18  activities   395 non-null    object
 19  nursery      395 non-null    object
 20  higher       395 non-null    object
 21  internet     395 non-null    object
 22  romantic     395 non-null    object
 23  famrel       395 non-null    int64
 24  freetime     395 non-null    int64
 25  goout        395 non-null    int64
 26  Dalc         395 non-null    int64
 27  Walc         395 non-null    int64
 28  health       395 non-null    int64
 29  absences     395 non-null    float64
 30  G1           395 non-null    float64
 31  G2           395 non-null    float64
 32  G3           395 non-null    float64
dtypes: float64(5), int64(11), object(17)
memory usage: 102.0+ KB
```

Cleaned dataset saved successfully!

RESULT:

Thus, missing values and inappropriate data in the given dataset were successfully identified and handled. Duplicate rows were removed, categorical values were standardized, inappropriate values were replaced with suitable values, and missing numerical and categorical values were filled using the median and mode, respectively. The cleaned dataset was saved successfully as `Cleaned_Student_Dataset.csv`.

Experiment No:14
Experiment to understand feature scaling.

AIM:
To understand feature scaling.

DESCRIPTION:
Understand the importance of feature scaling

CODE:

```
import numpy as np
import pandas as pd

df = pd.read_csv('/content/50_Startups.csv')

print(df)

df["State"] = df["State"].fillna(df["State"].mode()[0])

features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values

from sklearn.impute import SimpleImputer

age = SimpleImputer(strategy="mean", missing_values=np.nan)
salary = SimpleImputer(strategy="mean", missing_values=np.nan)

age.fit(features[:, [0]])
salary.fit(features[:, [1]])

features[:, [0]] = age.transform(features[:, [0]])
features[:, [1]] = salary.transform(features[:, [1]])

print(features)
```

```
from sklearn.preprocessing import OneHotEncoder

oh = OneHotEncoder(sparse_output=False)
State = oh.fit_transform(features[:, [3]])

print(State)

final_set = np.concatenate((State, features[:, [0, 1, 2]]),
axis=1)

print(final_set)

from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
sc.fit(final_set)

feat_standard_scaler = sc.transform(final_set)

print(feat_standard_scaler)

from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)

feat_minmax_scaler = mms.transform(final_set)

print(feat_minmax_scaler)
```

OUTPUT:

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83

1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94
5	131876.90	99814.71	362861.36	New York	156991.12
6	134615.46	147198.87	127716.82	California	156122.51
7	130298.13	145530.06	323876.68	Florida	155752.60
8	120542.52	148718.95	311613.29	New York	152211.77
9	123334.88	108679.17	304981.62	California	149759.96
10	101913.08	110594.11	229160.95	Florida	146121.95
11	100671.96	91790.61	249744.55	California	144259.40
12	93863.75	127320.38	249839.44	Florida	141585.52
13	91992.39	135495.07	252664.93	California	134307.35
14	119943.24	156547.42	256512.92	Florida	132602.65
15	114523.61	122616.84	261776.23	New York	129917.04
16	78013.11	121597.55	264346.06	California	126992.93
17	94657.16	145077.58	282574.31	New York	125370.37
18	91749.16	114175.79	294919.57	Florida	124266.90
19	86419.70	153514.11	0.00	New York	122776.86
20	76253.86	113867.30	298664.47	California	118474.03
21	78389.47	153773.43	299737.29	New York	111313.02
22	73994.56	122782.75	303319.26	Florida	110352.25
23	67532.53	105751.03	304768.73	Florida	108733.99
24	77044.01	99281.34	140574.81	New York	108552.04
25	64664.71	139553.16	137962.62	California	107404.34
26	75328.87	144135.98	134050.07	Florida	105733.54
27	72107.60	127864.55	353183.81	New York	105008.31
28	66051.52	182645.56	118148.20	Florida	103282.38
29	65605.48	153032.06	107138.38	New York	101004.64
30	61994.48	115641.28	91131.24	Florida	99937.59
31	61136.38	152701.92	88218.23	New York	97483.56
32	63408.86	129219.61	46085.25	California	97427.84
33	55493.95	103057.49	214634.81	Florida	96778.92
34	46426.07	157693.92	210797.67	California	96712.80
35	46014.02	85047.44	205517.64	New York	96479.51
36	28663.76	127056.21	201126.82	Florida	90708.19
37	44069.95	51283.14	197029.42	California	89949.14

```
38   20229.59       65947.93        185265.10    New York   81229.06
39   38558.51       82982.09        174999.30  California   81005.76
40   28754.33      118546.05        172795.67  California   78239.91
41   27892.92       84710.77        164470.71     Florida   77798.83
42   23640.93       96189.63        148001.11  California   71498.49
43   15505.73      127382.30         35534.17    New York   69758.98
44   22177.74      154806.14         28334.72  California   65200.33
45    1000.23      124153.04          1903.93    New York   64926.08
46    1315.46      115816.21        297114.46     Florida   49490.75
47       0.00      135426.92             0.00  California   42559.73
48     542.05       51743.15             0.00    New York   35673.41
49       0.00      116983.80         45173.06  California   14681.40
[[165349.2 136897.8 471784.1 'New York']
 [162597.7 151377.59 443898.53 'California']
 [153441.51 101145.55 407934.54 'Florida']
 [144372.41 118671.85 383199.62 'New York']
 [142107.34 91391.77 366168.42 'Florida']
 [131876.9 99814.71 362861.36 'New York']
 [134615.46 147198.87 127716.82 'California']
 [130298.13 145530.06 323876.68 'Florida']
 [120542.52 148718.95 311613.29 'New York']
 [123334.88 108679.17 304981.62 'California']
 [101913.08 110594.11 229160.95 'Florida']
 [100671.96 91790.61 249744.55 'California']
 [93863.75 127320.38 249839.44 'Florida']
 [91992.39 135495.07 252664.93 'California']
 [119943.24 156547.42 256512.92 'Florida']
 [114523.61 122616.84 261776.23 'New York']
 [78013.11 121597.55 264346.06 'California']
 [94657.16 145077.58 282574.31 'New York']
 [91749.16 114175.79 294919.57 'Florida']
 [86419.7 153514.11 0.0 'New York']
 [76253.86 113867.3 298664.47 'California']
 [78389.47 153773.43 299737.29 'New York']
 [73994.56 122782.75 303319.26 'Florida']
 [67532.53 105751.03 304768.73 'Florida']
 [77044.01 99281.34 140574.81 'New York']
```

```
 [64664.71 139553.16 137962.62 'California']
 [75328.87 144135.98 134050.07 'Florida']
 [72107.6 127864.55 353183.81 'New York']
 [66051.52 182645.56 118148.2 'Florida']
 [65605.48 153032.06 107138.38 'New York']
 [61994.48 115641.28 91131.24 'Florida']
 [61136.38 152701.92 88218.23 'New York']
 [63408.86 129219.61 46085.25 'California']
 [55493.95 103057.49 214634.81 'Florida']
 [46426.07 157693.92 210797.67 'California']
 [46014.02 85047.44 205517.64 'New York']
 [28663.76 127056.21 201126.82 'Florida']
 [44069.95 51283.14 197029.42 'California']
 [20229.59 65947.93 185265.1 'New York']
 [38558.51 82982.09 174999.3 'California']
 [28754.33 118546.05 172795.67 'California']
 [27892.92 84710.77 164470.71 'Florida']
 [23640.93 96189.63 148001.11 'California']
 [15505.73 127382.3 35534.17 'New York']
 [22177.74 154806.14 28334.72 'California']
 [1000.23 124153.04 1903.93 'New York']
 [1315.46 115816.21 297114.46 'Florida']
 [0.0 135426.92 0.0 'California']
 [542.05 51743.15 0.0 'New York']
 [0.0 116983.8 45173.06 'California']]
[[0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]
 [1. 0. 0.]
```

[0. 1. 0.]
[1. 0. 0.]
[0. 1. 0.]
[0. 0. 1.]
[1. 0. 0.]
[0. 0. 1.]
[0. 1. 0.]
[0. 0. 1.]
[1. 0. 0.]
[0. 0. 1.]
[0. 1. 0.]
[0. 1. 0.]
[0. 0. 1.]
[1. 0. 0.]
[0. 1. 0.]
[0. 0. 1.]
[0. 1. 0.]
[0. 0. 1.]
[0. 1. 0.]
[0. 0. 1.]
[1. 0. 0.]
[0. 1. 0.]
[1. 0. 0.]
[0. 0. 1.]
[0. 1. 0.]
[1. 0. 0.]
[0. 0. 1.]
[1. 0. 0.]
[1. 0. 0.]
[0. 1. 0.]
[1. 0. 0.]
[0. 0. 1.]
[1. 0. 0.]
[0. 0. 1.]
[0. 1. 0.]
[1. 0. 0.]
[0. 0. 1.]

```
 [1. 0. 0.]]
[[0.0 0.0 1.0 165349.2 136897.8 471784.1]
 [1.0 0.0 0.0 162597.7 151377.59 443898.53]
 [0.0 1.0 0.0 153441.51 101145.55 407934.54]
 [0.0 0.0 1.0 144372.41 118671.85 383199.62]
 [0.0 1.0 0.0 142107.34 91391.77 366168.42]
 [0.0 0.0 1.0 131876.9 99814.71 362861.36]
 [1.0 0.0 0.0 134615.46 147198.87 127716.82]
 [0.0 1.0 0.0 130298.13 145530.06 323876.68]
 [0.0 0.0 1.0 120542.52 148718.95 311613.29]
 [1.0 0.0 0.0 123334.88 108679.17 304981.62]
 [0.0 1.0 0.0 101913.08 110594.11 229160.95]
 [1.0 0.0 0.0 100671.96 91790.61 249744.55]
 [0.0 1.0 0.0 93863.75 127320.38 249839.44]
 [1.0 0.0 0.0 91992.39 135495.07 252664.93]
 [0.0 1.0 0.0 119943.24 156547.42 256512.92]
 [0.0 0.0 1.0 114523.61 122616.84 261776.23]
 [1.0 0.0 0.0 78013.11 121597.55 264346.06]
 [0.0 0.0 1.0 94657.16 145077.58 282574.31]
 [0.0 1.0 0.0 91749.16 114175.79 294919.57]
 [0.0 0.0 1.0 86419.7 153514.11 0.0]
 [1.0 0.0 0.0 76253.86 113867.3 298664.47]
 [0.0 0.0 1.0 78389.47 153773.43 299737.29]
 [0.0 1.0 0.0 73994.56 122782.75 303319.26]
 [0.0 1.0 0.0 67532.53 105751.03 304768.73]
 [0.0 0.0 1.0 77044.01 99281.34 140574.81]
 [1.0 0.0 0.0 64664.71 139553.16 137962.62]
 [0.0 1.0 0.0 75328.87 144135.98 134050.07]
 [0.0 0.0 1.0 72107.6 127864.55 353183.81]
 [0.0 1.0 0.0 66051.52 182645.56 118148.2]
 [0.0 0.0 1.0 65605.48 153032.06 107138.38]
 [0.0 1.0 0.0 61994.48 115641.28 91131.24]
 [0.0 0.0 1.0 61136.38 152701.92 88218.23]
 [1.0 0.0 0.0 63408.86 129219.61 46085.25]
 [0.0 1.0 0.0 55493.95 103057.49 214634.81]
 [1.0 0.0 0.0 46426.07 157693.92 210797.67]
 [0.0 0.0 1.0 46014.02 85047.44 205517.64]
```

```
 [0.0 1.0 0.0 28663.76 127056.21 201126.82]
 [1.0 0.0 0.0 44069.95 51283.14 197029.42]
 [0.0 0.0 1.0 20229.59 65947.93 185265.1]
 [1.0 0.0 0.0 38558.51 82982.09 174999.3]
 [1.0 0.0 0.0 28754.33 118546.05 172795.67]
 [0.0 1.0 0.0 27892.92 84710.77 164470.71]
 [1.0 0.0 0.0 23640.93 96189.63 148001.11]
 [0.0 0.0 1.0 15505.73 127382.3 35534.17]
 [1.0 0.0 0.0 22177.74 154806.14 28334.72]
 [0.0 0.0 1.0 1000.23 124153.04 1903.93]
 [0.0 1.0 0.0 1315.46 115816.21 297114.46]
 [1.0 0.0 0.0 0.0 135426.92 0.0]
 [0.0 0.0 1.0 542.05 51743.15 0.0]
 [1.0 0.0 0.0 0.0 116983.8 45173.06]]
[[-7.17740563e-01 -6.85994341e-01  1.39326109e+00  2.01641149e+00
   5.60752915e-01  2.15394309e+00]
 [ 1.39326109e+00 -6.85994341e-01 -7.17740563e-01  1.95586034e+00
   1.08280658e+00  1.92360040e+00]
 [-7.17740563e-01  1.45773797e+00 -7.17740563e-01  1.75436374e+00
  -7.28257028e-01  1.62652767e+00]
 [-7.17740563e-01 -6.85994341e-01  1.39326109e+00  1.55478369e+00
  -9.63646307e-02  1.42221024e+00]
 [-7.17740563e-01  1.45773797e+00 -7.17740563e-01  1.50493720e+00
  -1.07991935e+00  1.28152771e+00]
 [-7.17740563e-01 -6.85994341e-01  1.39326109e+00  1.27980001e+00
  -7.76239071e-01  1.25421046e+00]
 [ 1.39326109e+00 -6.85994341e-01 -7.17740563e-01  1.34006641e+00
   9.32147208e-01 -6.88149930e-01]
 [-7.17740563e-01  1.45773797e+00 -7.17740563e-01  1.24505666e+00
   8.71980011e-01  9.32185978e-01]
 [-7.17740563e-01 -6.85994341e-01  1.39326109e+00  1.03036886e+00
   9.86952101e-01  8.30886909e-01]
 [ 1.39326109e+00 -6.85994341e-01 -7.17740563e-01  1.09181921e+00
  -4.56640246e-01  7.76107440e-01]
 [-7.17740563e-01  1.45773797e+00 -7.17740563e-01  6.20398248e-01
  -3.87599089e-01  1.49807267e-01]
 [ 1.39326109e+00 -6.85994341e-01 -7.17740563e-01  5.93085418e-01
```

-1.06553960e+00 3.19833623e-01]
[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 4.43259872e-01
  2.15449064e-01 3.20617441e-01]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 4.02077603e-01
  5.10178953e-01 3.43956788e-01]
[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 1.01718075e+00
  1.26919939e+00 3.75742273e-01]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 8.97913123e-01
  4.58678535e-02 4.19218702e-01]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 9.44411957e-02
  9.11841968e-03 4.40446224e-01]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 4.60720127e-01
  8.55666318e-01 5.91016724e-01]
[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 3.96724938e-01
 -2.58465367e-01 6.92992062e-01]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 2.79441650e-01
  1.15983657e+00 -1.74312698e+00]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 5.57260867e-02
 -2.69587651e-01 7.23925995e-01]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 1.02723599e-01
  1.16918609e+00 7.32787791e-01]
[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 6.00657792e-03
  5.18495648e-02 7.62375876e-01]
[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 -1.36200724e-01
 -5.62211268e-01 7.74348908e-01]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 7.31146008e-02
 -7.95469167e-01 -5.81939297e-01]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 -1.99311688e-01
  6.56489139e-01 -6.03516725e-01]
[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 3.53702028e-02
  8.21717916e-01 -6.35835495e-01]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 -3.55189938e-02
  2.35068543e-01 1.17427116e+00]
[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 -1.68792717e-01
  2.21014050e+00 -7.67189437e-01]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 -1.78608540e-01
  1.14245677e+00 -8.58133663e-01]

[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 -2.58074369e-01
 -2.05628659e-01 -9.90357166e-01]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 -2.76958231e-01
  1.13055391e+00 -1.01441945e+00]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 -2.26948675e-01
  2.83923813e-01 -1.36244978e+00]
[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 -4.01128925e-01
 -6.59324033e-01 2.98172434e-02]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 -6.00682122e-01
  1.31053525e+00 -1.87861793e-03]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 -6.09749941e-01
 -1.30865753e+00 -4.54931587e-02]
[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 -9.91570153e-01
  2.05924691e-01 -8.17625734e-02]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 -6.52532310e-01
 -2.52599402e+00 -1.15608256e-01]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 -1.17717755e+00
 -1.99727037e+00 -2.12784866e-01]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 -7.73820359e-01
 -1.38312156e+00 -2.97583276e-01]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 -9.89577015e-01
 -1.00900218e-01 -3.15785883e-01]
[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 -1.00853372e+00
 -1.32079581e+00 -3.84552407e-01]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 -1.10210556e+00
 -9.06937535e-01 -5.20595959e-01]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 -1.28113364e+00
  2.17681524e-01 -1.44960468e+00]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 -1.13430539e+00
  1.20641936e+00 -1.50907418e+00]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 -1.60035036e+00
  1.01253936e-01 -1.72739998e+00]
[-7.17740563e-01 1.45773797e+00 -7.17740563e-01 -1.59341322e+00
 -1.99321741e-01 7.11122474e-01]
[1.39326109e+00 -6.85994341e-01 -7.17740563e-01 -1.62236202e+00
  5.07721876e-01 -1.74312698e+00]
[-7.17740563e-01 -6.85994341e-01 1.39326109e+00 -1.61043334e+00

```
 -2.50940884e+00 -1.74312698e+00]
 [ 1.39326109e+00 -6.85994341e-01 -7.17740563e-01 -1.62236202e+00
  -1.57225506e-01 -1.36998473e+00]]
[[0.         0.         1.         1.         0.65174393 1.        ]
 [1.         0.         0.         0.98335946 0.76197173 0.94089337]
 [0.         1.         0.         0.92798459 0.37957895 0.8646636 ]
 [0.         0.         1.         0.87313643 0.51299839 0.81223513]
 [0.         1.         0.         0.85943772 0.30532804 0.77613557]
 [0.         0.         1.         0.797566   0.3694479  0.76912588]
 [1.         0.         0.         0.81412828 0.73016111 0.27071031]
 [0.         1.         0.         0.7880179  0.71745725 0.68649342]
 [0.         0.         1.         0.72901786 0.74173276 0.66049977]
 [1.         0.         0.         0.74590551 0.43692884 0.64644319]
 [0.         1.         0.         0.61635061 0.45150637 0.48573267]
 [1.         0.         0.         0.60884455 0.30836422 0.52936195]
 [0.         1.         0.         0.56766982 0.57883556 0.52956308]
 [1.         0.         0.         0.55635219 0.64106561 0.53555202]
 [0.         1.         0.         0.72539353 0.8013272  0.54370828]
 [0.         0.         1.         0.69261666 0.54302973 0.55486446]
 [1.         0.         0.         0.47180821 0.53527036 0.56031151]
 [0.         0.         1.         0.57246821 0.71401273 0.59894835]
 [0.         1.         0.         0.55488118 0.47877201 0.62511553]
 [0.         0.         1.         0.52264964 0.77823604 0.        ]
 [1.         0.         0.         0.46116861 0.47642362 0.63305328]
 [0.         0.         1.         0.47408436 0.78021012 0.63532724]
 [0.         1.         0.         0.4475048  0.54429273 0.64291963]
 [0.         1.         0.         0.40842369 0.4146383  0.64599195]
 [0.         0.         1.         0.46594728 0.3653876  0.29796428]
 [1.         0.         0.         0.39107967 0.67195793 0.29242745]
 [0.         1.         0.         0.45557444 0.70684477 0.28413435]
 [0.         0.         1.         0.43609283 0.58297807 0.74861321]
 [0.         1.         0.         0.39946683 1.         0.25042853]
 [0.         0.         1.         0.39676926 0.77456642 0.22709197]
 [0.         1.         0.         0.37493063 0.48992809 0.19316302]
 [0.         0.         1.         0.36974101 0.77205322 0.18698856]
 [1.         0.         0.         0.38348453 0.5932935  0.09768292]
 [0.         1.         0.         0.33561668 0.39413365 0.45494286]
```

```
[1.         0.         0.         0.2807759  0.81005496 0.44680961]
[0.         0.         1.         0.2782839  0.25703165 0.43561799]
[0.         1.         0.         0.17335288 0.57682456 0.42631115]
[1.         0.         0.         0.26652654 0.         0.41762624]
[0.         0.         1.         0.12234465 0.11163611 0.39269043]
[1.         0.         0.         0.23319442 0.24130912 0.3709309 ]
[1.         0.         0.         0.17390063 0.51204073 0.36626005]
[0.         1.         0.         0.16869099 0.25446874 0.34861436]
[1.         0.         0.         0.14297577 0.34185188 0.31370517]
[0.         0.         1.         0.09377566 0.57930693 0.07531871]
[1.         0.         0.         0.13412668 0.78807166 0.06005866]
[0.         0.         1.         0.0060492  0.5547241  0.0040356 ]
[0.         1.         0.         0.00795565 0.49125975 0.62976785]
[1.         0.         0.         0.         0.64054682 0.        ]
[0.         0.         1.         0.00327821 0.00350184 0.        ]
[1.         0.         0.         0.         0.50014806 0.09574943]]
```

RESULT:

Thus, feature scaling was successfully implemented using the StandardScaler and MinMaxScaler methods. The categorical State feature was converted into numerical values using OneHotEncoder, and missing values were handled using SimpleImputer. The experiment demonstrated how feature scaling transforms data into a suitable format for machine learning applications.

Experiment No:15
Experiment to understand the data preprocessing in Data Science

AIM:
To understand the data preprocessing in Datascience

DESCRIPTION:
Understand the importance of Data preprocessing in data science

CODE:

```
import numpy as np
import pandas as pd

# Create sample dataset
df = pd.DataFrame({
    'Country': ['France', 'Spain', np.nan, 'Spain', 'Germany',
                'France', 'Spain', 'Germany', 'France',
'Germany'],
    'Age': [44, 27, 30, 38, 40, 35, np.nan, 48, 50, 37],
    'Salary': [72000, 48000, 54000, 61000, np.nan,
               58000, 52000, 79000, 83000, 67000],
    'Purchased': ['No', 'Yes', 'No', 'No', 'Yes',
                  'Yes', 'No', 'Yes', 'No', 'Yes']
})

# Save dataset
df.to_csv('/content/preprocessing_sample.csv', index=False)

# Read dataset
df = pd.read_csv('/content/preprocessing_sample.csv')
print(df)
print("\n\n")
```

```
# Display dataset information
df.info()
print("\n\n")

# Display Country column
print(df['Country'])
print("\n\n")

# Find mode of Country
print(df['Country'].mode())
print("\n\n")

# Display first mode
print(df['Country'].mode()[0])
print("\n\n")

# Display type of mode
print(type(df['Country'].mode()))
print("\n\n")

# Fill missing values
df['Country'] = df['Country'].fillna(df['Country'].mode()[0])
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(round(df['Salary'].mean()))

# Display updated dataset
print(df)
print("\n\n")

# Convert Country into dummy variables
print(pd.get_dummies(df['Country']))
print("\n\n")
```

```
# Combine dummy variables with other columns
updated_dataset = pd.concat(
    [pd.get_dummies(df['Country']), df.iloc[:, [1, 2, 3]]],
    axis=1
)

print(updated_dataset)
print("\n\n")

# Display dataset information
updated_dataset.info()
print("\n\n")

# Convert Purchased values into numbers
updated_dataset['Purchased'] =
updated_dataset['Purchased'].replace(
    ['No', 'Yes'], [0, 1]
)

# Display final dataset
print(updated_dataset)
```

OUTPUT:

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	NaN	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	Germany	48.0	79000.0	Yes
8	France	50.0	83000.0	No
9	Germany	37.0	67000.0	Yes

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Country    9 non-null      object 
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object 
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

```
0     France
1      Spain
2        NaN
3      Spain
4    Germany
5     France
6      Spain
7    Germany
8     France
9    Germany
Name: Country, dtype: object
```

```
0     France
1    Germany
2      Spain
Name: Country, dtype: object
```

France

<class 'pandas.core.series.Series'>

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	France	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	63778.0	Yes
5	France	35.0	58000.0	Yes
6	Spain	38.0	52000.0	No
7	Germany	48.0	79000.0	Yes
8	France	50.0	83000.0	No
9	Germany	37.0	67000.0	Yes

	France	Germany	Spain
0	True	False	False
1	False	False	True
2	True	False	False
3	False	False	True
4	False	True	False
5	True	False	False
6	False	False	True
7	False	True	False
8	True	False	False
9	False	True	False

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	No
1	False	False	True	27.0	48000.0	Yes
2	True	False	False	30.0	54000.0	No
3	False	False	True	38.0	61000.0	No
4	False	True	False	40.0	63778.0	Yes
5	True	False	False	35.0	58000.0	Yes
6	False	False	True	38.0	52000.0	No
7	False	True	False	48.0	79000.0	Yes
8	True	False	False	50.0	83000.0	No
9	False	True	False	37.0	67000.0	Yes

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   France     10 non-null     bool
 1   Germany    10 non-null     bool
 2   Spain      10 non-null     bool
 3   Age        10 non-null     float64
 4   Salary     10 non-null     float64
 5   Purchased  10 non-null     object
dtypes: bool(3), float64(2), object(1)
memory usage: 402.0+ bytes
```

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	0
1	False	False	True	27.0	48000.0	1
2	True	False	False	30.0	54000.0	0
3	False	False	True	38.0	61000.0	0
4	False	True	False	40.0	63778.0	1
5	True	False	False	35.0	58000.0	1

6	False	False	True	38.0	52000.0	0
7	False	True	False	48.0	79000.0	1
8	True	False	False	50.0	83000.0	0
9	False	True	False	37.0	67000.0	1

/tmp/ipykernel_1895/1211043690.py:70: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
  updated_dataset['Purchased'] = updated_dataset['Purchased'].replace(

RESULT:

Thus, data preprocessing was successfully performed using Pandas and NumPy. Missing values were handled using mode, median, and mean imputation. Categorical data in the `Country` column was converted into dummy variables, and the `Purchased` column was encoded into numerical values (0 and 1). The final dataset was cleaned and prepared for further data analysis and machine learning.

Experiment No:16
Experiment to understand EDA-Quantitative and Qualitative analysis.

AIM:
To understand EDA-Quantitative and Qualitative analysis.

DESCRIPTION:
Understand the importance of EDA-Quantitative and Qualitative analysis.

CODE:

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
tips=sns.load_dataset('tips')
print(tips.head())
print("\n\n")

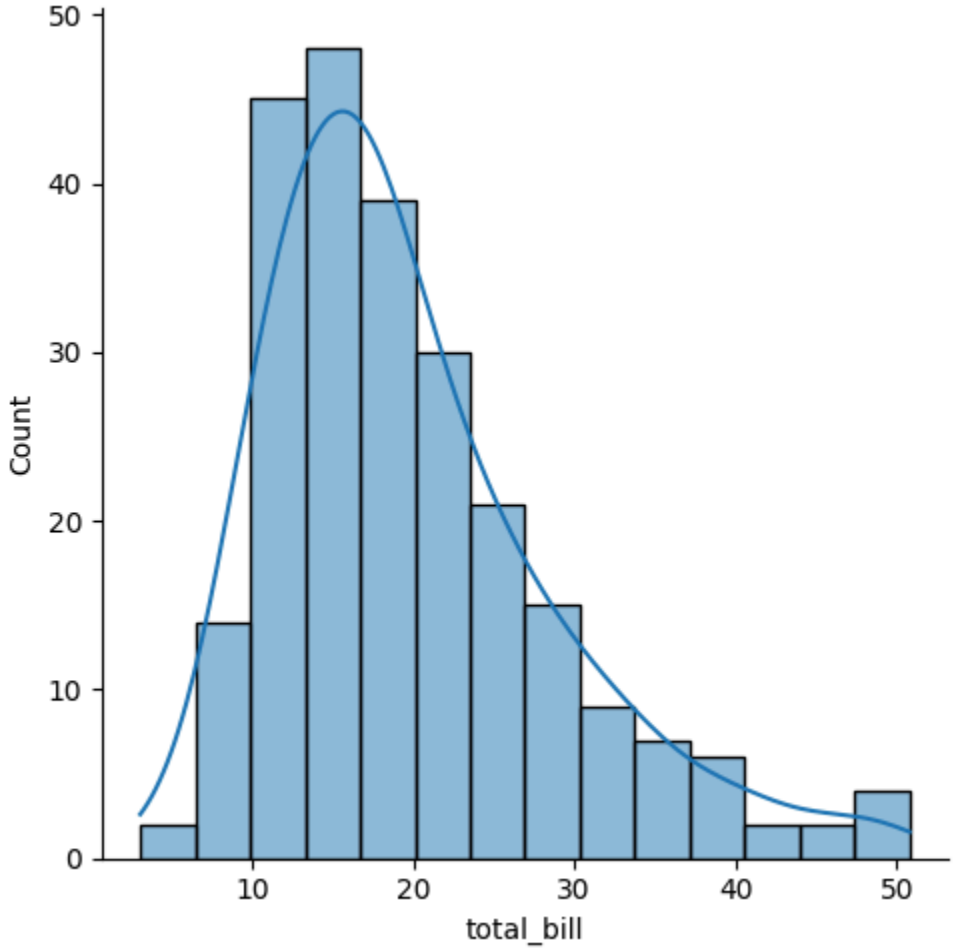

sns.displot(tips.total_bill,kde=True)
plt.show()

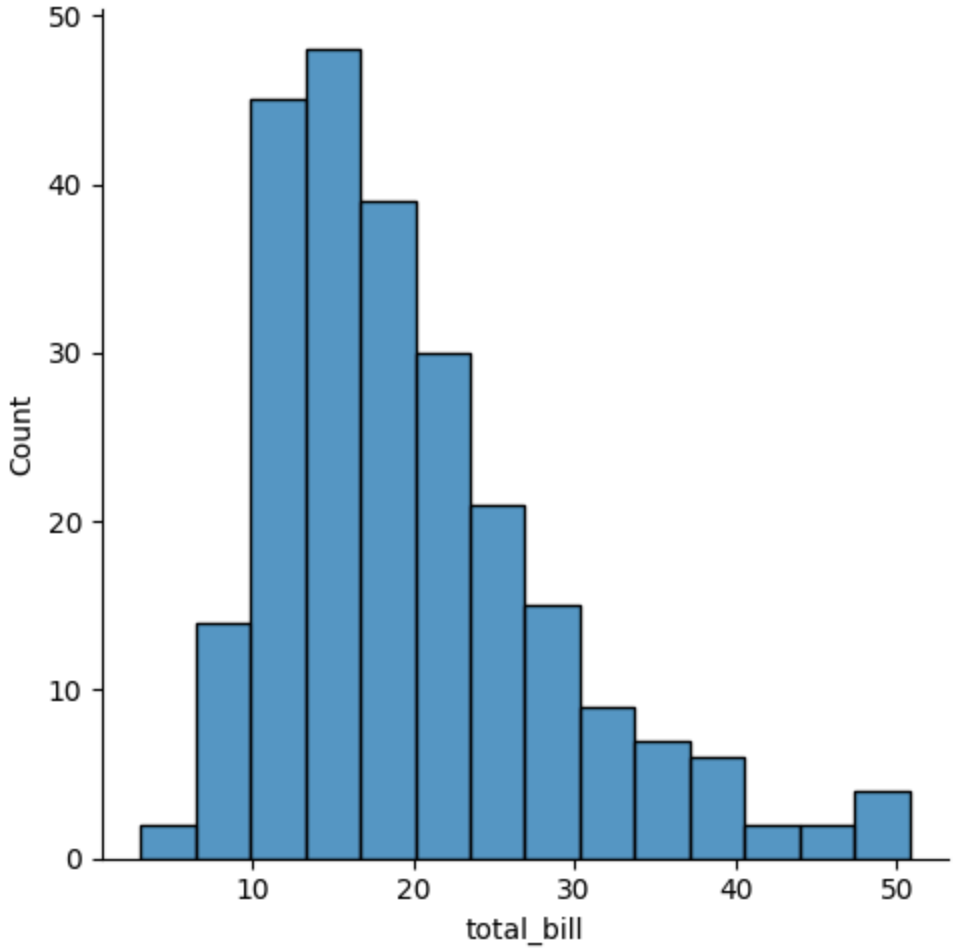

sns.displot(tips.total_bill,kde=False)
plt.show()
sns.jointplot(x=tips.tip,y=tips.total_bill)
plt.show()
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="reg")plt.show
()
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="hex")
plt.show()
```

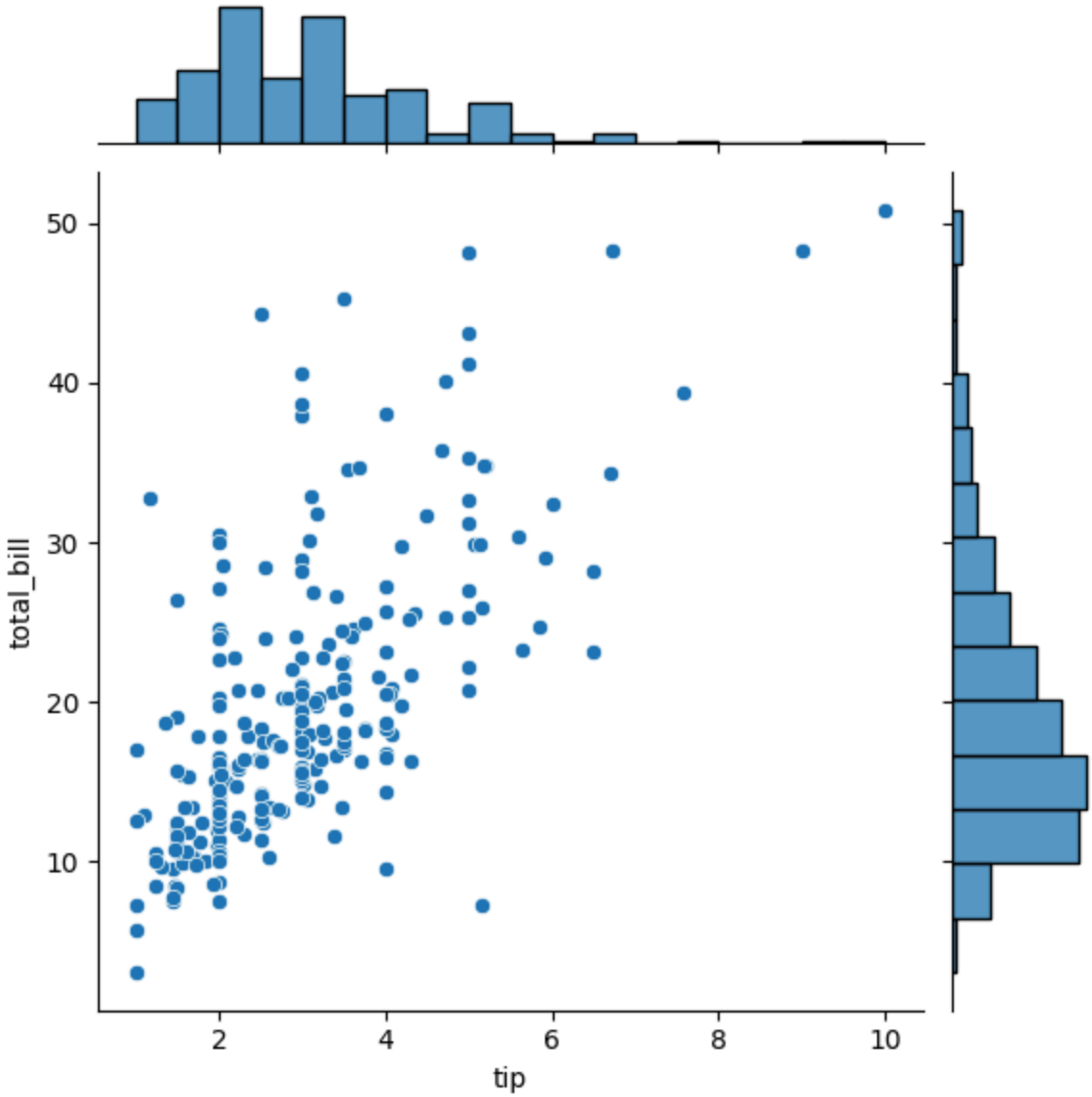
```
sns.pairplot(tips)
plt.show()
print(tips.time.value_counts())
print("\n\n")
sns.pairplot(tips,hue='time')
plt.show()
sns.pairplot(tips,hue='day')
plt.show()
sns.heatmap(tips.corr(numeric_only=True),annot=True)
plt.show()
sns.boxplot(tips.total_bill)
plt.show()
sns.boxplot(tips.tip)
plt.show()
sns.countplot(tips.day)
plt.show()
sns.countplot(tips.sex)
plt.show()
tips.sex.value_counts().plot(kind='pie')
plt.show()
tips.sex.value_counts().plot(kind='bar')
plt.show()
sns.countplot(tips[tips.time=='Dinner']['day'])
plt.show()
```

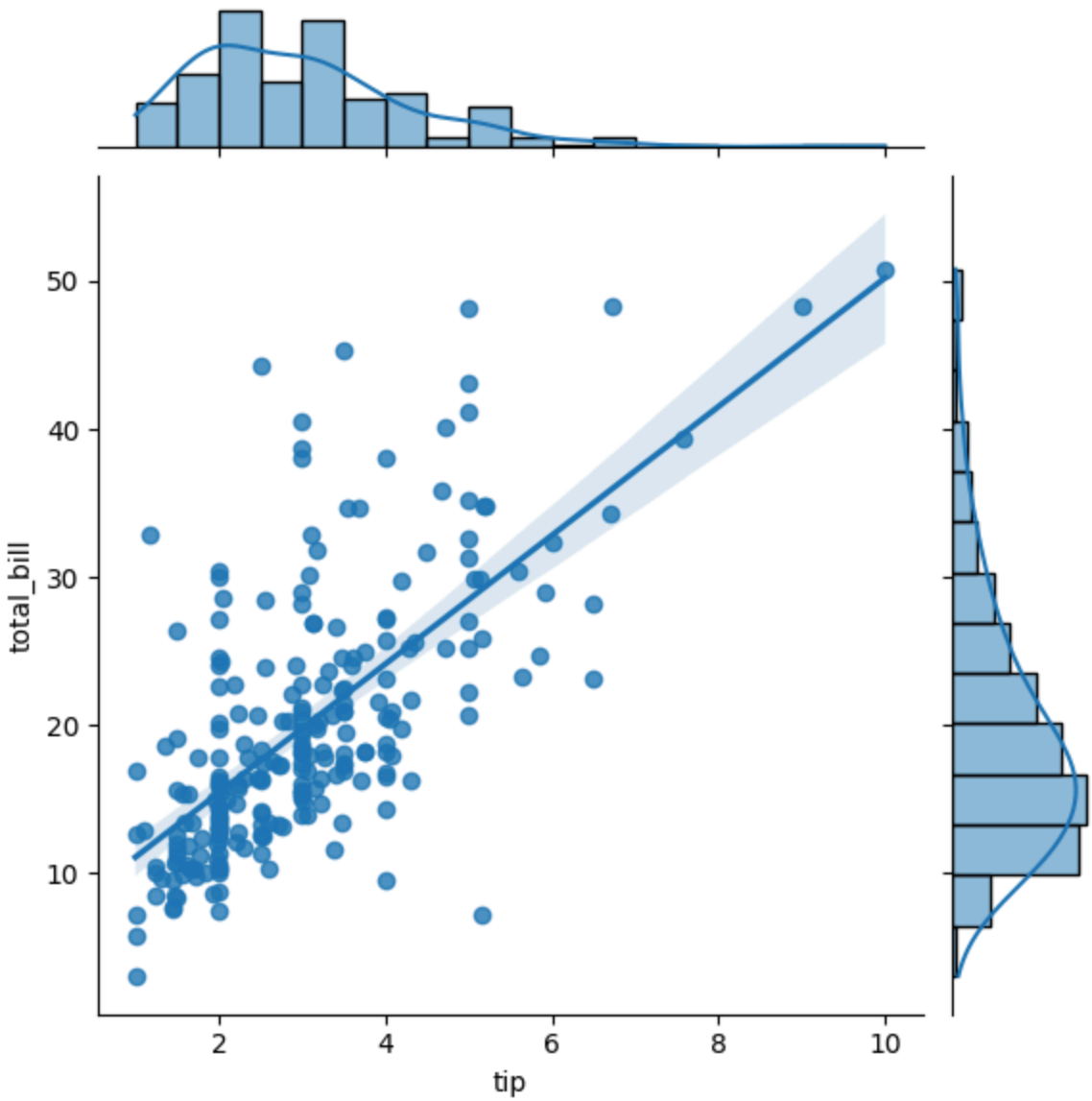
OUTPUT:

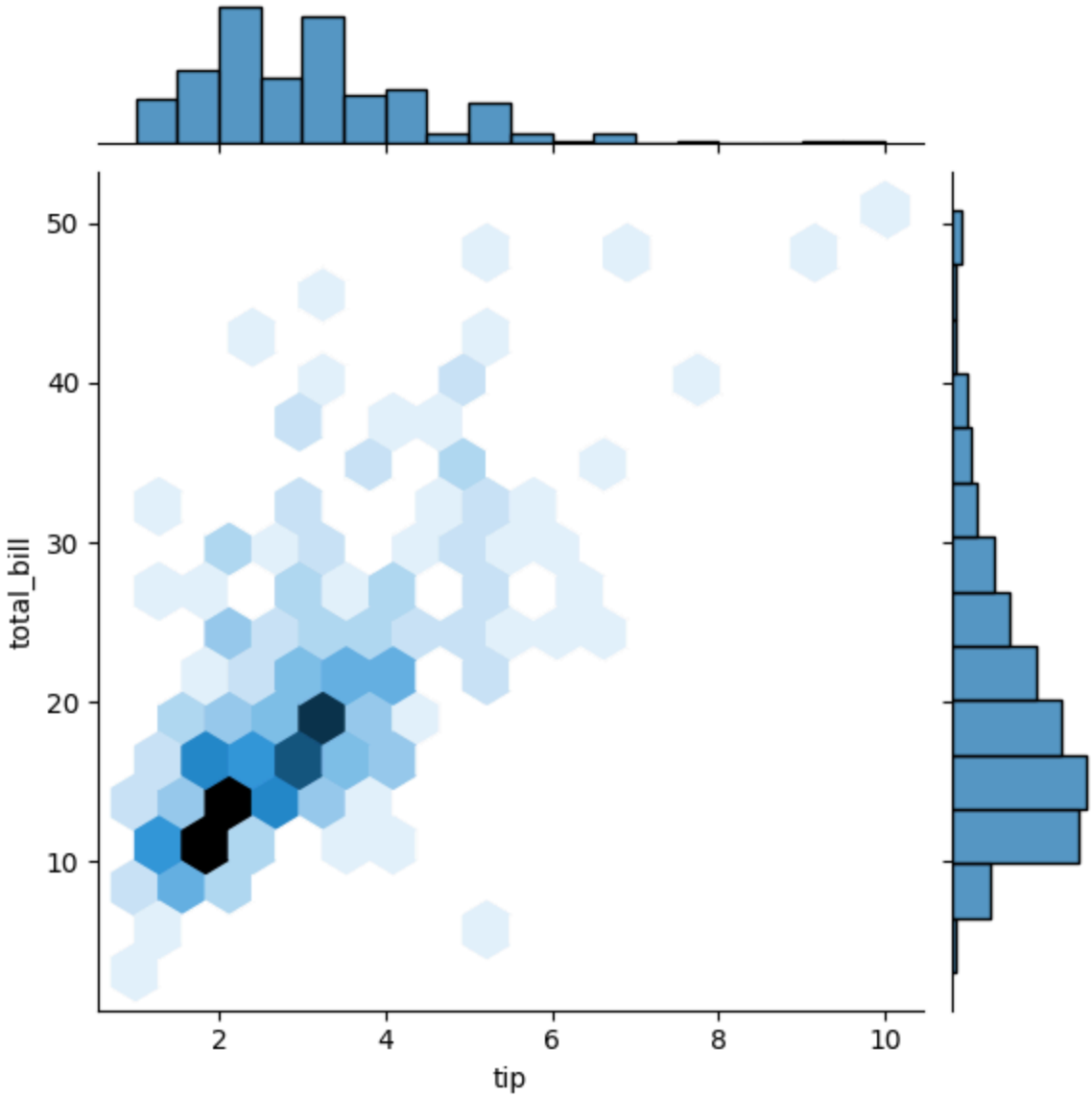
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

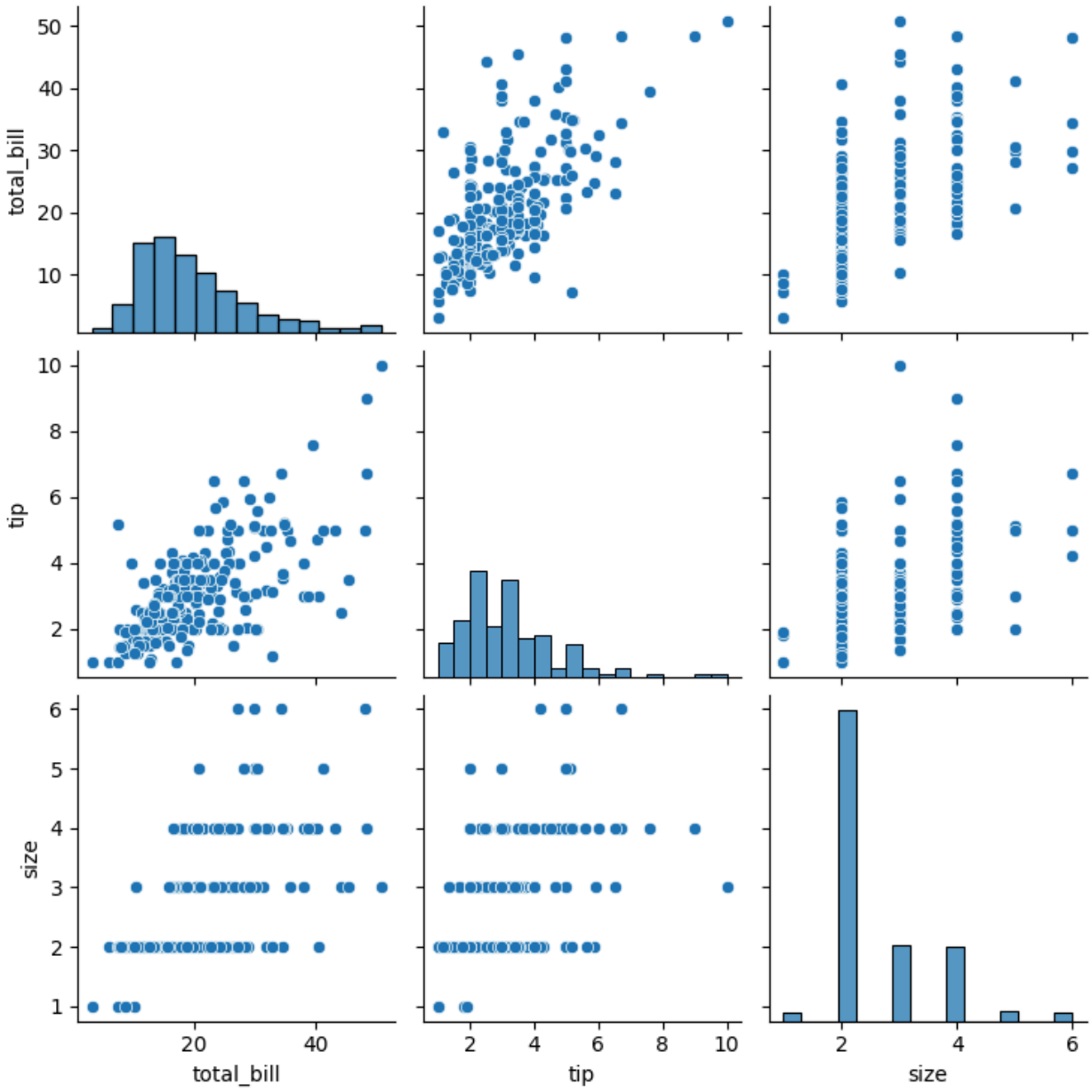

50
40
30
20
10
0
Count
10
20
30
40
50
total_bill


50
40
30
20
10
0
Count
10
20
30
40
50
total_bill

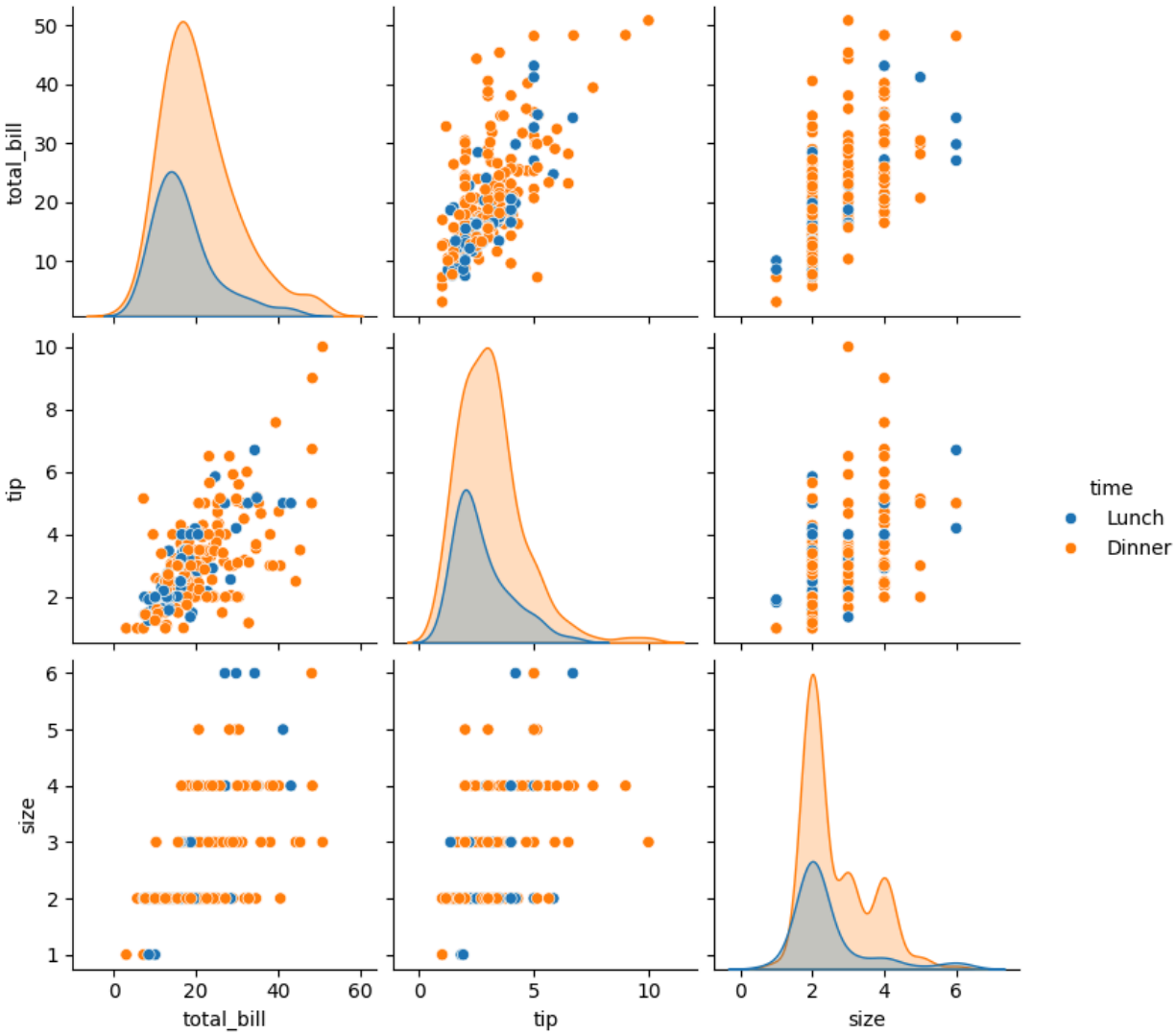

50
40
30
20
10
total_bill
2
4
6
8
10
tip

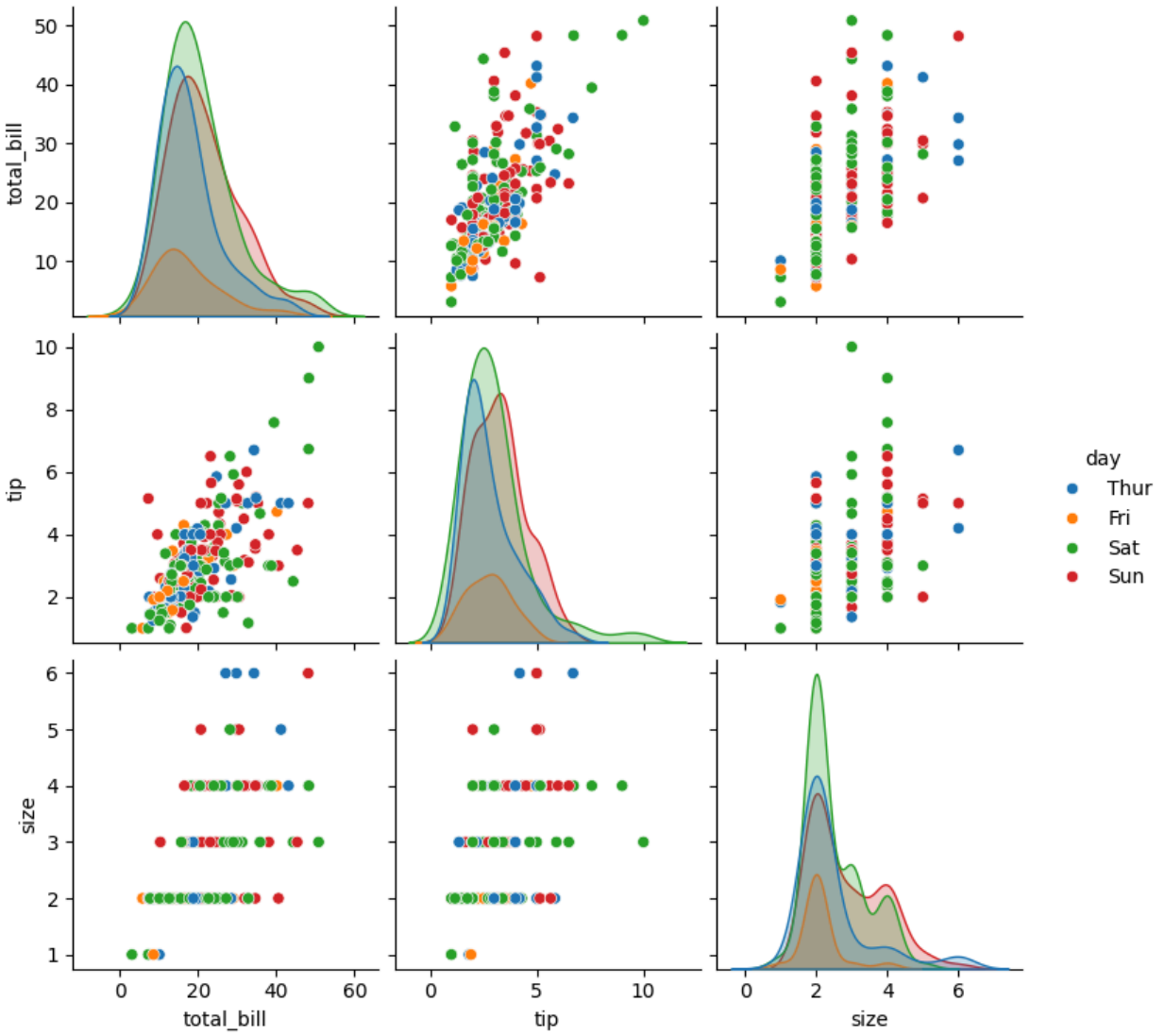

total_bill
tip
50
40
30
20
10
2
4
6
8
10

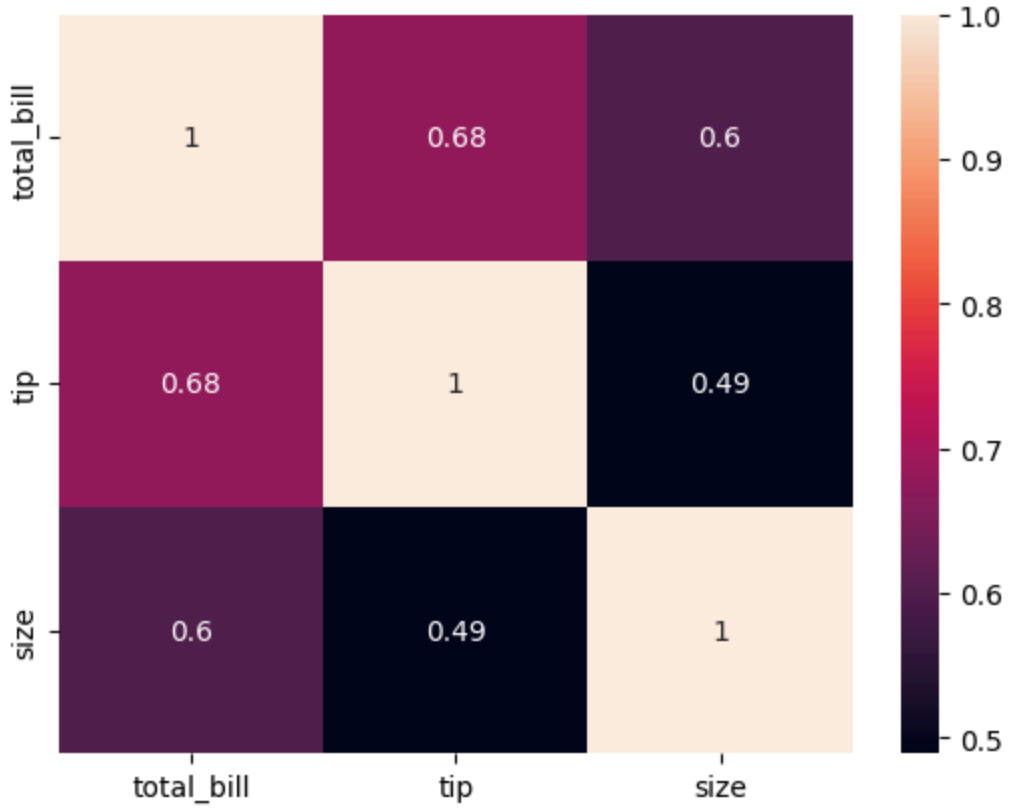

50
40
30
20
10
total_bill
2
4
6
8
10
tip

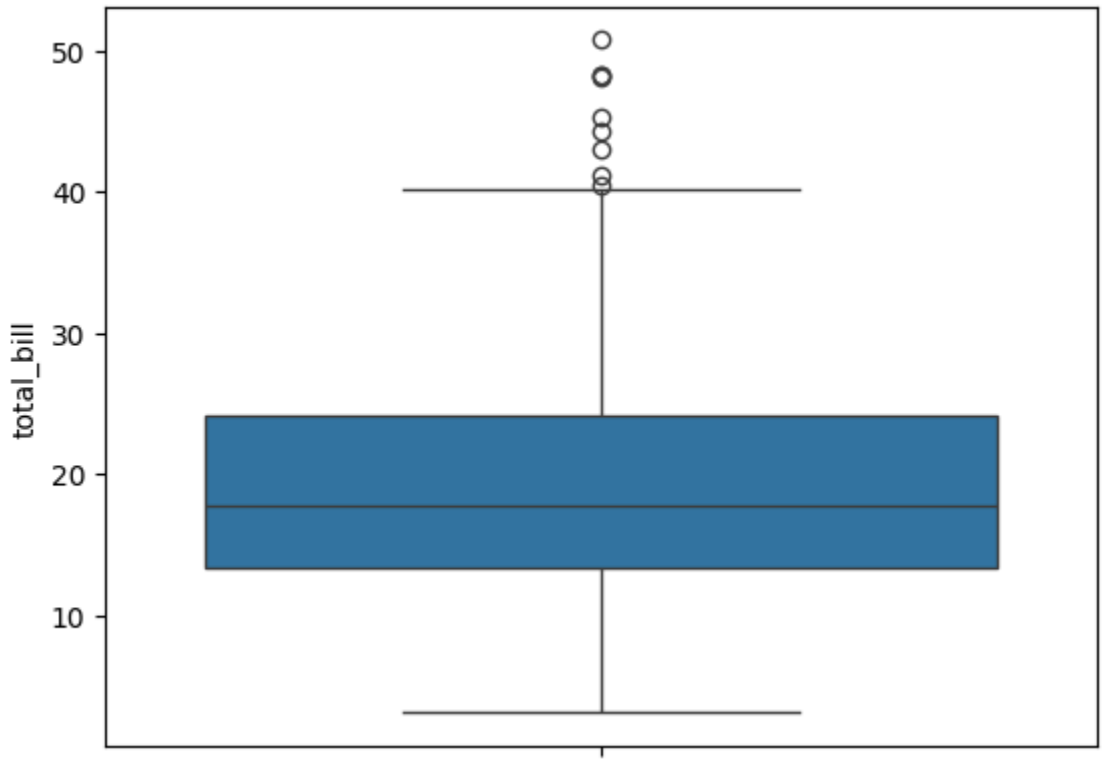


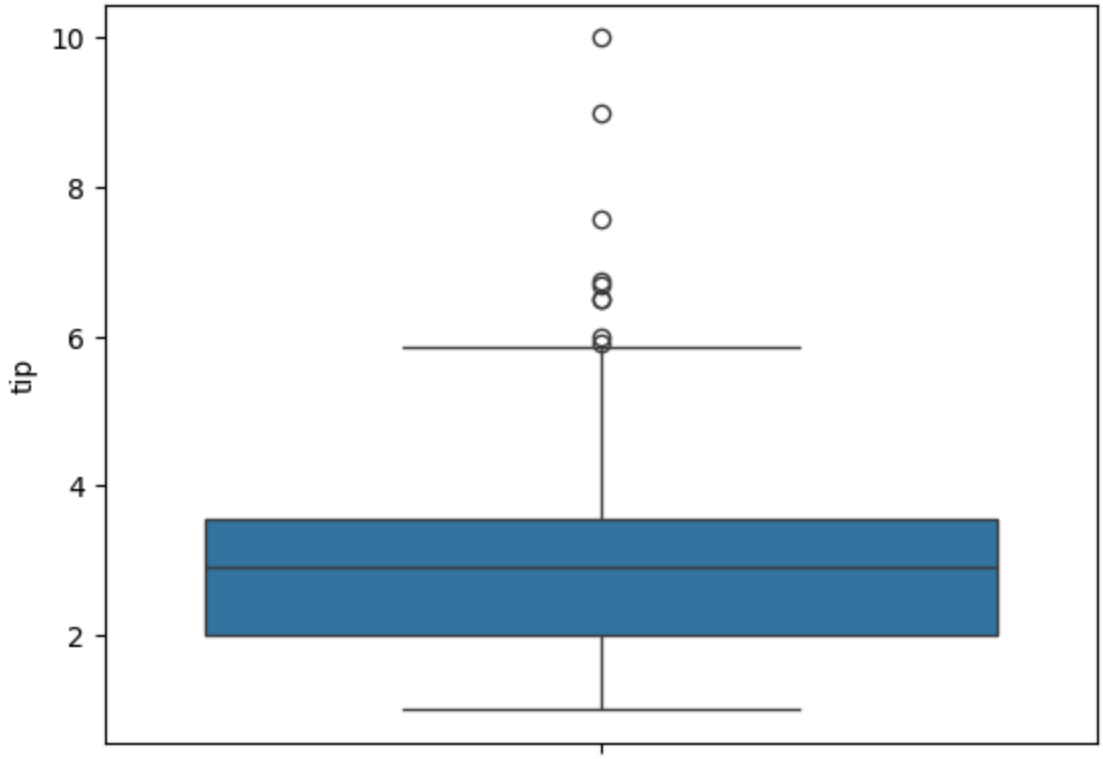
time
Dinner 176
Lunch 68
Name: count, dtype: int64

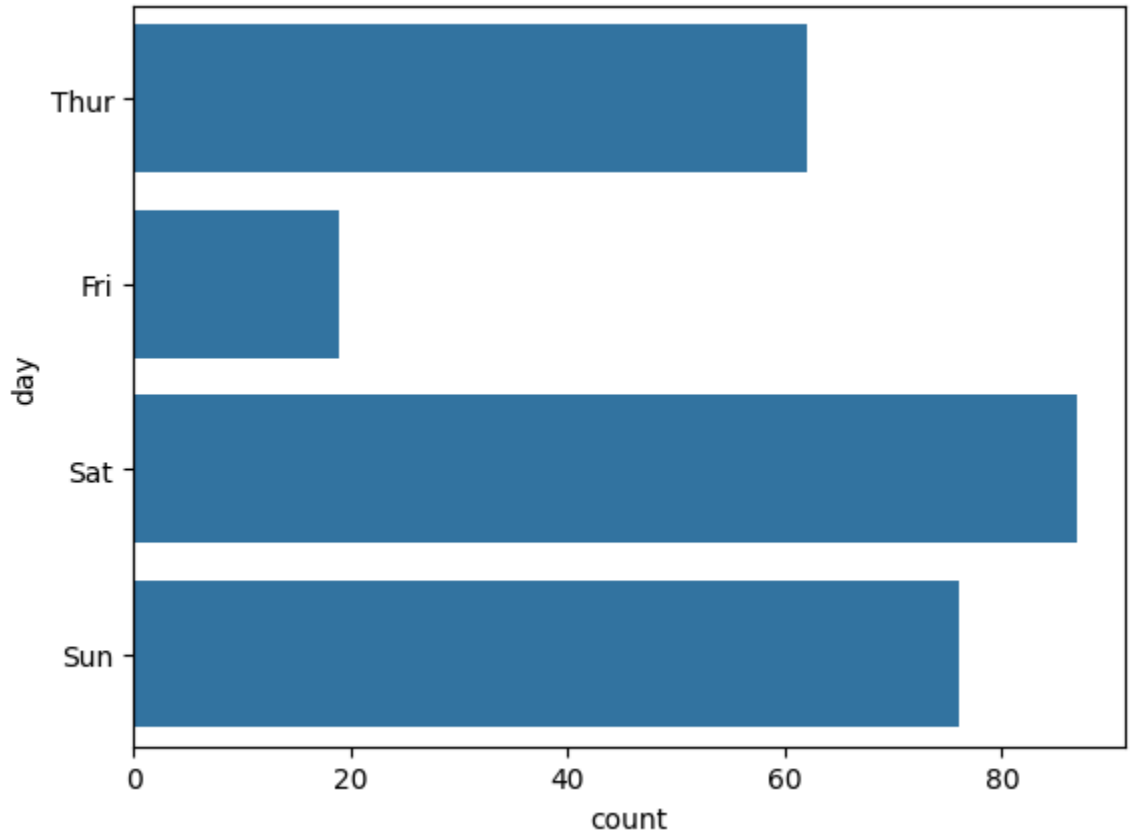

total_bill
tip
size
time
Lunch
Dinner

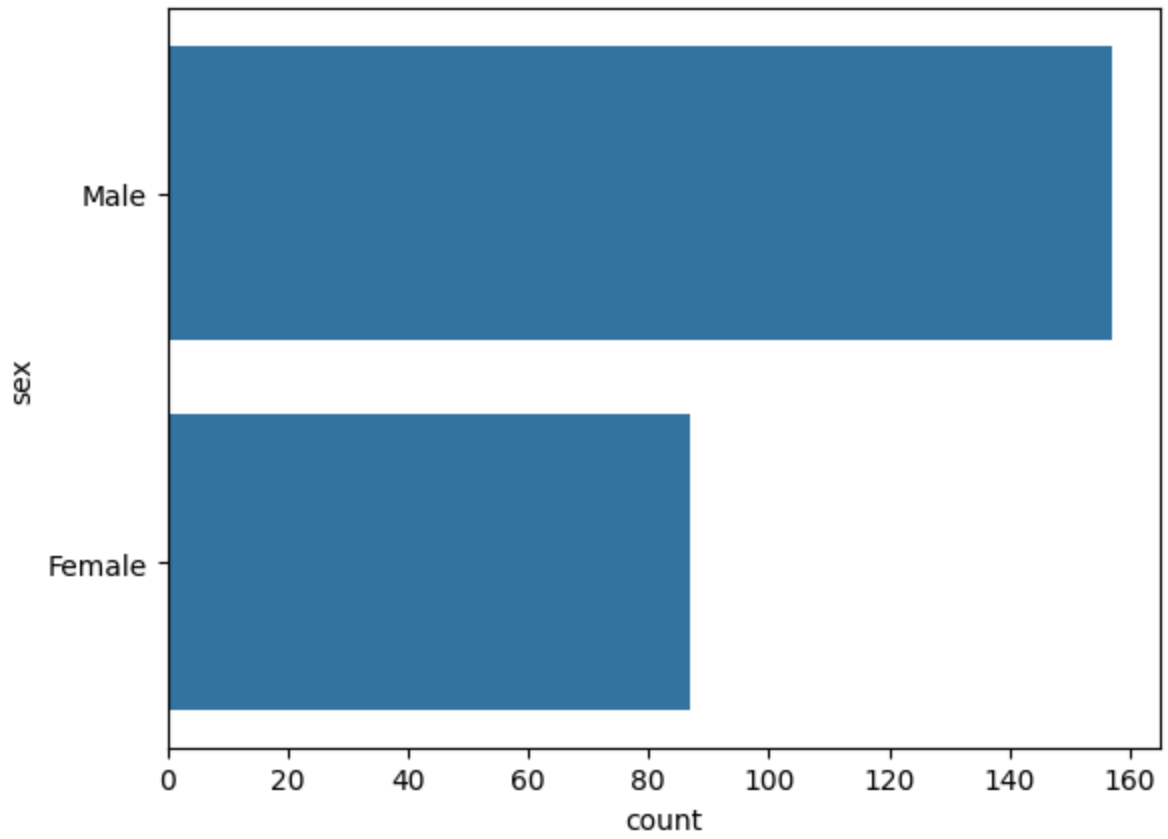

total_bill
tip
size
day
Thur
Fri
Sat
Sun

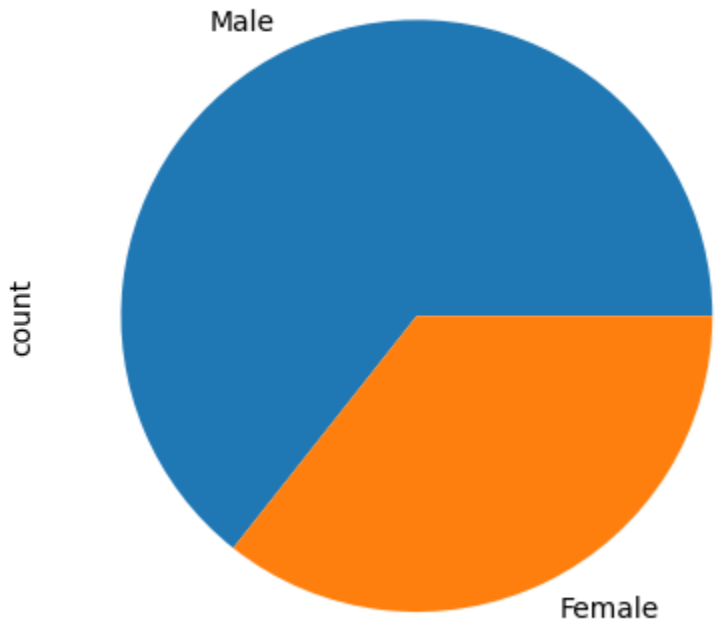

total_bill
tip
size
1
0.68
0.6
0.68
1
0.49
0.6
0.49
1
total_bill
tip
size
1.0
0.9
0.8
0.7
0.6
0.5

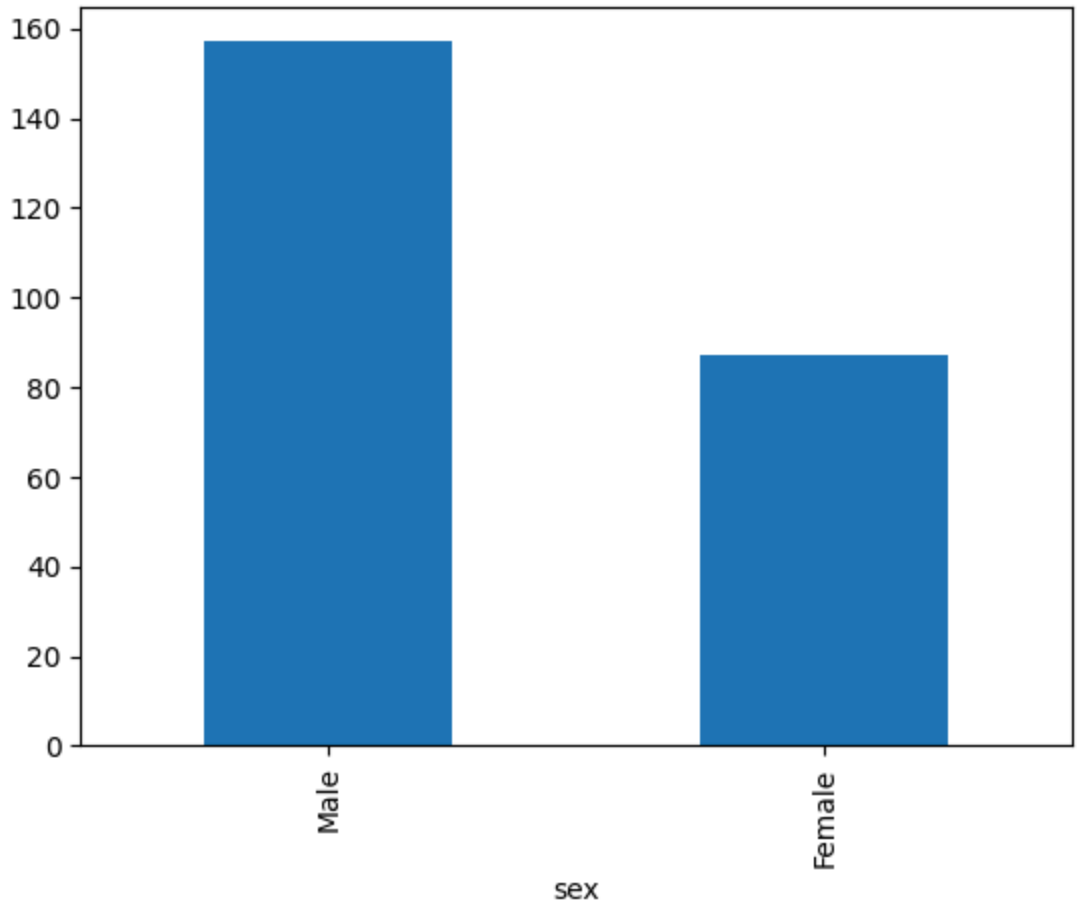

total_bill
50
40
30
20
10

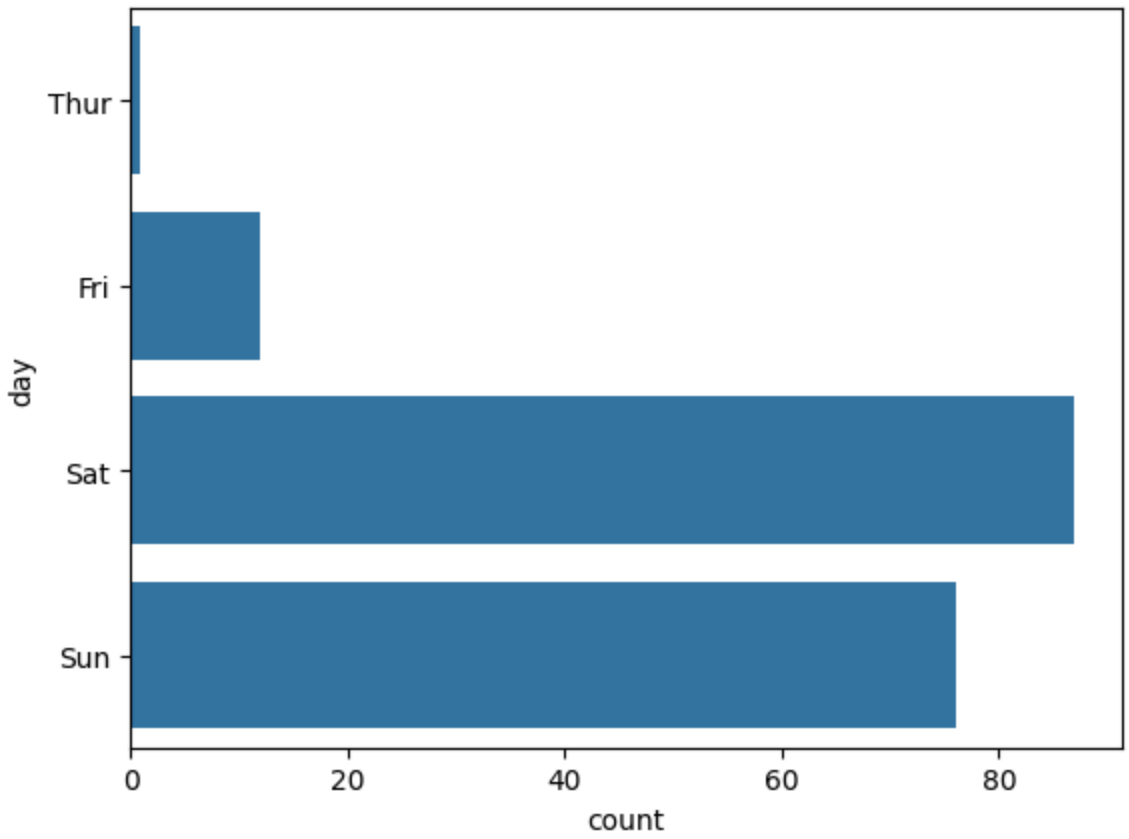

10
8
6
4
2
tip


Thur
Fri
Sat
Sun
day
0
20
40
60
80
count


Male
Female
sex
0
20
40
60
80
100
120
140
160
count


Male
Female
count


160
140
120
100
80
60
40
20
0
Male
Female
sex



RESULT:

Thus, Exploratory Data Analysis (EDA) was successfully performed on the Seaborn `tips` dataset using quantitative and qualitative analysis. Quantitative analysis was carried out using distribution plots, joint plots, pair plots, box plots, and a correlation heatmap. Qualitative analysis was performed using count plots, bar charts, and pie charts to study categorical variables such as `sex`, `day`, and `time`. The dataset was analyzed to understand distributions, relationships, and patterns in the data.

Experiment No:17
Experiment to understand Linear Regression For a given data set.

AIM:
To understand Linear Regression for a given data set.

DESCRIPTION:
Understand the Linear regression for the dataset given.

CODE:

```
import numpy as np
import pandas as pd

df=pd.read_csv('/content/student_data.csv')
df
df.info()
df.dropna(inplace=True)
df.info()
df.describe()

features=df.iloc[:,[2]].values
label=df.iloc[:,[32]].values

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,te
st_size=0.2,random_state=42)

from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
model.score(x_train,y_train)

model.score(x_test,y_test)
```

```
model.coef_

model.intercept_

import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
model=pickle.load(open('SalaryPred.model','rb'))

age=float(input("Enter Student Age: "))
age_NP=np.array([[age]])
grade=model.predict(age_NP)

print("Predicted final grade for age {} is
{}".format(age,grade))
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    object
 1   sex         395 non-null    object
 2   age         395 non-null    int64
 3   address     395 non-null    object
 4   famsize     395 non-null    object
 5   Pstatus     395 non-null    object
 6   Medu        395 non-null    int64
 7   Fedu        395 non-null    int64
 8   Mjob        395 non-null    object
 9   Fjob        395 non-null    object
 10  reason      395 non-null    object
 11  guardian    395 non-null    object
 12  traveltime  395 non-null    int64
 13  studytime   395 non-null    int64
```

```
 14  failures    395 non-null    int64
 15  schoolsup   395 non-null    object
 16  famsup      395 non-null    object
 17  paid        395 non-null    object
 18  activities  395 non-null    object
 19  nursery     395 non-null    object
 20  higher      395 non-null    object
 21  internet    395 non-null    object
 22  romantic    395 non-null    object
 23  famrel      395 non-null    int64
 24  freetime    395 non-null    int64
 25  goout       395 non-null    int64
 26  Dalc        395 non-null    int64
 27  Walc        395 non-null    int64
 28  health      395 non-null    int64
 29  absences    395 non-null    int64
 30  G1          395 non-null    int64
 31  G2          395 non-null    int64
 32  G3          395 non-null    int64
dtypes: int64(16), object(17)
memory usage: 102.0+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    object
 1   sex         395 non-null    object
 2   age         395 non-null    int64
 3   address     395 non-null    object
 4   famsize     395 non-null    object
 5   Pstatus     395 non-null    object
 6   Medu        395 non-null    int64
 7   Fedu        395 non-null    int64
 8   Mjob        395 non-null    object
 9   Fjob        395 non-null    object
 10  reason      395 non-null    object
 11  guardian    395 non-null    object
 12  traveltime  395 non-null    int64
 13  studytime   395 non-null    int64
```

```
 14  failures    395 non-null    int64
 15  schoolsup   395 non-null    object
 16  famsup      395 non-null    object
 17  paid        395 non-null    object
 18  activities  395 non-null    object
 19  nursery     395 non-null    object
 20  higher      395 non-null    object
 21  internet    395 non-null    object
 22  romantic    395 non-null    object
 23  famrel      395 non-null    int64
 24  freetime    395 non-null    int64
 25  goout       395 non-null    int64
 26  Dalc        395 non-null    int64
 27  Walc        395 non-null    int64
 28  health      395 non-null    int64
 29  absences    395 non-null    int64
 30  G1          395 non-null    int64
 31  G2          395 non-null    int64
 32  G3          395 non-null    int64
dtypes: int64(16), object(17)
memory usage: 102.0+ KB
Enter Student Age: 44
Predicted final grade for age 44.0 is [[-4.07873452]]
```

RESULT:

Thus, Linear Regression was successfully implemented on the student dataset to predict the final grade (`G3`) based on the student's age. The model was trained and tested using the `train_test_split()` method, and its performance was evaluated using the score function. The trained model was saved and loaded using Pickle, and the predicted final grade was obtained for the given age.

Experiment No:18
Experiment to understand KNN algorithm for a given data set

AIM:
To understand KNN algorithm for a given dataset

DESCRIPTION:
Understand the KNN algorithm for the dataset given.

CODE:

```
import numpy as np
import pandas as pd
df=pd.read_csv('/content/Iris.csv')
df.info()
df.Species.value_counts()
df.head()
features=df.iloc[:,1:-1].values
label=df.iloc[:,5].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain,xtest,ytrain,ytest=train_test_split(features,label,test
_size=.2,random_state=42)
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
from sklearn.metrics import confusion_matrix
print(confusion_matrix(ytest,model_KNN.predict(xtest)))
from sklearn.metrics import classification_report
print(classification_report(ytest,model_KNN.predict(xtest)))
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
0.9666666666666667
1.0
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
                precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30
```

RESULT:

Thus, the K-Nearest Neighbors (KNN) algorithm was successfully implemented on the Iris dataset to classify iris flowers into three species: Iris-setosa, Iris-versicolor, and Iris-virginica. The model was trained using 80% of the dataset and tested using the remaining 20%. It achieved 100% test accuracy, and the confusion matrix and classification report were generated successfully.

Experiment No:19
Experiment to understand Logistic Regression for a given data set

AIM:
To understand Logistic Regression for a given data set.

DESCRIPTION:
Understand the Logistic Regression algorithm for the dataset given.

CODE:

```
import numpy as np
import pandas as pd

df=pd.read_csv('/content/Breast_Cancer_Logistic_Regression.csv')

df.head()

features=df.iloc[:,:-1].values
label=df.iloc[:,-1].values

features
label

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

for i in range(1,401):

x_train,x_test,y_train,y_test=train_test_split(features,label,te
st_size=0.2,random_state=i)

    model=LogisticRegression(max_iter=10000)
```

```
    model.fit(x_train,y_train)

    train_score=model.score(x_train,y_train)
    test_score=model.score(x_test,y_test)

    if test_score>train_score:
        print("Test {} Train {} Random State
{}".format(test_score,train_score,i))

x_train,x_test,y_train,y_test=train_test_split(features,label,te
st_size=0.2,random_state=42)

finalModel=LogisticRegression(max_iter=10000)
finalModel.fit(x_train,y_train)

from sklearn.metrics import classification_report

print(classification_report(y_test,finalModel.predict(x_test)))
```

OUTPUT:

```
Test 0.9649122807017544 Train 0.9538461538461539 Random State 5
Test 0.9649122807017544 Train 0.9560439560439561 Random State 8
Test 0.9649122807017544 Train 0.9582417582417583 Random State 14
Test 0.956140350877193 Train 0.9560439560439561 Random State 22
Test 0.9649122807017544 Train 0.9560439560439561 Random State 23
Test 0.9736842105263158 Train 0.9626373626373627 Random State 26
Test 0.956140350877193 Train 0.9560439560439561 Random State 27
Test 0.9649122807017544 Train 0.9538461538461539 Random State 34
Test 0.9649122807017544 Train 0.9648351648351648 Random State 35
Test 0.9649122807017544 Train 0.9560439560439561 Random State 36
Test 0.9736842105263158 Train 0.9582417582417583 Random State 38
Test 0.9912280701754386 Train 0.9516483516483516 Random State 39
Test 0.9736842105263158 Train 0.9538461538461539 Random State 40
Test 0.9912280701754386 Train 0.9516483516483516 Random State 41
Test 0.9824561403508771 Train 0.9560439560439561 Random State 43
Test 0.9736842105263158 Train 0.9604395604395605 Random State 44
Test 0.9736842105263158 Train 0.9604395604395605 Random State 50
```

Test 0.9736842105263158 Train 0.9626373626373627 Random State 52
Test 0.9736842105263158 Train 0.9582417582417583 Random State 53
Test 0.9736842105263158 Train 0.9582417582417583 Random State 54
Test 0.9649122807017544 Train 0.9582417582417583 Random State 57
Test 0.9824561403508771 Train 0.9560439560439561 Random State 58
Test 0.9649122807017544 Train 0.9604395604395605 Random State 60
Test 0.9649122807017544 Train 0.9538461538461539 Random State 63
Test 0.9736842105263158 Train 0.9538461538461539 Random State 65
Test 0.9649122807017544 Train 0.9582417582417583 Random State 66
Test 0.9736842105263158 Train 0.9560439560439561 Random State 71
Test 0.9824561403508771 Train 0.9648351648351648 Random State 75
Test 0.9649122807017544 Train 0.9494505494505494 Random State 82
Test 0.9736842105263158 Train 0.9582417582417583 Random State 84
Test 0.956140350877193 Train 0.9538461538461539 Random State 88
Test 0.956140350877193 Train 0.9560439560439561 Random State 89
Test 0.9736842105263158 Train 0.9538461538461539 Random State 91
Test 0.9649122807017544 Train 0.9560439560439561 Random State 92
Test 0.9649122807017544 Train 0.9582417582417583 Random State 93
Test 0.9824561403508771 Train 0.9494505494505494 Random State 94
Test 0.956140350877193 Train 0.9560439560439561 Random State 98
Test 0.9649122807017544 Train 0.9560439560439561 Random State 99
Test 0.956140350877193 Train 0.9538461538461539 Random State 100
Test 0.9649122807017544 Train 0.9626373626373627 Random State 101
Test 0.9649122807017544 Train 0.9538461538461539 Random State 105
Test 0.9649122807017544 Train 0.9626373626373627 Random State 109
Test 0.9736842105263158 Train 0.9626373626373627 Random State 110
Test 0.9736842105263158 Train 0.9560439560439561 Random State 114
Test 0.956140350877193 Train 0.9538461538461539 Random State 117
Test 0.9736842105263158 Train 0.9560439560439561 Random State 118
Test 0.9912280701754386 Train 0.9516483516483516 Random State 121
Test 0.9824561403508771 Train 0.9560439560439561 Random State 123
Test 0.9736842105263158 Train 0.9604395604395605 Random State 125
Test 0.956140350877193 Train 0.9560439560439561 Random State 128
Test 0.9824561403508771 Train 0.9494505494505494 Random State 131
Test 0.956140350877193 Train 0.9560439560439561 Random State 132
Test 0.9649122807017544 Train 0.9582417582417583 Random State 135
Test 0.9649122807017544 Train 0.9516483516483516 Random State 138
Test 0.9649122807017544 Train 0.9604395604395605 Random State 140
Test 0.9736842105263158 Train 0.9582417582417583 Random State 143
Test 1.0 Train 0.9516483516483516 Random State 144
Test 0.9736842105263158 Train 0.9692307692307692 Random State 146
Test 0.9736842105263158 Train 0.9538461538461539 Random State 150
Test 0.9824561403508771 Train 0.9494505494505494 Random State 152
Test 0.9649122807017544 Train 0.9582417582417583 Random State 156

Test 0.9649122807017544 Train 0.9626373626373627 Random State 157
Test 0.956140350877193 Train 0.9516483516483516 Random State 158
Test 0.9736842105263158 Train 0.9604395604395605 Random State 160
Test 0.9649122807017544 Train 0.9516483516483516 Random State 164
Test 0.956140350877193 Train 0.9516483516483516 Random State 168
Test 0.956140350877193 Train 0.9560439560439561 Random State 170
Test 0.9649122807017544 Train 0.9582417582417583 Random State 179
Test 0.9649122807017544 Train 0.9560439560439561 Random State 180
Test 0.956140350877193 Train 0.9516483516483516 Random State 184
Test 0.9649122807017544 Train 0.9560439560439561 Random State 188
Test 0.9736842105263158 Train 0.9516483516483516 Random State 191
Test 0.956140350877193 Train 0.9560439560439561 Random State 196
Test 0.9649122807017544 Train 0.9648351648351648 Random State 197
Test 0.9649122807017544 Train 0.9516483516483516 Random State 204
Test 0.9649122807017544 Train 0.9582417582417583 Random State 207
Test 0.9649122807017544 Train 0.9560439560439561 Random State 211
Test 0.9736842105263158 Train 0.9582417582417583 Random State 212
Test 0.9736842105263158 Train 0.9560439560439561 Random State 213
Test 0.9649122807017544 Train 0.9626373626373627 Random State 214
Test 0.9824561403508771 Train 0.9538461538461539 Random State 215
Test 0.9824561403508771 Train 0.9626373626373627 Random State 216
Test 0.9824561403508771 Train 0.9582417582417583 Random State 218
Test 0.9736842105263158 Train 0.9560439560439561 Random State 222
Test 0.9649122807017544 Train 0.9560439560439561 Random State 224
Test 0.9649122807017544 Train 0.9626373626373627 Random State 225
Test 0.9649122807017544 Train 0.9538461538461539 Random State 231
Test 0.9824561403508771 Train 0.9560439560439561 Random State 235
Test 0.9824561403508771 Train 0.9538461538461539 Random State 240
Test 0.9912280701754386 Train 0.9538461538461539 Random State 241
Test 0.9649122807017544 Train 0.9582417582417583 Random State 248
Test 0.9649122807017544 Train 0.9538461538461539 Random State 249
Test 0.956140350877193 Train 0.9560439560439561 Random State 252
Test 0.9736842105263158 Train 0.9538461538461539 Random State 253
Test 0.9649122807017544 Train 0.9516483516483516 Random State 254
Test 0.956140350877193 Train 0.9538461538461539 Random State 257
Test 0.9736842105263158 Train 0.9472527472527472 Random State 258
Test 0.9736842105263158 Train 0.967032967032967 Random State 259
Test 0.9736842105263158 Train 0.9516483516483516 Random State 260
Test 0.9824561403508771 Train 0.9560439560439561 Random State 265
Test 0.9649122807017544 Train 0.9560439560439561 Random State 266
Test 0.9649122807017544 Train 0.9648351648351648 Random State 267
Test 0.9736842105263158 Train 0.9516483516483516 Random State 268
Test 0.9649122807017544 Train 0.9648351648351648 Random State 269
Test 0.9824561403508771 Train 0.9494505494505494 Random State 273

Test 0.956140350877193 Train 0.9560439560439561 Random State 275
Test 0.9824561403508771 Train 0.945054945054945 Random State 279
Test 0.9649122807017544 Train 0.9604395604395605 Random State 280
Test 0.9649122807017544 Train 0.9538461538461539 Random State 283
Test 0.956140350877193 Train 0.9560439560439561 Random State 285
Test 0.9649122807017544 Train 0.9538461538461539 Random State 286
Test 0.9736842105263158 Train 0.9538461538461539 Random State 287
Test 0.9649122807017544 Train 0.9582417582417583 Random State 288
Test 0.9824561403508771 Train 0.9538461538461539 Random State 292
Test 0.9736842105263158 Train 0.9538461538461539 Random State 294
Test 0.9824561403508771 Train 0.9560439560439561 Random State 297
Test 0.9649122807017544 Train 0.9582417582417583 Random State 299
Test 0.9736842105263158 Train 0.9560439560439561 Random State 302
Test 0.956140350877193 Train 0.9560439560439561 Random State 306
Test 0.956140350877193 Train 0.9516483516483516 Random State 308
Test 0.9649122807017544 Train 0.9604395604395605 Random State 309
Test 0.9736842105263158 Train 0.9604395604395605 Random State 310
Test 0.9736842105263158 Train 0.9582417582417583 Random State 313
Test 0.9912280701754386 Train 0.9516483516483516 Random State 314
Test 0.9736842105263158 Train 0.9560439560439561 Random State 316
Test 0.956140350877193 Train 0.9560439560439561 Random State 317
Test 0.9649122807017544 Train 0.9582417582417583 Random State 318
Test 0.9649122807017544 Train 0.9560439560439561 Random State 323
Test 0.9649122807017544 Train 0.9604395604395605 Random State 333
Test 0.956140350877193 Train 0.9560439560439561 Random State 334
Test 0.956140350877193 Train 0.9560439560439561 Random State 340
Test 0.9649122807017544 Train 0.9494505494505494 Random State 341
Test 0.9824561403508771 Train 0.9516483516483516 Random State 342
Test 0.9736842105263158 Train 0.9560439560439561 Random State 346
Test 0.9649122807017544 Train 0.9582417582417583 Random State 347
Test 0.9649122807017544 Train 0.9560439560439561 Random State 350
Test 0.9736842105263158 Train 0.9560439560439561 Random State 351
Test 0.956140350877193 Train 0.9560439560439561 Random State 352
Test 0.9912280701754386 Train 0.9494505494505494 Random State 353
Test 0.9736842105263158 Train 0.9626373626373627 Random State 356
Test 0.9649122807017544 Train 0.9626373626373627 Random State 361
Test 0.956140350877193 Train 0.9494505494505494 Random State 366
Test 0.9736842105263158 Train 0.9582417582417583 Random State 367
Test 0.9824561403508771 Train 0.9472527472527472 Random State 370
Test 0.9824561403508771 Train 0.9516483516483516 Random State 374
Test 0.9824561403508771 Train 0.9560439560439561 Random State 375
Test 0.9649122807017544 Train 0.9604395604395605 Random State 376
Test 0.9649122807017544 Train 0.9516483516483516 Random State 378
Test 0.9649122807017544 Train 0.9604395604395605 Random State 379

```
Test 0.9649122807017544 Train 0.9604395604395605 Random State 380
Test 0.9649122807017544 Train 0.9494505494505494 Random State 384
Test 0.9649122807017544 Train 0.9604395604395605 Random State 389
Test 0.9649122807017544 Train 0.9604395604395605 Random State 390
Test 0.9649122807017544 Train 0.9626373626373627 Random State 391
Test 0.9649122807017544 Train 0.9516483516483516 Random State 397
Test 0.9824561403508771 Train 0.9604395604395605 Random State 398
              precision    recall  f1-score   support

           0       0.97      0.91      0.94        43
           1       0.95      0.99      0.97        71

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```

RESULT:

Thus, Logistic Regression was successfully implemented on the Breast Cancer dataset to classify the data into two categories. The dataset was divided into training and testing sets, and the model was trained using `LogisticRegression`. Its performance was evaluated using a classification report, achieving 96% accuracy on the test dataset.

Experiment No:20
Experiment to understand K-means clustering algorithm for a given data set.

AIM:
To understand K-means clustering algorithm for a given data set.

DESCRIPTION:
Understand the K-means clustering algorithm for the dataset given.

CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

df=pd.read_csv('/content/Mall_Customers.csv')

df.info()
df.head()

sns.pairplot(df)

features=df.iloc[:,[3,4]].values

from sklearn.cluster import KMeans

model=KMeans(n_clusters=5,random_state=42,n_init=10)

model.fit(features)

Final=df.iloc[:,[3,4]].copy()
Final['label']=model.predict(features)
```

```
Final.head()

sns.set_style("whitegrid")

g=sns.FacetGrid(Final,hue="label",height=8)
g.map(plt.scatter,"Annual Income (k$)","Spending Score (1-100)")
g.add_legend()

plt.show()

features_el=df.iloc[:,[2,3,4]].values

wcss=[]

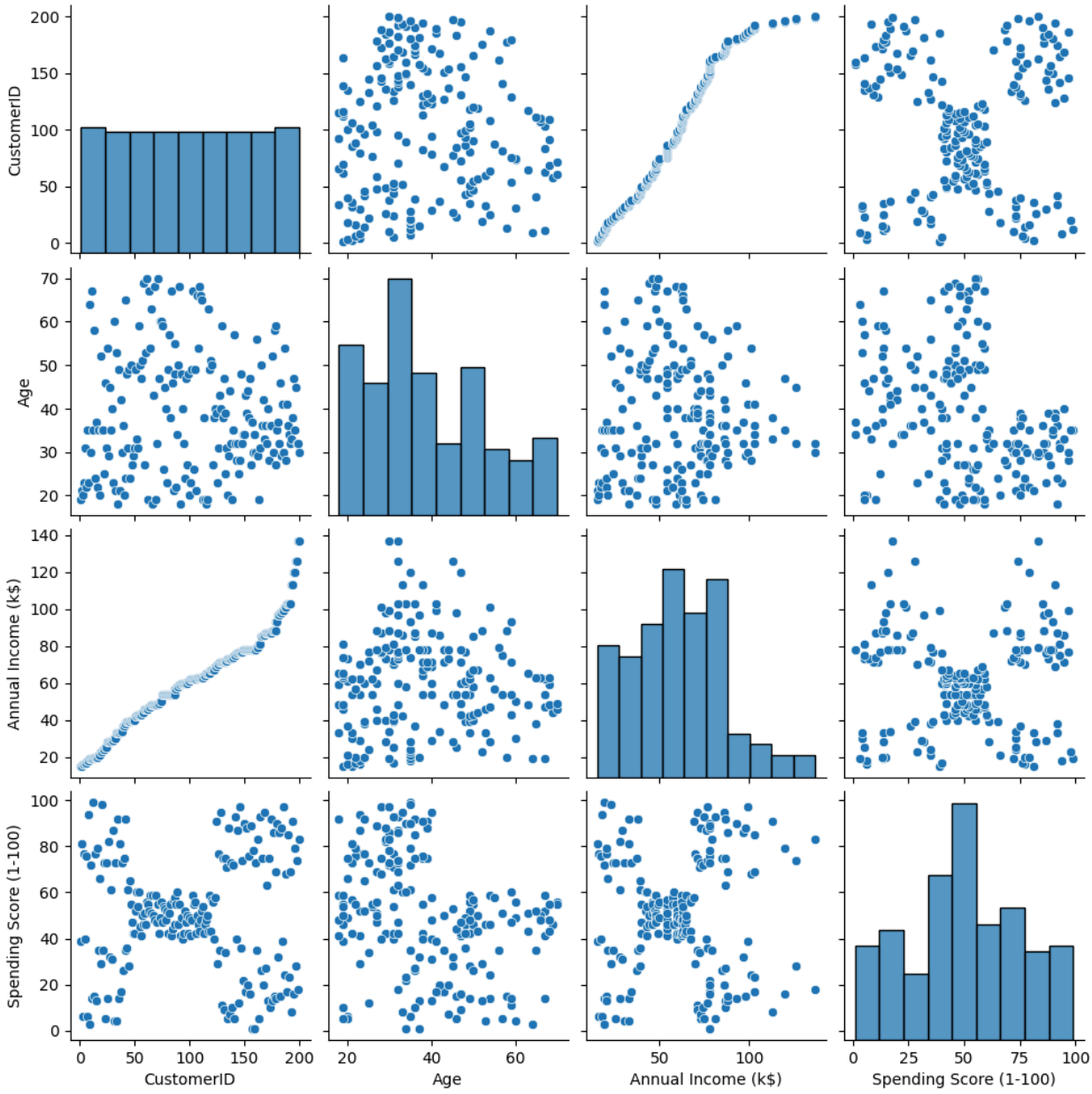
for i in range(1,10):
    model=KMeans(n_clusters=i,random_state=42,n_init=10)
    model.fit(features_el)
    wcss.append(model.inertia_)

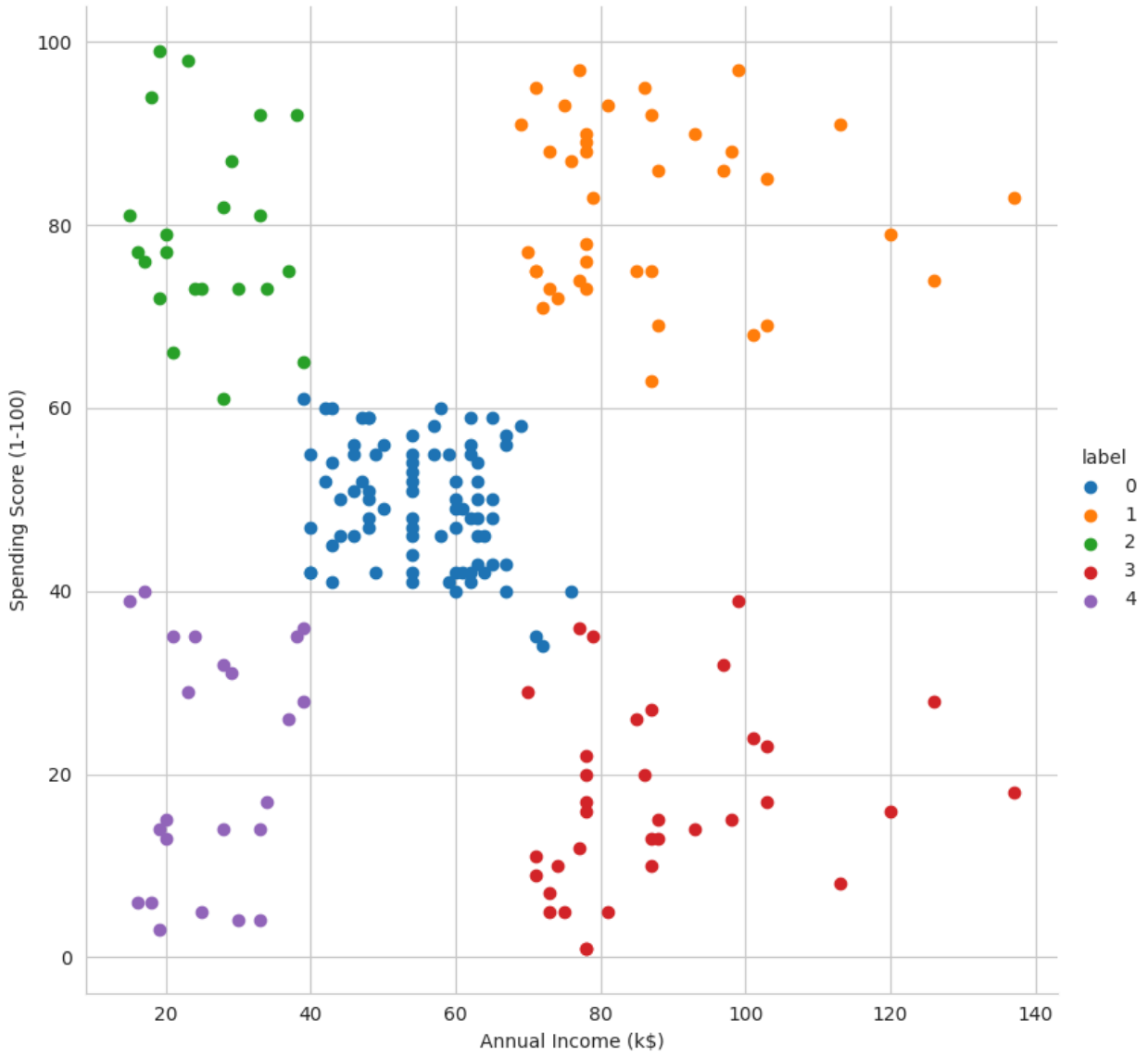
plt.plot(range(1,10),wcss,marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
```

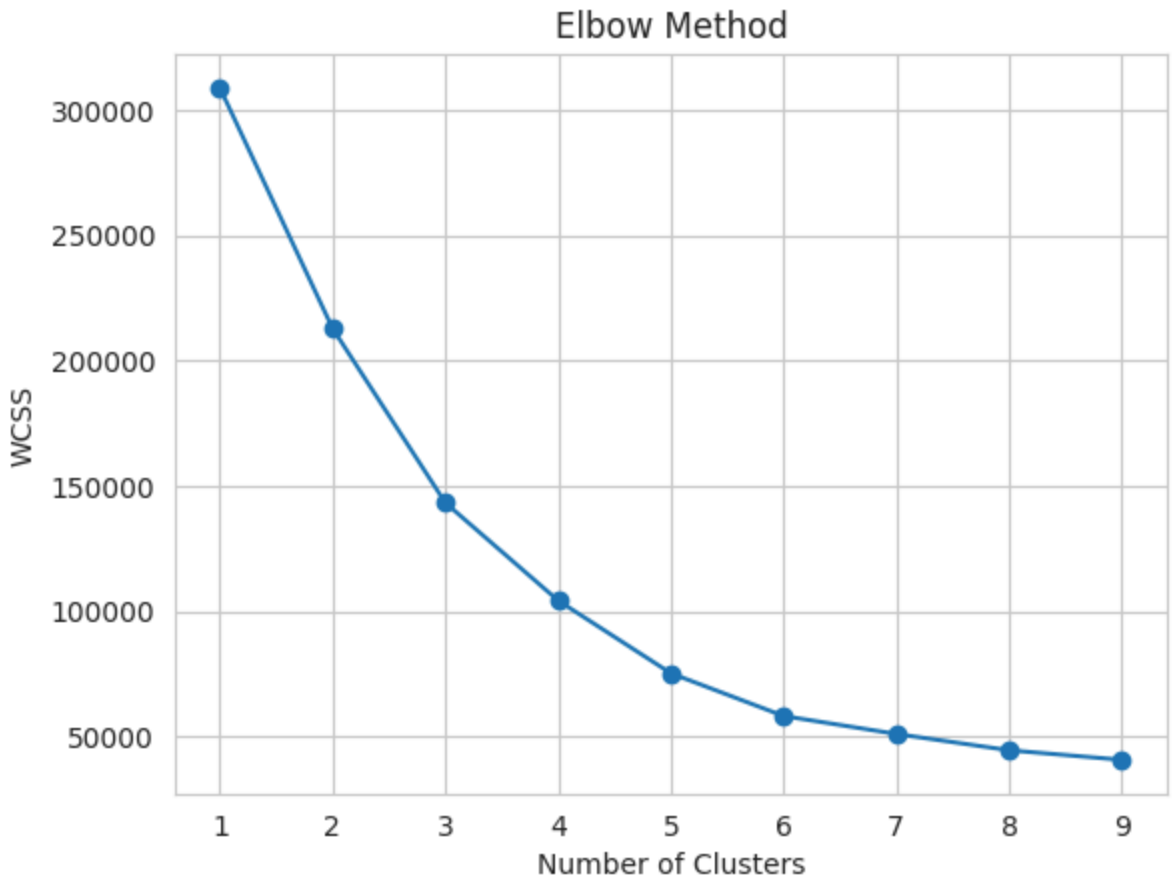
OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
```

```
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```




Spending Score (1-100)
Annual Income (k$)
label
0
1
2
3
4
0
20
40
60
80
100
20
40
60
80
100
120
140



RESULT:

Thus, the K-means clustering algorithm was successfully implemented on the Mall Customers dataset. The customers were grouped into five clusters based on their annual income and spending scores. The clusters were visualized using a scatter plot, and the optimal number of clusters was analyzed using the Elbow Method with the Within-Cluster Sum of Squares (WCSS) graph.